

华中科技大学

2026

离散数学（一） 课程实验报告

专 业：	计算机类
班 级：	2501 班
学 号：	U202514699
姓 名：	彭婉茹
完成日期：	2026 年 7 月 10 日



计算机科学与技术学院

目 录

1	数理逻辑	1
1.1	实验目的	1
1.2	实验内容	1
1.2.1	推理与证明	1
1.2.2	命题逻辑	2
1.2.3	逻辑与 AI 表示	3
1.3	实验过程	3
1.3.1	推理与证明	3
1.3.2	命题逻辑	11
1.3.3	逻辑与 AI 表示	14
2	集合、关系及函数	18
2.1	实验内容	18
2.2	实验内容	18
2.2.1	集合的表示和基本运算	18
2.2.2	关系表示及其运算	18
2.2.3	函数	19
2.3	实验过程	19
2.3.1	集合的表示和基本运算	19
2.3.2	关系表示及其运算	23
2.3.3	函数	36
3	初等数论及组合数学	44
3.1	实验目的	44
3.2	实验内容	44
3.2.1	初等数论	44
3.2.2	组合数学	45
3.3	实验过程	47
3.3.1	初等数论	47
3.3.2	组合数学	63
4	图论	82
4.1	实验目的	82
4.2	实验内容	82
4.2.1	图论	82
4.3	实验过程	83
4.3.1	图论	83
5	总结与课程建议	103
5.1	课程总结	103

5.2	问题及建议	104
-----	-------------	-----

1 数理逻辑

1.1 实验目的

在本实验旨在掌握命题逻辑和谓词逻辑的基本理论，理解常见推理规则、逻辑等价及归结原理等知识。通过编程实现逻辑表达式的验证、真值表生成以及自然语言到谓词逻辑的符号化表示，加深对形式化推理方法的理解，提高利用程序解决逻辑问题的能力，为后续离散数学知识学习奠定基础。

1.2 实验内容

1.2.1 推理与证明

(1) 构造二难推理的有效性验证：

任务描述：在逻辑学中，存在许多被证明是永远有效的推理形式，它们可以作为我们进行正确推理的“模板”。构造性二难（Constructive Dilemma）就是其中一种常见的有效推理形式。此实验内容是使用 sympy 库编程来验证构造性二难推理的有效性。其结构如下：

前提 1: $P \Rightarrow Q$ (如果 P, 那么 Q)

前提 2: $R \Rightarrow S$ (如果 R, 那么 S)

前提 3: $P \vee R$ (P 或 R)

结论: $Q \vee S$ (因此, Q 或 S)

(2) 识别无效推理并寻找反例：

任务描述：在逻辑推理中，除了有效的推理形式，还存在许多看似正确但实际上是错误的推理形式，这些被称为逻辑谬误（Fallacy）。一个常见的逻辑谬误是“肯定后件”（Affirming the consequent）。

本关任务是：

1. 首先证明“肯定后件”这个推理形式是无效的。
2. 然后找出一个反例（即一组能使所有前提为真，但结论为假的赋值）来证明其无效性。

推理结构如下：

前提 1: $P \Rightarrow Q$ (如果 P, 那么 Q)

前提 2: Q (Q 成立)

结论: P (因此, P 成立)

(3) 归结原理的应用:

任务描述: 归结(Resolution)是自动定理证明(Automated Theorem Proving)领域中的一个核心推理规则, 尤其在基于逻辑的 AI 系统中被广泛使用。它允许计算机通过一个简单的规则来系统地推导出新的结论。

本关任务是验证归结原理的基本形式是有效的。其结构如下:

前提 1: $P \mid Q$

前提 2: $\sim P \mid R$

结论: $Q \mid R$

(4) 用等价演算进行证明:

任务描述: 本关任务是证明 $\sim(P \Rightarrow Q)$ 与 $P \& \sim Q$ 这两个命题公式是逻辑等价的。我们将不使用真值表, 而是使用 sympy 的符号计算能力来模拟等价演算的过程, 即通过一系列已知的等价规则(如德·摩根定律)进行逐步化简。

1.2.2 命题逻辑

(1) 三变量真值生成器

任务描述: 本编写一个程序, 输入一个包含 P、Q、R 三个命题变量的逻辑表达式字符串, 程序能自动生成并打印该表达式的完整真值表。

(2) 分配律的验证

任务描述: 本关任务: 逻辑等价中的分配律是化简表达式的重要工具。请使用 sympy 库编写一个程序, 验证以下两个分配律是否为重言式(永真式), 从而证明它们是成立的。

$$P \& (Q \mid R) \Leftrightarrow (P \& Q) \mid (P \& R)$$

$$P \mid (Q \& R) \Leftrightarrow (P \mid Q) \& (P \mid R)$$

1.2.3 逻辑与 AI 表示

(1) 全称规则

任务描述：假设你正在为一个人工智能构建知识库，它需要理解关于世界的一些普遍规则。本关任务是将以下两条自然语言规则，符号化为谓词逻辑表达式的字符串，以便存入机器人的知识库。

1. “所有的矩形都是四边形。”
2. “没有不遵守交通规则司机。”

(2) 存在事实

任务描述：除了需要理解普遍规则外，一个 AI 知识库也需要存储关于世界的具体事实。本关任务是将以下两条自然语言描述的事实，符号化为谓词逻辑表达式的字符串，以便 AI 能够理解和使用。

1. “有些机器人会编程。”
2. “并非所有金属都能导电。”（等价于“有些金属不能导电”）

1.3 实验过程

1.3.1 推理与证明

(1) 构造二难推理的有效性验证：

程序设计思想：

首先定义命题变量，使用 `symbols()` 创建四个命题变量：

```
P, Q, R, S_ = symbols('P Q R S')
```

然后构造前提与结论，程序中分别构造三个前提以及结论：

```
premise1 = P >> Q, premise2 = R >> S_, premise3 = P | R. //前提
conclusion = Q | S_      // 结论
```

随后将三个前提通过逻辑与连接：

```
premises_conj = premise1 & premise2 & premise3
```

为了验证推理是否有效，需要判断下面式子是否为真：

$$[(P \rightarrow Q) \wedge (R \rightarrow S) \wedge (P \vee R)] \rightarrow (Q \vee S)$$

对应程序实现为：

```
inference_expr = premises_conj >> conclusion
```

然后利用 `satisfiable()` 进行验证程序调用：

```
satisfiable(inference_expr)
```

判断该公式是否可满足。其核心思想为：

若推理式恒成立，则不存在使其为假的赋值；即不存在“前提全真而结论为假”的情况；因此该推理规则有效。程序最终根据返回结果判断：

```
if satisfiable(inference_expr):  
    is_valid = True  
else:  
    is_valid = False
```

并输出验证结果。

时间复杂度分析：因为本题目限定了变量数 $n = 4$ 是固定常数，实际运行时间是常数级 $O(1)$ 。

空间复杂度分析：整体空间复杂度 $O(n)$ ，其中限定了本题 $n = 4$ 命题变元的数量，所以空间复杂度仍然是常数级 $O(1)$ 。

彭婉茹
 49826

推理与证明
实验总用时: 00:08:35

更多操作

第1关: 构造性二难推... 100

学习内容 记录 评论

- 任务描述
- 相关知识
 - 推理的有效性
 - 在 `sympy` 中验证
- 编程要求
- 测试说明

任务描述

在逻辑学中, 存在许多被证明是永远有效的推理形式, 它们可以作为我们进行正确推理的“模板”。构造性二难 (Constructive Dilemma) 就是其中一种常见的有效推理形式。

本关任务是使用 `sympy` 库, 通过编程来验证构造性二难推理的有效性。其结构如下:

- 前提1: $P \gg Q$ (如果P, 那么Q)
- 前提2: $R \gg S$ (如果R, 那么S)
- 前提3: $P \mid R$ (P或R)
- 结论: $Q \mid S$ (因此, Q或S)

相关知识

为了完成本关任务, 你需要掌握: 1. 如何判断一个推理的有效性, 2. 如何在 `sympy` 中构建并验证推理公式。

代码文件

```

1 from sympy import symbols
2
3 try:
4     from sympy.logic.inference import
      satisfiable
5 except Exception:
6     try:
7         from sympy.logic.algorithms.dpll
          import dpll_satisfiable as satisfiable
8     except Exception:
9         satisfiable = None
10
11 def prove_constructive_dilemma():
12     """
13     验证构造性二难推理的有效性, 并按题面格式输
      出。
14     """
15
16     P, Q, R, S_ = symbols('P Q R S')
17
18     # 用于验证的 sympy 表达式
          
```

测试结果

1/1 全部通过

测试集1 消耗内存117.54MB 代码执行时长: 0.71秒

预期输出

实际输出

展示原始输出

前提: $P \gg Q, R \gg S, P \mid R$
 结论: $Q \mid S$
 构造性二难推理是否有效: True

前提: $P \gg Q, R \gg S, P \mid R$
 结论: $Q \mid S$
 构造性二难推理是否有效: True

本关最大执行时间: 20秒 本次评测耗时(编译)

下一关 自测运行

(2) 识别无效推理并寻找反例:

程序设计思路:

首先, 使用 `SymPy` 定义命题变元 P, Q :

```
P, Q = symbols('P Q')
```

然后构造前提与结论:

```
premise1 = P >> Q
premise2 = Q
conclusion = P
```

随后将两个前提进行逻辑与运算:

```
premises_conj = premise1 & premise2
```

得到整体前提：

$$(P \rightarrow Q) \wedge Q$$

为了判断推理是否有效，需要验证：

程序实现为：

```
inference_expr = premises_conj >> conclusion
```

程序通过：

```
inference_expr.equals(true)
```

判断该公式是否恒为真。

逻辑含义为：若公式恒为真，则推理有效；若公式不是永真式，则推理无效。

程序中：

```
if ~inference_expr.equals(true):  
    is_valid = False  
else:  
    is_valid = True
```

由于肯定后件并不是永真推理，因此最终：

```
is_valid = False
```

为了证明推理无效，需要找到：所有前提为真；但结论为假；的一个赋值。

程序构造：

```
counterexample_formula = premises_conj & ~conclusion
```

对应逻辑公式：

$$(P \rightarrow Q) \wedge Q \wedge \neg P$$

随后调用：

```
model = satisfiable(counterexample_formula)
```

寻找满足条件的模型。

时间复杂度分析：由于代码固定只有 2 个命题变元 P、Q 真值组合只有 4 种，算法最多检查 4 次就结束 表达式构建、等价判断、求解反例、输出打印都是固定次数操作，没有循环、没有递归、没有随数据规模增长的操作。所以时间复杂度是 $O(1)$ 。

空间复杂度分析：只创建2个符号变量+5个逻辑表达式所有临时变量、字符串、输出内容占用空间固定不变。所以空间复杂度为 $O(1)$ 。

彭婉茹
 49826

推理与证明

实验总用时: 00:34:35

更多操作

第2关: 识别无效推理... 100

学习内容

记录

评论

- 任务描述
- 相关知识
 - 证明推理无效
 - 什么是反例 (Counterexample)
 - 如何寻找反例
- 编程要求
- 测试说明

任务描述

在逻辑推理中, 除了有效的推理形式, 还存在许多看似正确但实际上是错误的推理形式, 这些被称为**逻辑谬误 (Fallacy)**。一个常见的逻辑谬误是“肯定后件” (Affirming the consequent)。

本关任务是:

- 首先证明“肯定后件”这个推理形式是**无效的**。
- 然后找出一个**反例** (即一组能使所有前提为真, 但结论为假的赋值) 来证明其无效性。

推理结构如下:

- 前提1: $P \gg Q$ (如果P, 那么Q)
- 前提2: Q (Q成立)
- 结论: P (因此, P成立)

代码文件

```

1 from sympy import symbols, true
2
3 try:
4     from sympy.logic.inference import
      satisfiable
5 except Exception:
6     try:
7         from sympy.logic.algorithms.dp11
          import dp11_satisfiable as satisfiable
8     except Exception:
9         satisfiable = None
10
11 def find_fallacy_counterexample():
12     """
13     证明“肯定后件”是无效推理, 并找到一个反例。
14     """
15     P, Q = symbols('P Q')
16
17     # 定义前提和结论 (用于计算)
18     premise1 = P >> Q
19     premise2 = Q
20     conclusion = P
21
22     # ----- START -----
23     # 1) 构造前提合取 premises_conj
24     premises_conj = premise1 & premise2
  
```

测试结果

1/1 全部通过

测试集1

消耗内存113.86MB 代码执行时长: 0.95秒

说点什么

本关最大执行时间: 20秒

本次评测耗时

上一关

下一关

自测运行

(3) 归结原理的应用

程序设计思路:

首先, 使用SymPy定义命题变元 P, Q, R :

```
P, Q, R = symbols('P Q R')
```

然后构造前提与结论:

```
premise1 = Or(P, Q)
premise2 = Or(Not(P), R)
conclusion = Or(Q, R)
```

随后将两个前提进行逻辑与运算:

```
premises_conj = premise1 & premise2
```

得到整体前提:

$$(P \vee Q) \wedge (\neg P \vee R)$$

为了判断推理是否有效，需要验证：是否不存在赋值使得前提为真而结论为假。

程序实现为构造反例公式：

```
counterexample_formula = premises_conj & ~conclusion
```

对应逻辑公式：

$$(P \vee Q) \wedge (\neg P \vee R) \wedge \neg(Q \vee R)$$

程序通过：

```
if ~satisfiable(counterexample_formula):
    is_valid = True
else:
    is_valid = False
```

判断该公式是否可满足。

逻辑含义为：若该公式不可满足，则说明不存在“前提为真、结论为假”的赋值，因此推理有效；若公式可满足，则推理无效。

程序中：由于归结原理是永真推理，因此最终：

```
is_valid = True
```

时间复杂度分析：由于代码固定只有 3 个命题变元 P 、 Q 、 R ，真值组合只有 8 种，算法最多检查 8 次就结束。表达式构建、可满足性判断、输出打印都是固定次数操作，没有循环、没有递归、没有随数据规模增长的操作。所以时间复杂度是 $O(1)$ 。

空间复杂度分析：只创建 3 个符号变量和 4 个逻辑表达式，所有临时变量、字符串、输出内容占用空间固定不变。所以空间复杂度为 $O(1)$ 。

彭婉茹
 49826

推理与证明

实验总用时: 00:15:12

更多操作

第3关: 归结原理的应用 100

学习内容

记录

评论

- 任务描述
- 相关知识
 - 归结原理
 - 验证方法
- 编程要求
- 测试说明

任务描述

归结 (Resolution) 是自动定理证明 (Automated Theorem Proving) 领域中的一个核心推理规则, 尤其在基于逻辑的 AI 系统中被广泛使用。它允许计算机通过一个简单的规则来系统地推导出新的结论。

本关任务是验证归结原理的基本形式是有效的。其结构如下:

- 前提1: $P \mid Q$
- 前提2: $\sim P \mid R$
- 结论: $Q \mid R$

相关知识

为了完成本关任务, 你需要掌握: 1. 什么是归结原理, 2. 如何应用上一关学到的知识来验证其有效性。

代码文件

```

24 conclusion = Or(Q, R)
25
26 # ----- START -----
27 premises_conj = premise1 & premise2
28 if ~satisfiable(premises_conj &
~conclusion):
29     is_valid = True
30 else:
31     is_valid = False
32 # ----- END -----
33
34 # — 按题面要求的固定字符串格式输出 — #
35 print("前提: P | Q, R | ~P")
36 print("结论: Q | R")
37 print(f"归结原理是否有效: {is_valid}")
38
39 # 调用函数
40 if __name__ == "__main__":
41     prove_resolution_principle()
42

```

测试结果

1/1 全部通过

测试集1

消耗内存137.41MB 代码执行时长: 0.76秒

预期输出

实际输出

展示原始输出

前提: P | Q, R | ~P

结论: Q | R

归结原理是否有效: True

前提: P | Q, R | ~P

结论: Q | R

归结原理是否有效: True

说点什么

👍

本关最大执行时间: 20秒 本次评测耗时:

上一关

下一关

自测运行

(4) 用等价演算进行证明:

程序设计思路:

首先, 使用 `SymPy` 定义命题变元 P, Q :

```
P, Q = symbols('P Q')
```

然后构造两个待验证的逻辑表达式:

```
expr1 = Not (Implies (P, Q))
expr2 = And (P, Not (Q))
```

为了证明两个表达式等价, 我们对 `expr1` 进行化简处理:

```
simplified_expr1 = simplify_logic (expr1)
```

得到化简后的表达式。

程序通过直接对比化简后的表达式与 $expr2$ 是否相等，来验证二者的等价性。

逻辑含义为：若两个逻辑表达式化简后的结果完全相同，则它们在所有真值指派下的取值都一致，因此是等价的。程序中：

化简后的 $expr1$ 与 $expr2$ 完全相同，因此：

$simplified_expr1 == expr2$ 的结果为 `True`，证明了逻辑等价式 $\neg(P \rightarrow Q) \Leftrightarrow (P \wedge \neg Q)$ 成立。

时间复杂度分析：由于代码固定只有2个命题变元 P 、 Q ，真值组合只有4种，算法最多检查4次就结束。表达式构建、逻辑化简、等值判断、输出打印都是固定次数操作，没有循环、没有递归、没有随数据规模增长的操作。所以时间复杂度是 $O(1)$ 。

空间复杂度分析：只创建2个符号变量+3个逻辑表达式，所有临时变量、字符串、输出内容占用空间固定不变。所以空间复杂度为 $O(1)$ 。

彭婉茹 49826

推理与证明

实验总用时: 00:05:10

更多操作

第4关: 用等价演算进... 100

学习内容

记录

评论

- 任务描述
- 相关知识
 - 等价演算
 - 使用 `simplify_logic` 函数
- 编程要求
- 测试说明

任务描述

本关任务是证明 $\sim(P \gg Q)$ 与 $P \& \sim Q$ 这两个命题公式是逻辑等价的。我们将不使用真值表, 而是使用 `sympy` 的符号计算能力来模拟等价演算的过程, 即通过一系列已知的等价规则 (如德·摩根定律) 进行逐步化简。

相关知识

为了完成本关任务, 你需要掌握: 1. 什么是等价演算, 2. 如何使用 `sympy` 的化简函数来模拟这一过程。

等价演算

等价演算 (Equivalence Calculus) 是通过应用一系列已知的逻辑等价式, 将一个公式逐步转换为另一个形式上不同但逻辑上等价的公式的过程。这是证明逻辑等价性

代码文件

```

10 # 计算所用的表达式
11 expr1 = Not(Implies(P, Q)) # ~(P
>> Q)
12 expr2 = And(P, Not(Q)) # P & ~Q
13
14 # — 按题面要求的固定字符串输出 — #
15 print("表达式1: ~(P >> Q)")
16 print("表达式2: P & ~Q")
17
18 # ----- START -----
19 simplified_expr1 = simplify_logic
(expr1)
20 # ----- END -----
21
22 print(f"化简后的表达式1:
{simplified_expr1}")
23 print(f"化简结果是否与表达式2相等:
{simplified_expr1 == expr2}")
24
25 # 调用函数

```

测试结果

1/1 全部通过

测试集1 消耗内存155.34MB 代码执行时长: 0.94秒

预期输出

实际输出

展示原始输出

表达式1: ~(P >> Q)

表达式2: P & ~Q

化简后的表达式1: P & ~Q

化简结果是否与表达式2相等: True

表达式1: ~(P >> Q)

表达式2: P & ~Q

化简后的表达式1: P & ~Q

化简结果是否与表达式2相等: True

说点什么

上关

自测运行

1.3.2 命题逻辑

(1) 三变量真值生成器

程序设计思想:

首先, 用 `SymPy` 定义命题变元 P, Q, R :

```
P, Q, R = symbols('P Q R')
```

然后把传入的字符串表达式安全地转成 `SymPy` 表达式对象:

```
expression = sympify(expression_str)
```

接着用 `itertools.product` 枚举 P, Q, R 的所有真值组合:

```
product([True, False], repeat=3)
```

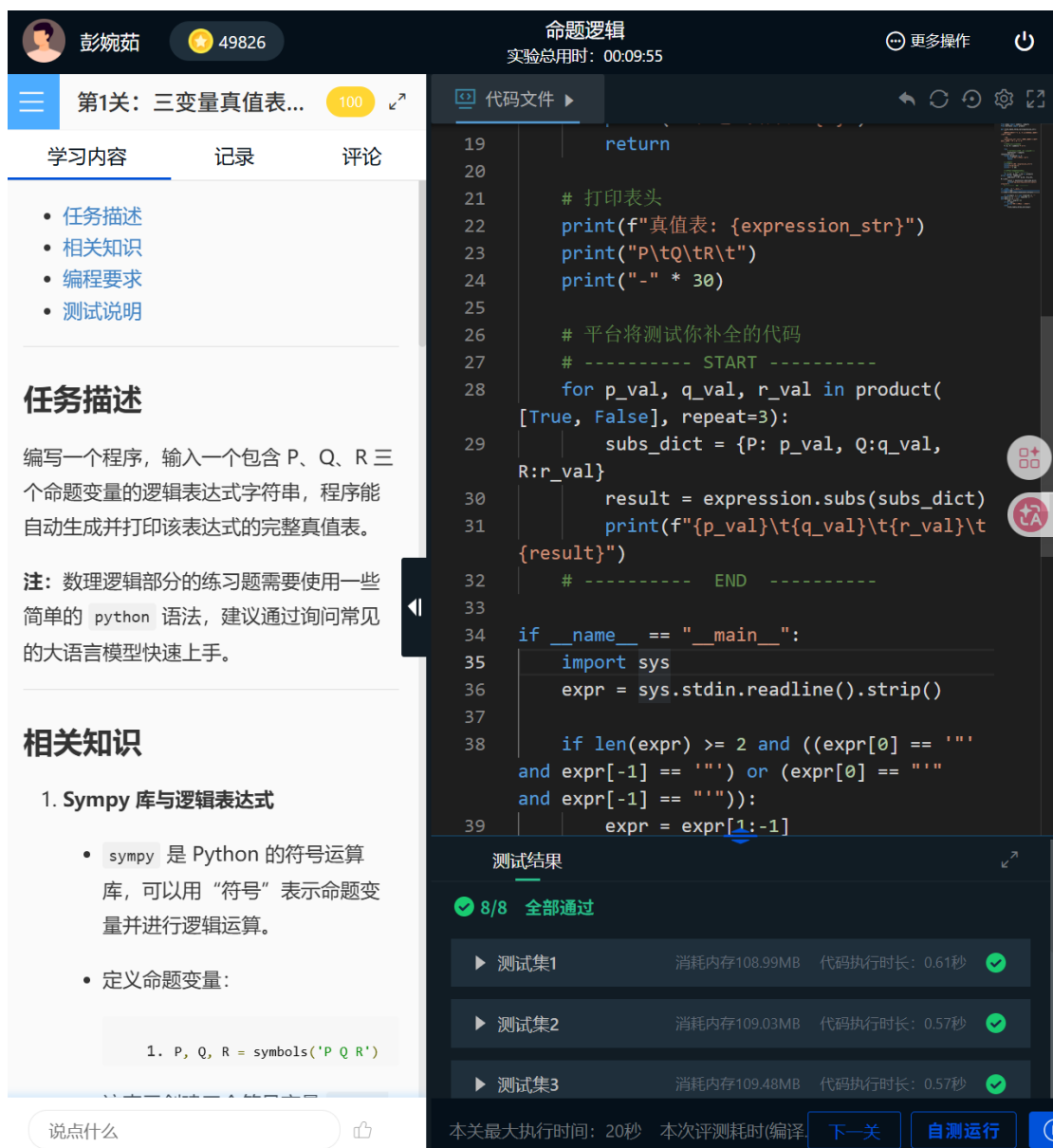
一共8种组合。

对每一组赋值，构造替换字典`subs_dict`，用`expression.subs(subs_dict)` 算出该组合下表达式的真值。

按固定格式逐行打印`P`、`Q`、`R`取值和对应结果，形成完整真值表。

时间复杂度分析：固定3个变量，共8种赋值组合，循环执行8次，每次计算与打印都是常数时间。所以时间复杂度为 $O(1)$ 。

空间复杂度分析：只创建3个符号变量、1个表达式对象、少量临时字典和字符串，空间固定不变。所以空间复杂度为 $O(1)$ 。



彭婉茹 49826 实验总用时: 00:09:55

第1关: 三变量真值表... 100

学习内容 记录 评论

- 任务描述
- 相关知识
- 编程要求
- 测试说明

任务描述

编写一个程序，输入一个包含 P、Q、R 三个命题变量的逻辑表达式字符串，程序能自动生成并打印该表达式的完整真值表。

注：数理逻辑部分的练习题需要使用一些简单的 python 语法，建议通过询问常见的大语言模型快速上手。

相关知识

1. Sympy 库与逻辑表达式

- sympy 是 Python 的符号运算库，可以用“符号”表示命题变量并进行逻辑运算。
- 定义命题变量：

```
1. P, Q, R = symbols('P Q R')
```

```
19     return
20
21     # 打印表头
22     print(f"真值表: {expression_str}")
23     print("P\tQ\tR\t")
24     print("-" * 30)
25
26     # 平台将测试你补全的代码
27     # ----- START -----
28     for p_val, q_val, r_val in product(
29         [True, False], repeat=3):
30         subs_dict = {P: p_val, Q: q_val,
31                     R: r_val}
32         result = expression.subs(subs_dict)
33         print(f"{p_val}\t{q_val}\t{r_val}\t{result}")
34     # ----- END -----
35
36 if __name__ == "__main__":
37     import sys
38     expr = sys.stdin.readline().strip()
39
40     if len(expr) >= 2 and ((expr[0] == "'"
41                             and expr[-1] == "'") or (expr[0] == "("
42                             and expr[-1] == ")):
43         expr = expr[1:-1]
```

测试结果

8/8 全部通过

测试集	消耗内存	代码执行时长	状态
测试集1	108.99MB	0.61秒	通过
测试集2	109.03MB	0.57秒	通过
测试集3	109.48MB	0.57秒	通过

本关最大执行时间: 20秒 本次评测耗时(编译) 下一关 自测运行

(2) 分配律的验证

程序设计思想：

首先，使用*SymPy*定义命题变元*P, Q, R*：

```
P, Q, R = symbols('P Q R')
```

然后构造分配律 1 的左右两边表达式：

```
formula1 = And (P, Or (Q, R))  
formula2 = Or (And (P, Q), And (P, R))
```

再构造分配律 2 的左右两边表达式：

```
formula3 = Or (P, And (Q, R))  
formula4 = And (Or (P, Q), Or (P, R))
```

随后分别将每一组左右两边的表达式用等价联结词*Equivalent*连接，构造出等价式：

```
equiv_expr = Equivalent (formula1, formula2)  
equiv_expr1 = Equivalent (formula3, formula4)
```

为了验证分配律是否为永真式，程序对每个等价式取反：

```
~equiv_expr
```

对应逻辑公式：

$$\neg((P \wedge (Q \vee R)) \leftrightarrow ((P \wedge Q) \vee (P \wedge R)))$$

程序通过调用*satisfiable*判断取反后的等价式是否可满足：

```
is_tauto1 = not satisfiable (~equiv_expr)  
is_tauto2 = not satisfiable (~equiv_expr1)
```

逻辑含义为：若取反后的等价式不可满足，则说明不存在赋值能使原等价式为假，因此原等价式是永真式；若取反后的等价式可满足，则说明原等价式不是永真式。

程序中，由于分配律是命题逻辑中的基本永真式，因此最终：

```
is_tauto1 = True  
is_tauto2 = True
```

时间复杂度分析：由于代码固定只有 3 个命题变元*P, Q, R*，真值组合只有 8 种，算法最多检查 8 次就结束。表达式构建、等价式构造、可满足性判断、输出打印都是固定次数操作，没有循环、没有递归、没有随数据规模增长的操作。所以时间复杂度是 $O(1)$ 。

空间复杂度分析：只创建 3 个符号变量 + 6 个逻辑表达式，所有临时变量、字符串、输出内容占用空间固定不变。所以空间复杂度为 $O(1)$

彭婉茹

49826

命题逻辑

实验总用时: 00:11:28

更多操作

🔌

第2关: 分配律的验证

100

学习内容

记录

评论

- 任务描述
- 相关知识
 - 逻辑等价 `Equivalent`
 - 判断重言式
- 编程要求
- 测试说明

任务描述

本关任务：逻辑等价中的分配律是化简表达式的重要工具。请使用 `sympy` 库编写一个程序，验证以下两个分配律是否为重言式（永真式），从而证明它们是成立的。

- $P \& (Q \mid R) \Leftrightarrow (P \& Q) \mid (P \& R)$
- $P \mid (Q \& R) \Leftrightarrow (P \mid Q) \& (P \mid R)$

相关知识

为了完成本关任务，你需要掌握：1. 如何在 `sympy` 中表示逻辑等价，2. 如何判断一个公式是否为重言式。

逻辑等价 `Equivalent`

在 `sympy` 中，我们可以使用 `Equivalent` 函数来表示两个命题公式之间的逻辑等价关系（ $\Leftrightarrow$ ）。它会创建一个表达式，当且

代码文件

```

10
11
12 def verify_distributive_laws():
13     P, Q, R = symbols('P Q R')
14
15     # ----- START -----
16     formula1 = And(P, Or(Q, R))
17     formula2 = Or(And(P, Q), And(P, R))
18     equiv_expr = Equivalent(formula1,
19                             formula2)
20     is_tauto1 = not satisfiable
21     (~equiv_expr)
22
23     formula3 = Or(P, And(Q, R))
24     formula4 = And(Or(P, Q), Or(P, R))
25     equiv_expr1 = Equivalent(formula3,
26                              formula4)
27     is_tauto2 = not satisfiable(~eq)
28     # ----- END -----
29
30     # 按题面要求的固定字符串输出

```

1/1 全部通过

测试集1 消耗内存101.77MB 代码执行时长: 0.71秒

预期输出

实际输出 展示原始输出

待验证定律1: $(P \& (Q \mid R)) \Leftrightarrow (P \& Q) \mid (P \& R)$

待验证定律2: $(P \mid (Q \& R)) \Leftrightarrow (P \mid Q) \& (P \mid R)$

定律1是否为重言式: True

定律2是否为重言式: True

待验证定律1: $(P \& (Q \mid R)) \Leftrightarrow (P \& Q) \mid (P \& R)$

待验证定律2: $(P \mid (Q \& R)) \Leftrightarrow (P \mid Q) \& (P \mid R)$

定律1是否为重言式: True

定律2是否为重言式: True

本关最大执行时间: 20秒 本次评测耗时(编译)

上一关

自测运行

1.3.3 逻辑与 AI 表示

(1) 全称规则

程序设计思想：

首先，对于规则 1 “所有的矩形都是四边形”，先定义对应的谓词， $R(x)$ 表示 x 是矩形， $Q(x)$ 表示 x 是四边形。根据全称量词的逻辑规则，所有满足 $R(x)$ 的个体都满足 $Q(x)$ ，因此符号化为 $\text{forall } x, R(x) \Rightarrow Q(x)$ 。

对于规则 2 “没有不遵守交通规则的司机”，先定义谓词 $D(x)$ 表示 x 是司机， $T(x)$ 表示 x 遵守交通规则。这句话等价于所有司机都遵守交通规则，因

此同样使用全称量词进行符号化，表示为 $\text{forall } x, D(x) \gg T(x)$ 。

程序依次输出两条规则的自然语言形式和对应的符号化结果，完成自然语言规则到逻辑表达式的转换。

时间复杂度分析：程序仅执行固定的字符串赋值和打印操作，没有循环、递归或任何与输入规模相关的计算，所有操作都在常数时间内完成，因此时间复杂度为 $O(1)$ 。

空间复杂度分析：程序只使用了固定数量的字符串变量存储规则和符号化结果，没有动态分配内存或使用复杂数据结构，内存占用保持不变，因此空间复杂度为 $O(1)$ 。

彭婉茹49826

逻辑与AI知识表示

实验总用时: 00:07:14

第1关: 全称规则

100

学习内容

记录

评论

任务描述

相关知识

AI知识表示与谓词逻辑

全称量词 (Universal Quantifier)

编程要求

测试说明

任务描述

假设你正在为一个人工智能构建知识库，它需要理解关于世界的一些普遍规则。本关任务是将以下两条自然语言规则，符号化为谓词逻辑表达式的字符串，以便存入机器人的知识库。

1. “所有的矩形都是四边形。”

2. “没有不遵守交通规则的司机。”

相关知识

为了完成本关任务，你需要掌握：1. 什么是AI知识表示，2. 如何将自然语言规则符号化。

AI知识表示与谓词逻辑

经典AI系统（也称为符号主义AI）的核心思想之一，就是将人类拥有的知识用一种

说什么

代码文件

```
1 def symbolize_universal_rules():
2     """
3     将用于AI知识库的普遍规则（自然语言）进行符号化。
4     """
5     # 规则1: “所有的矩形都是四边形。”
6     # 定义谓词:
7     # R(x): x是矩形
8     # Q(x): x是四边形
9
10    # ----- START -----
11    rule1_symbolic = "forall x, R(x) >> Q(x)"
12    # ----- END -----
13
14    print("规则1: “所有的矩形都是四边形。”")
15    print(f"符号化表示: {rule1_symbolic}")
```

测试结果

1/1 全部通过

测试集1

消耗内存181.28MB 代码执行时长: 0.05秒

预期输出

实际输出

展示原始输出

规则1: “所有的矩形都是四边形。”
符号化表示: forall x, R(x) >> Q(x)

规则2: “没有不遵守交通规则的司机。”
符号化表示: forall x, D(x) >> T(x)

规则1: “所有的矩形都是四边形。”
符号化表示: forall x, R(x) >> Q(x)

规则2: “没有不遵守交通规则的司机。”
符号化表示: forall x, D(x) >> T(x)

本关最大执行时间: 20秒

本次评测耗时(编译)

下一关

自测运行

15

(2) 存在事实

程序设计思想:

首先, 对于事实 1 “有些机器人会编程”, 先定义谓词 $R(x)$ 表示 x 是机器人, $P(x)$ 表示 x 会编程。根据存在量词的逻辑含义, 存在某个个体既是机器人又会编程, 因此符号化为 $\text{exists } x, R(x) \& P(x)$ 。

对于事实 2 “并非所有金属都能导电”, 先定义谓词 $M(x)$ 表示 x 是金属, $C(x)$ 表示 x 能导电。这句话等价于存在某种金属不能导电, 因此使用存在量词符号化为 $\text{exists } x, M(x) \& \sim C(x)$ 。

程序依次输出两条事实的自然语言描述和对应的符号化结果, 清晰完成自然语言事实到逻辑公式的转换。

时间复杂度分析: 程序只执行固定的字符串赋值和打印操作, 没有循环、递归, 也没有随数据规模变化的计算, 所有操作都在固定步骤内完成, 因此时间复杂度为 $O(1)$ 。

空间复杂度分析: 程序仅使用固定数量的字符串变量存储事实与符号化结果, 没有动态数据结构, 内存占用始终保持不变, 因此空间复杂度为 $O(1)$ 。

彭婉茹49826

逻辑与AI知识表示
实验总用时: 00:04:17

更多操作

第2关: 存在事实100

学习内容

记录

评论

任务描述

相关知识

编程要求

测试说明

存在量词 (Existential Quantifier)

量词的否定

任务描述

除了需要理解普遍规则外, 一个AI知识库也需要存储关于世界的具体事实。本关任务是将以下两条自然语言描述的事实, 符号化为谓词逻辑表达式的字符串, 以便AI能够理解和使用。

1. “有些机器人会编程。”

2. “并非所有金属都能导电。” (等价于“有些金属不能导电”)

相关知识

为了完成本关任务, 你需要掌握: 1. 存在量词的用法, 2. 如何处理量词的否定。

存在量词 (Existential Quantifier)

存在量词用于陈述一个论域 (个体域) 中至少存在一个个体具有某种性质。它的符号化表示为: $\exists x (A(x) \wedge B(x))$

说点什么

代码文件

```
7 # R(x): x是机器人
8 # P(x): x会编程
9
10 # 平台将测试你补全的代码
11 # ----- START -----
12 fact1_symbolic = "exists x, R(x) & P(x)"
13
14 # ----- END -----
15
16 print("事实1: “有些机器人会编程。”")
17 print(f"符号化表示: {fact1_symbolic}
18 \n")
19
20 # 事实2: “并非所有金属都能导电。”
21 # 定义谓词:
22 # M(x): x是金属
23 # C(x): x能导电
```

测试结果

1/1 全部通过

测试集1

消耗内存162.88MB 代码执行时长: 0.04秒

预期输出

实际输出

展示原始输出

事实1: “有些机器人会编程。”
符号化表示: exists x, R(x) & P(x)

事实1: “有些机器人会编程。”
符号化表示: exists x, R(x) & P(x)

事实2: “并非所有金属都能导电。”
符号化表示: exists x, M(x) & ~C(x)

事实2: “并非所有金属都能导电。”
符号化表示: exists x, M(x) & ~C(x)

本关最大执行时间: 20秒 本次评测耗时(编译)

上一关

自测运行

2 集合、关系及函数

2.1 实验内容

本实验旨在掌握集合、关系与函数的基本概念及其运算方法，理解集合运算、等价关系、偏序关系、关系矩阵、闭包运算以及函数性质等核心内容。通过程序实现相关算法，培养利用计算机解决离散数学问题的能力，提高分析问题、设计算法和实现数据结构运算的综合能力。

2.2 实验内容

2.2.1 集合的表示和基本运算

(1) 求集合的幂集：

本关任务：编写一个程序，求出一个集合的幂集。

(2) 容斥原理：

本关任务：给定一个数 n 和 m 个质数，要求利用容斥原理求出 $1 \sim n$ 中有多少数至少被这 m 个数中的至少一个整除。

2.2.2 关系表示及其运算

(1) 特殊关系——等价类与商集

本关任务：给定集合 $A = \{0, 1, 2, \dots, n\}$ 和整数 $m (m \leq n)$, R 是 A 上以 m 为模的同余关系。

求集合 A 关于 R 的商集，即 A/R 。

(2) 特殊关系——偏序关系与特殊元素

本关任务：给定集合 $A = \{1, 2, \dots, n\}$ 上的偏序关系 R ，再给定 A 的子集 B ，求出 B 的最大元、最小元、极大元、极小元。

(3) 二元关系——笛卡尔积

本关任务：编写一个程序，实现集合笛卡尔积运算，并按字典序输出。

(4) 二元关系——关系矩阵

本关任务：给定从集合 A 到集合 B 的二元关系 R ，输出其关系矩阵。

(5) 二元关系——关系的幂运算

本关任务：给定关系 R 的关系矩阵，求其 m 次幂 R^m 。

(6) 二元关系——关系的传递性

本关任务：给定一个关系矩阵，判断其是否具有传递性。

(7) 二元关系——关系的闭包运算

本关任务：给给定一个关系矩阵，输出其传递闭包。建议使用 *Warshall* 算法 解决此问题。

2.2.3 函数

(1) 函数的判断——函数的定义

本关任务：给定集合 A 到 B 的关系 f ，判断 f 是否是 A 到 B 上的函数（映射）

(2) 函数的运算——函数的类型

本关任务：给定集合 A 与 B ，再给定集合 A 到 B 的函数 f ，判断 f 是否是 A 到 B 上的单射、满射，或双射。

(3) 函数的运算——函数的运算与置换

本关任务：给定集合 A 与 A 上的置换函数 f ，求其复合 m 次后的置换

(4) 函数的运算——函数的应用

本关任务：设 $A_n = \{a_1, a_2, a_3, \dots, a_n\}$ 是一个包含 n 个元素的有限集合，定义集合 $B_n = \{b_1 b_2 b_3 \dots b_n \mid b_i \in \{0,1\}\}$ ，即所有长度为 n 的 01 字符串的集合。建立从幂集 $P(A_n)$ 到 B_n 的一个双射，满足：对任意子集 $S \in P(A_n)$ ，令 $f(S) = b_1 b_2 b_3 \dots b_n$ 。其中：

$$b_i = \begin{cases} 1, & a_i \in S \\ 0, & a_i \notin S \end{cases}, i = 1, 2, \dots, n$$

2.3 实验过程

2.3.1 集合的表示和基本运算

(1) 求集合的幂集：

程序设计思想：

首先，程序先读取集合的大小 n 和集合 A 的元素，存入数组 `setA` 中。

然后，按照题目要求，首先输出空集 `emptyset`。

随后程序通过组合生成算法，按子集大小从 1 到 n 依次生成所有非空子集：

对于每一个子集大小 k ，先初始化一个下标数组 p ，用来记录当前组合在 setA 中的位置，初始为 $0, 1, \dots, k-1$ ，这是当前长度下字典序最小的组合。

接着进入循环：

1. 根据 p 数组中记录的下标，从 setA 中取出对应元素，按题目格式输出当前子集，如 $\{a, b, c\}$ 。
2. 寻找下一个字典序的组合：从组合的末尾向前查找第一个可以增大的下标 $p[j]$ ，增大它的值，并把后面的下标依次更新为 $p[j] + 1, p[j] + 2, \dots$ ，以保持组合的字典序性质。
3. 如果找不到这样的 j ，说明当前长度下的所有组合已经生成完毕，退出循环。

当所有长度的子集都生成并输出后，程序结束，完成了按字典序输出集合所有子集的功能。

时间复杂度分析：程序需要生成一个 n 元集合的所有子集，总共有 2^n 个子集，生成每个子集都需要常数时间的操作，循环执行次数与子集数量成正比，因此时间复杂度为 $O(2^n)$ 。

空间复杂度分析：程序使用了固定大小的数组存储集合元素和组合下标，数组长度不超过 25，所有变量占用的空间都是常数级，因此空间复杂度为 $O(1)$ 。

彭婉茹
 49826

集合的表示和基本运算

实验总用时: 00:38:20

更多操作

第1关: 求集合的幂集 100

学习内容
 记录
 评论

• 任务描述
 • 相关知识
 • 集合的幂集
 • 编程要求
 • 测试说明

任务描述

本关任务: 编写一个程序, 求出一个集合的幂集

相关知识

为了完成本关任务, 你需要掌握: 集合的幂集。

集合的幂集

设 A 为任意集合, 把 A 的所有不同子集构成的集合叫做 A 的幂集 (power set), 记为 $P(A)$ 或 2^A 。

其符号化表示为 $P(A) = \{x | \forall x \subseteq A\}$

编程要求

第一行输入一个集合 A 的大小 n 。

说什么

代码文件

```

31 for (int i = 0; i < k; i++) {
32     p[i] = i;
33 }
34
35 while (1) {
36     // 2. 按照题目要求格式输出当前组
37     printf("{");
38     for (int i = 0; i < k; i++) {
39         printf("%d", setA[p[i]]);
40         if (i < k - 1) {
41             printf(",");
42         }
43     }
44     printf("}\n");
45
46     // 3. 寻找下一个字典序的组合
47     int j = k - 1;
48     while (j >= 0 && p[j] == n - k
    
```

测试结果

5/5 全部通过

▶ 测试集1	消耗内存20.52MB	代码执行时长: 0.41秒	
▶ 测试集2	消耗内存28.6MB	代码执行时长: 0.41秒	
▶ 测试集3	消耗内存30.27MB	代码执行时长: 0.08秒	
▶ 测试集4	消耗内存30.3MB	代码执行时长: 0.04秒	
▶ 测试集5	消耗内存33.11MB	代码执行时长: 0.13秒	

本关最大执行时间: 5秒 本次评测耗时(编译...

下一关
 自测运行

(2) 容斥原理:

程序设计思路:

首先, 程序通过 `scanf` 读入数值范围上限 n 和质数个数 m , 再读入 m 个质数存入数组 `primes` 中, 完成数据输入。

程序的核心是使用容斥原理, 计算 $1 \sim n$ 中能被这些质数中至少一个整除的数的个数。

为了实现容斥原理, 程序使用二进制枚举法遍历所有非空子集:

用 `for(int mask = 1; mask < (1<<m); mask++)` 遍历 m 个质数的所有非空子集, 每一个二进制位表示对应质数是否被选中。

对于每一个子集，程序执行以下步骤：

1. 初始化乘积 *pro* 为 1、元素个数 *cnt* 为 0；
2. 遍历每个质数，若该质数在当前子集中，则累乘到 *pro* 中，并将 *cnt* 加 1；
3. 如果乘积 *pro* 超过 *n*，说明该子集没有有效贡献，直接跳过；
4. 根据子集元素个数的奇偶性，执行加减操作：若 *cnt* 为奇数，将 n/pro 加到 *result* 中；若为偶数，则从 *result* 中减去 n/pro 。

最终 *result* 即为 $1 \sim n$ 中能被至少一个质数整除的数的总数，程序将其输出。

时间复杂度分析：程序需要枚举 *m* 个质数的所有非空子集，共有 $2^m - 1$ 个子集，每个子集的处理需要遍历 *m* 个质数，时间复杂度为 $O(m * 2^m)$ 。

空间复杂度分析：程序仅使用了固定大小的数组 *primes* 和若干变量，所有内存占用均为常数级，不随输入规模变化，因此空间复杂度为 $O(1)$ 。

2. 内层循环遍历集合元素 j , 0 到 n , 判断 $j \% m == i$, 匹配则输出:

```
for(int j=0; j<=n; j++)
{
    if(j % m == i)
    {
        printf("%d ", j);
    }
}
```

每个等价类输出完毕后换行, 实现分组打印。

时间复杂度分析: $O(nm)$ 外层循环执行 m 次, 内层循环每次执行 $n+1$ 次, 总操作次数与 n 和 m 的乘积成正比。

空间复杂度分析: $O(1)$ 仅使用有限个变量, 内存占用不随输入规模变化, 为常数级空间复杂度。

然后根据偏序集性质，若极大元唯一则验证其是否大于等于 B 中所有元素，满足条件即为最大元存入 *great_e* 数组；若极小元唯一则验证其是否小于等于 B 中所有元素，满足条件即为最小元存入 *least_e* 数组。

最后使用 *qsort* 函数对四个结果数组分别进行升序排序，并按顺序输出最大元、最小元、极大元、极小元。

时间复杂度： $O(k^2)$ ，其中 k 为子集 B 的大小，主要开销在于寻找极大元和极小元时需要对于子集元素进行双重循环遍历。

空间复杂度： $O(n^2)$ ，主要用于存储 $n \times n$ 的偏序关系矩阵，此外还需要 $O(k)$ 的额外空间存储子集及结果数组。

彭婉茹49826

第2关：偏序关系与特...100

学习内容记录评论

任务描述

给定集合 $A = \{1, 2, \dots, n\}$ 上的偏序关系 R ，再给定 A 的子集 B ，求出 B 的 **最大元、最小元、极大元、极小元**。

相关知识

为完成本关任务，你需要掌握：

1. 偏序关系
2. 特殊元素（最大元、最小元、极大元、极小元）

偏序关系

设 R 是非空集合 A 上的关系，若 R 同时满足

- 自反性
- 反对称性
- 传递性

则称 R 为 A 上的 **偏序关系 (Partial Order Relation)**，简称 **偏序**，记作 $\leq$ ，读作“小于等于”。

序偶 $\langle A, \leq \rangle$ 称为 **偏序集 (Partial Order)**

特殊关系

实验总用时：00:37:21

更多操作

代码文件

```
1 #include <stdio.h>
2 #include <stdlib.h> // for qsort
3
4 // 比较函数, 用于 qsort 升序排序
5 int comp(const void *a, const void *b) {
6     return (*(int*)a - *(int*)b);
7 }
8
9 int main() {
10     // n: 集合 A 的大小
11     int n;
12     scanf("%d", &n);
13
14     // rel_mat: 存储 n x n 的偏序关系矩阵
```

测试结果

12/12 全部通过

▶ 测试集1	消耗内存21.99MB	代码执行时长: 0.04秒	✓
▶ 测试集2	消耗内存21.99MB	代码执行时长: 0.02秒	✓
▶ 测试集3	消耗内存21.99MB	代码执行时长: 0.02秒	✓
▶ 测试集4	消耗内存21.99MB	代码执行时长: 0.03秒	✓
▶ 测试集5	消耗内存21.99MB	代码执行时长: 0.03秒	✓
▶ 测试集6	消耗内存21.99MB	代码执行时长: 0.02秒	✓
▶ 测试集7	消耗内存21.99MB	代码执行时长: 0.03秒	✓
▶ 测试集8	消耗内存21.99MB	代码执行时长: 0.02秒	✓

本关最大执行时间: 3秒 本次评测耗时(编译... 上一关 自测运行

26

(3) 特殊关系——笛卡尔积

程序设计思想：

首先读取集合 A 的大小 n 和集合 B 的大小 m ，然后依次读取集合 A 的 n 个元素存入数组 $setA$ ，读取集合 B 的 m 个元素存入数组 $setB$ 。

接着采用双重循环结构计算笛卡尔积，外层循环遍历集合 A 中的每个元素 $setA[i]$ ，内层循环遍历集合 B 中的每个元素 $setB[j]$ ，对于每一对组合按照“A 中元素 B 中元素”的格式输出序偶，每个序偶独占一行。

最后程序正常结束返回。

时间复杂度分析： $O(n \times m)$ ，其中 n 为集合 A 的大小， m 为集合 B 的大小，需要遍历所有可能的元素组合并输出，共需执行 $n \times m$ 次输出操作。

空间复杂度： $O(n + m)$ ，主要用于存储集合 A 和集合 B 的元素数组，不涉及额外的复杂数据结构。

彭婉茹49826

二元关系

实验总用时: 00:05:13

更多操作

第1关: 笛卡尔积

100

学习内容

记录

评论

任务描述

本关任务: 编写一个程序, 实现集合笛卡尔积运算, 并按字典序输出。

相关知识

为完成本关任务, 你需要掌握:

1. 序偶的定义
2. 笛卡尔积的定义

序偶的定义

由两个元素 x 、 y 按一定次序组成的二元组称为**有序偶对 (序偶)**, 记作 $\langle x, y \rangle$, 其中 x 称为序偶的**第一元素**, y 称为序偶的**第二元素**。

笛卡尔积

设 A 、 B 为两个集合, 称集合 $A \times B = \{\langle x, y \rangle \mid x \in A \wedge y \in B\}$ 为集合 A 与 B 的**笛卡尔积 (Descartes Product)**。

- 笛卡尔积 $A \times B$ 仍为集合;
- 其元素为序偶, 第一元素取自 A , 第

代码文件

```
1 #include <stdio.h>
2
3 int main() {
4     // 变量 n, m 分别表示集合 A, B 的大小
5     int n, m;
6     scanf("%d %d", &n, &m);
7
8     // 数组 setA 用于存储集合 A 的元素
9     int setA[100];
10    for (int i = 0; i < n; i++) {
11        scanf("%d", &setA[i]);
12    }
13
14    // 数组 setB 用于存储集合 B 的元素
15    int setB[100];
16    for (int i = 0; i < m; i++) {
17        scanf("%d", &setB[i]);
18    }
19
20    /****** Begin *****/
21    // 在此区域编写核心代码
22    // 1. 遍历集合 A 中的每一个元素。
23    // 2. 对于 A 中的每一个元素, 再遍历集合
24    B 中的所有元素。
25    // 3. 按照 "A 中元素 B 中元素" 的格式输出
26    序偶, 每个序偶占一行。
```

测试结果

12/12 全部通过

测试集1	消耗内存21.06MB	代码执行时长: 0.05秒	✓
测试集2	消耗内存21.06MB	代码执行时长: 0.04秒	✓
测试集3	消耗内存21.06MB	代码执行时长: 0.03秒	✓

本关最大执行时间: 3秒 本次评测耗时(编译... 下一关 自测运行

(4) 特殊关系——关系矩阵:

程序设计思想:

首先定义必要的变量 n 、 m 、 k 分别表示集合 A 的大小、集合 B 的大小以及关系 R 中包含的序偶数量, 并通过`scanf`函数读取这三个整数值。

随后声明一个二维数组`relationMatrix[100][100]`用于存储关系矩阵, 并使用`memset`函数将整个矩阵初始化为 0, 表示初始状态下任意两个元素之均不存在关系, 代码实现为`memset(relationMatrix, 0, sizeof(relationMatrix))`。

接着进入核心逻辑区域, 通过 `for` 循环执行 k 次读取操作, 每次读取一个序偶的两个元素 u 和 v 。由于题目中输入的元素编号从1开始, 而 C 语言数组下标从

0 开始，因此需要将读取的 u 和 v 分别减1作为矩阵的行下标和列下标，将 $relationMatrix[u - 1][v - 1]$ 的值置为1，表示元素 u 与元素 v 之间存在关系。通过这种方式，程序能够根据输入的序偶列表准确构建出完整的关系矩阵。

最后通过双重循环遍历整个 $n \times m$ 的关系矩阵，外层循环控制行索引 i 从0到 $n-1$ ，内层循环控制列索引 j 从0到 $m-1$ ，按行优先顺序逐个输出矩阵元素的值。

时间复杂度分析： $O(k + n \times m)$ ，其中 k 为关系序偶的数量，用于读取并标记 k 个序偶的位置，时间开销为 $O(k)$ ； $n \times m$ 为关系矩阵的规模，用于输出整个矩阵，时间开销为 $O(n \times m)$ 。

空间复杂度： $O(n \times m)$ ，主要用于存储 n 行 m 列的关系矩阵 $relationMatrix$ ，数组大小固定为 100×100 ，实际有效空间为 $n \times m$ 。除此之外仅使用了常数个整型变量存储输入参数和循环索引，额外空间开销为 $O(1)$ ，因此整体空间复杂度由关系矩阵决定，为 $O(n \times m)$ 。

彭婉茹

49826

二元关系

实验总用时: 00:05:02

更多操作

电源

第2关: 关系矩阵

100

学习内容

记录

评论

任务描述

给定从集合 A 到集合 B 的二元关系 R ，输出其关系矩阵。

相关知识

为完成本关任务，你需要掌握：**关系矩阵**。

关系矩阵

设集合 $A = \{a_1, a_2, \dots, a_n\}$ ， $B = \{b_1, b_2, \dots, b_m\}$ ， R 是从 A 到 B 的二元关系。

称 $n \times m$ 的 0-1 矩阵 $M_R = (r_{ij})$ 为关系 R 的**关系矩阵 (Relation Matrix)**，其中

$$r_{ij} = \begin{cases} 1, & \langle a_i, b_j \rangle \in R \\ 0, & \langle a_i, b_j \rangle \notin R \end{cases} \quad (1 \leq i \leq n, 1 \leq j \leq m).$$

- 行下标对应 A 中元素顺序，列下标对应 B 中元素顺序；
- 关系矩阵又称**布尔矩阵**。

代码文件

```

1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     // n: 集合 A 的大小, m: 集合 B 的大小,
6     // k: 关系 R 中的序偶数量
7     int n, m, k;
8     scanf("%d %d %d", &n, &m, &k);
9
10    // relationMatrix: 存储 n x m 的关系矩阵 M_R
11    int relationMatrix[100][100];
12
13    // 默认情况下关系不存在, 因此将矩阵所有元素初始化为 0
14    memset(relationMatrix, 0, sizeof

```

测试结果

12/12 全部通过

测试集	消耗内存	代码执行时长	结果
测试集1	20.65MB	0.05秒	通过
测试集2	20.65MB	0.04秒	通过
测试集3	20.65MB	0.08秒	通过
测试集4	20.65MB	0.03秒	通过
测试集5	20.65MB	0.04秒	通过
测试集6	20.65MB	0.05秒	通过
测试集7	20.65MB	0.05秒	通过

说点什么

👍

本关最大执行时间: 3秒 本次评测耗时:

上一关

下一关

自测运行

(5) 特殊关系——关系的幂运算:

程序设计思想:

首先读取集合的大小 n 和幂次 m ，然后读取 $n \times n$ 的关系矩阵 R ，其中 $R[i][j] = 1$ 表示元素 $i + 1$ 与元素 $j + 1$ 之间存在关系。接着声明 $result$ 数组用于存储最终结果矩阵 R^m ，声明 $temp$ 数组用于在矩阵乘法过程中存储中间计算结果，避免在计算过程中因原地更新导致数据污染。

接着处理幂次 $m = 0$ 的特殊情况，根据数学定义任何关系矩阵的 0 次幂为单位矩阵，因此通过双重循环将 $result$ 矩阵对角线元素置为 1、其余元素置为 0，代码实现为 $result[i][j] = (i == j)? 1: 0$ 。若 $m > 0$ 则先将 $result$ 矩阵初始化为原始关系矩阵 R ，作为幂运算的初始状态，即 $R^1 = R$ 。

然后进行布尔矩阵乘法迭代计算，外层循环控制迭代次数 r 从 1 到 $m - 1$ ，共执行 $m - 1$ 次矩阵乘法。每次迭代中通过三重循环实现布尔矩阵乘法：最外层控制结果矩阵的行 i ，中间层控制列 j ，最内层遍历中间变量 k ，通过逻辑与判断 $result[i][k] \&\& R[k][j]$ 是否同时为 1，若满足则说明存在传递路径，将 $temp[i][j]$ 置为 1 并 *break* 跳出内层循环以提高效率。核心代码为 $if(result[i][k] \&\& R[k][j]) \{ temp[i][j] = 1; break; \}$ 。每完成一次矩阵乘法后，将 $temp$ 中的中间结果拷贝回 $result$ 数组，为下一轮迭代做准备，代码实现为双重循环执行 $result[i][j] = temp[i][j]$

最后通过双重循环按行优先顺序输出最终的 $n \times n$ 结果矩阵，每行元素输出完毕后换行，从而呈现关系矩阵 R 的 m 次幂运算结果。

时间复杂度分析： $O(m \times n^3)$ ，其中 n 为关系矩阵的阶数， m 为幂次。每次布尔矩阵乘法需要三重循环遍历，时间开销为 $O(n^3)$ ，共需执行 $m - 1$ 次乘法运算，因此总时间复杂度为 $O(m \times n^3)$ 。当 $m = 0$ 或 $m = 1$ 时可视为常数时间操作。

空间复杂度分析： $O(n^2)$ ，主要用于存储原始关系矩阵 R 、结果矩阵 $result$ 和临时矩阵 $temp$ ，三个二维数组的大小均为 $n \times n$ 。除此之外仅使用了常数个整型变量存储输入参数和循环索引，额外空间开销为 $O(1)$ ，因此整体空间复杂度由矩阵存储决定，为 $O(n^2)$ 。

彭婉茹

★ 49826

三元关系
实验总用时: 00:41:34

更多操作

🔍

三第3关：关系的幂运算100↗️

学习内容记录评论

任务描述

给定关系 R 的关系矩阵，求其 m 次幂 R^m 。

相关知识

为完成本关任务，你需要掌握：

1. 关系的复合运算
2. 关系的幂运算

关系的复合运算

设 A, B, C 为三个集合， R 是从 A 到 B 的关系 ($R: A \rightarrow B$)， S 是从 B 到 C 的关系 ($S: B \rightarrow C$)，则 R 与 S 的**复合关系**(合成关系) $R \circ S$ 是从 A 到 C 的关系，定义为：

$$R \circ S = \{ \langle a, c \rangle \mid \exists b \in B (\langle a, b \rangle \in R \wedge \langle b, c \rangle \in S) \}.$$

关系的幂运算

设 R 是集合 A 上的关系，其 n 次幂 R^n 递归定义如下：

1. $R^0 = I_A = \{ \langle a, a \rangle \mid a \in A \}$ (恒等关系)；

代码文件

```
1 #include <stdio.h>
2 #include <string.h>
3
4 int main() {
5     // n: 集合 A 的大小, m: 幂次
6     int n, m;
7     scanf("%d %d", &n, &m);
8
9     // R: 存储 n x n 的关系矩阵
10    int R[50][50];
11    for (int i = 0; i < n; i++) {
12        for (int j = 0; j < n; j++) {
13            scanf("%d", &R[i][j]);
14        }
15    }
16
17    // result: 存储最终的 n x n 关系矩阵 R^m
18    int result[50][50];
19    // temp: 用于在矩阵乘法中存储中间结果
20    int temp[50][50];
21
22    /***** Begin *****/
23    // 在此区域编写核心代码
24    // 1. 考虑 m = 0 的情况
25    // 2. 进行矩阵乘法
26 }
```

测试结果

🟢 12/12 全部通过

▶ 测试集1 🔒

消耗内存21.13MB 代码执行时长: 0.07秒 🟢

▶ 测试集2 🔒

消耗内存21.13MB 代码执行时长: 0.05秒 🟢

▶ 测试集3 🔒

消耗内存21.13MB 代码执行时长: 0.05秒 🟢

本关最大执行时间: 3秒 本次评测耗时(编)

上一关下一关自测题

(6) 特殊关系——关系的传递性:

程序设计思想:

程序设计思想：首先读取集合 A 的大小 n ，然后通过双重循环读取 $n \times n$ 的关系矩阵 *matrix*，其中 *matrix*[i][j] = 1 表示元素 $i + 1$ 与元素 $j + 1$ 之间存在关系。接着定义标志变量 *isTransitive* 并初始化为 1，用于标记该关系是否具有传递性，默认假设关系满足传递性。

接着通过三重嵌套循环遍历所有可能的三元组组合 (i, j, k) ，其中 i 、 j 、 k 的取值范围均为 0 到 $n-1$ ，分别对应关系中的三个元素。在循环体内检查是否存在违反传递性的情况，即当 $matrix[i][j] == 1$ 且 $matrix[j][k] == 1$ 时，若

$matrix[i][k] == 0$ 则说明存在 aRb 且 bRc 但 aRc 不成立的反例，此时关系不满足传递性。核心判断代码为

```
if(matrix[i][j] == 1 && matrix[j][k] == 1 && matrix[i][k] == 0) { isTransitive = 0; break; }。
```

一旦发现反例即将 $isTransitive$ 置为 0 并跳出当前循环，虽然代码中仅跳出最内层循环，但后续判断不会再改变标志位状态，不影响最终结果的正确性。

最后根据 $isTransitive$ 标志位的值输出判定结果，若 $isTransitive$ 仍为 1 则说明遍历完所有三元组均未发现反例，关系具有传递性，输出"*Transitivity*"; 否则输出"*None*"表示该关系不满足传递性。

时间复杂度分析： $O(n^3)$ ，其中 n 为关系矩阵的阶数。算法需要三重嵌套循环遍历所有可能的 (i, j, k) 三元组组合，最坏情况下需要执行 $n \times n \times n$ 次判断操作。虽然发现反例后可以提前终止，但最坏时间复杂度仍为立方级别。

空间复杂度分析： $O(n^2)$ ，主要用于存储 $n \times n$ 的关系矩阵 $matrix$ ，数组大小固定为 100×100 ，实际有效空间为 $n \times n$ 。除此之外仅使用了常数个整型变量存储输入参数、循环索引和标志位，额外空间开销为 $O(1)$ ，因此整体空间复杂度由关系矩阵决定，为 $O(n^2)$ 。

彭婉茹

49826

二元关系

实验总用时: 00:06:03

更多操作

🔌

第4关: 关系的传递性

100

学习内容

记录

评论

任务描述

本关任务: 给定一个关系矩阵, 判断其是否具有传递性。

相关知识

你需要掌握: 关系的传递性

关系的传递性

设 R 是集合 A 上的关系。若对任意 $x, y, z \in A$, 只要 $\langle x, y \rangle \in R$ 且 $\langle y, z \rangle \in R$, 就有 $\langle x, z \rangle \in R$, 则称 R 是传递的 (Transitive), 或称 R 具有传递性 (Transitivity)。

符号化表述:

R 在 A 上是传递的 $\iff$

$$(\forall x)(\forall y)(\forall z)((x, y, z \in A \wedge \langle x, y \rangle \in R \wedge \langle y, z \rangle \in R) \rightarrow \langle x, z \rangle \in R) = 1.$$

R 在 A 上不是传递的 $\iff$

$$(\exists x)(\exists y)(\exists z)((x, y, z \in A \wedge \langle x, y \rangle \in R \wedge \langle y, z \rangle \in R \wedge \langle x, z \rangle \notin R) = 1.$$

编程要求

说点什么

代码文件

```

1 #include <stdio.h>
2
3 int main() {
4     // n: 集合 A 的大小
5     int n;
6     scanf("%d", &n);
7
8     // matrix: 存储 n x n 的关系矩阵
9     int matrix[100][100];
10    for (int i = 0; i < n; i++) {
11        for (int j = 0; j < n; j++) {
12            scanf("%d", &matrix[i][j]);
13        }
14    }
15 }

```

测试结果

12/12 全部通过

▶ 测试集1	🔒	消耗内存20.89MB	代码执行时长: 0.05秒	✅
▶ 测试集2	🔒	消耗内存20.89MB	代码执行时长: 0.04秒	✅
▶ 测试集3	🔒	消耗内存20.89MB	代码执行时长: 0.03秒	✅
▶ 测试集4	🔒	消耗内存20.89MB	代码执行时长: 0.04秒	✅
▶ 测试集5	🔒	消耗内存20.89MB	代码执行时长: 0.03秒	✅
▶ 测试集6	🔒	消耗内存20.89MB	代码执行时长: 0.04秒	✅
▶ 测试集7	🔒	消耗内存20.89MB	代码执行时长: 0.04秒	✅
▶ 测试集8	🔒	消耗内存20.89MB	代码执行时长: 0.04秒	✅
▶ 测试集9	🔒	消耗内存20.89MB	代码执行时长: 0.03秒	✅

本关最大执行时间: 3秒

本次评测耗时(毫秒)

上一关

下一关

自测运行

(7) 特殊关系——关系的闭包运算:

程序设计思想:

首先读取集合 A 的大小 n , 然后通过双重循环读取 $n \times n$ 的关系矩阵 $matrix$, 其中 $matrix[i][j] = 1$ 表示元素 $i + 1$ 与元素 $j + 1$ 之间存在直接关系, $matrix[i][j] = 0$ 表示不存在直接关系。该矩阵将作为 *Warshall* 算法的输入, 并在原矩阵上直接进行更新计算传递闭包。

接着应用 *Warshall* 算法的核心思想进行传递闭包计算, 通过三重嵌套循环实现。最外层循环遍历中间节点 k 从 0 到 $n - 1$, 表示逐步考虑以每个元素作为传递路径的中间桥梁; 内层双重循环遍历所有可能的起点 i 和终点 j , 检查是否存在通过节点 k 的间接路径。核心更新逻辑为 $matrix[i][j] =$

$matrix[i][j] \parallel (matrix[i][k] \&\& matrix[k][j])$ ，该表达式的含义是：若原本 i 到 j 已存在路径，或者存在 i 到 k 的路径且同时存在 k 到 j 的路径，则 i 到 j 在传递闭包中应标记为可达。代码实现为：

```
for (int k = 0; k < n; k++) {
    for (int i = 0; i < n; i++) {
        for (int j = 0; j < n; j++) {
            matrix[i][j] = matrix[i][j] || (matrix[i][k] && matrix[k][j]);
        }
    }
}。
```

通过这种动态规划的方式，算法能够逐步将所有间接可达的关系补充到矩阵中，最终得到完整的传递闭包。

最后通过双重循环按行优先顺序输出计算完成的 $n \times n$ 传递闭包矩阵，每行元素之间用空格分隔，每行输出完毕后换行，从而以标准的矩阵格式呈现最终结果，其中 $matrix[i][j] = 1$ 表示元素 $i + 1$ 到元素 $j + 1$ 在传递闭包中存在可达关系。

时间复杂度分析： $O(n^3)$ ，其中 n 为关系矩阵的阶数。*Warshall*算法需要三重嵌套循环，外层循环执行 n 次，内层双重循环各执行 n 次，总共需要执行 $n \times n \times n$ 次逻辑判断与赋值操作。虽然每次操作仅为常数时间的逻辑或运算，但整体时间复杂度仍为立方级别，适用于中小规模的关系矩阵计算。

空间复杂度分析： $O(n^2)$ ，主要用于存储 $n \times n$ 的关系矩阵 $matrix$ ，数组大小固定为 100×100 ，实际有效空间为 $n \times n$ 。算法采用原地更新策略，直接在输入矩阵上计算传递闭包，无需额外的二维数组存储中间结果，仅使用了常数个整型变量存储循环索引，额外空间开销为 $O(1)$ ，因此整体空间复杂度由关系矩阵决定，为 $O(n^2)$ 。

彭婉茹

49826

二元关系

实验总用时: 00:07:59

更多操作

电源

第5关: 关系的闭包运算

100

学习内容

记录

评论

任务描述

本关任务: 给定一个关系矩阵, 输出其传递闭包。
建议使用 **Warshall 算法** 解决此问题。

相关知识

你需要掌握:

1. 关系的闭包运算
2. Warshall 算法

关系的闭包运算

设 R 是集合 A 上的关系, 若存在 A 上的关系 R' , 满足

1. R' 是传递的;
2. 对任何传递关系 R'' , 只要 $R \subseteq R''$, 就有 $R' \subseteq R''$;

则称 R' 为 R 的传递闭包 (Transitive Closure), 记作 $t(R)$ 。

传递闭包是在 R 中添加最少元素使其具备传递性的扩充。

Warshall 算法

代码文件

```

1 #include <stdio.h>
2
3 int main() {
4     // n: 集合 A 的大小
5     int n;
6     scanf("%d", &n);
7
8     // matrix: 存储 n x n 的关系矩阵, 将在此
      矩阵上直接计算传递闭包
9     int matrix[100][100];
10    for (int i = 0; i < n; i++) {
11        for (int j = 0; j < n; j++) {

```

测试结果

12/12 全部通过

▶ 测试集1	消耗内存21.1MB	代码执行时长: 0.05秒	✓
▶ 测试集2	消耗内存21.1MB	代码执行时长: 0.03秒	✓
▶ 测试集3	消耗内存21.1MB	代码执行时长: 0.04秒	✓
▶ 测试集4	消耗内存21.1MB	代码执行时长: 0.03秒	✓
▶ 测试集5	消耗内存21.1MB	代码执行时长: 0.03秒	✓
▶ 测试集6	消耗内存21.1MB	代码执行时长: 0.03秒	✓
▶ 测试集7	消耗内存21.1MB	代码执行时长: 0.03秒	✓
▶ 测试集8	消耗内存21.1MB	代码执行时长: 0.03秒	✓
▶ 测试集9	消耗内存21.1MB	代码执行时长: 0.03秒	✓

说点什么

1

本关最大执行时间: 3秒 本次评测耗时(编译...

上一关

自测运行

2.3.3 函数

(1) 函数的判断——函数的定义:

程序设计思想:

首先读取关系 f 中包含的序偶数量 n , 然后声明两个整型数组 u 和 v 分别存储 n 个序偶的第一个元素和第二个元素, 通过 for 循环依次读取每对序偶的值并存入对应数组中。接着定义标志变量 $is_function$ 并初始化为 1, 用于标记该关系是否满足函数的定义, 默认假设关系构成函数。

接着通过双重循环遍历所有序偶对来验证函数性质, 外层循环变量 i 从 0 遍历到 $n - 1$, 内层循环变量 j 从 $i + 1$ 遍历到 $n - 1$, 这样可以避免重复比较同一对序

偶。在循环体内检查是否存在违反函数定义的情况，即当两个序偶的第一个元素相同但第二个元素不同时，说明同一个输入值对应了多个不同的输出值，这违背了函数的单值性要求。一旦发现反例即将`is_function`置为0并跳出内层循环，随后通过`if(!is_function) break;`语句跳出外层循环，实现提前终止以提高效率。

最后根据`is_function`标志位的值输出判定结果，若`is_function`仍为1则说明遍历完所有序偶对均未发现违反单值性的情况，该关系满足函数定义，输出"YES"；否则输出"NO"表示该关系不构成函数。

时间复杂度分析： $O(n^2)$ ，其中 n 为序偶的数量。算法需要双重嵌套循环遍历所有可能的序偶对组合，最坏情况下需要执行 $n \times (n - 1)/2$ 次比较操作。虽然发现反例后可以提前终止循环，但最坏时间复杂度仍为平方级别。

空间复杂度分析： $O(n)$ ，主要用于存储 n 个序偶的两个一维数组 u 和 v 。除此之外仅使用了常数个整型变量存储输入参数、循环索引和标志位，额外空间开销为 $O(1)$ ，因此整体空间复杂度由序偶存储数组决定，为 $O(n)$ 。

彭婉茹
49826

第1关：函数的定义

100

学习内容

记录

评论

任务描述

给定集合 A 到 B 的关系 f ，判断 f 是否是 A 到 B 上的函数（映射）。

相关知识

为完成本关任务，你需要掌握：函数的定义。

函数的定义

设 f 是集合 A 到 B 的关系，如果对每个 $x \in A$ ，都存在唯一的 $y \in B$ ，使得 $\langle x, y \rangle \in f$ ，则称关系 f 为 A 到 B 的函数（Function）（或映射（Mapping）、变换（Transform）），记为 $f: A \rightarrow B$ 。

- A 为函数 f 的定义域，记为 $\text{dom } f = A$ ；
- $f(A)$ 为函数 f 的值域，记为 $\text{ran } f$ 。
- 当 $\langle x, y \rangle \in f$ 时，通常记为 $y = f(x)$ ，这时称 x 为函数 f 的自变量， y 为 x 在 f 下的函数值（或称 f 在 x 处的函数值）。

代码文件

```

1  #include <stdio.h>
2
3  int main() {
4      // n: 关系 f 中的序偶数量
5      int n;
6      scanf("%d", &n);
7
8      // u, v: 分别存储 n 个序偶的第一个和第二个元素
9      int u[100];
10     int v[100];
11     for (int i = 0; i < n; i++) {
12         scanf("%d %d", &u[i], &v[i]);
13     }
14
15     // is_function: 标记关系是否为函数，1 表示是，0 表示否。默认是。
16     int is_function = 1;
17
18     /****** Begin *****/
19     // 在此区域编写核心代码

```

测试结果

20/20 全部通过

▶ 测试集1	消耗内存39.96MB	代码执行时长: 0.09秒	✓
▶ 测试集2	消耗内存39.96MB	代码执行时长: 0.04秒	✓
▶ 测试集3	消耗内存39.96MB	代码执行时长: 0.04秒	✓
▶ 测试集4	消耗内存39.96MB	代码执行时长: 0.05秒	✓
▶ 测试集5	消耗内存39.96MB	代码执行时长: 0.05秒	✓

说点什么

👍

本关最大执行时间: 3秒 本次评测耗时(编译...

下一关

自测运行

（2）函数的判断——函数的类型：

程序设计思想：

首先读取集合 A 的大小 $sizeA$ 、集合 B 的大小 $sizeB$ 以及函数关系中包含的序偶数量 n ，然后声明两个整型数组 u 和 v 分别存储 n 个序偶的定义域元素和值域元素，通过 for 循环依次读取每对序偶的值并存入对应数组中。接着定义两个布尔型标志变量 $is_injective$ 和 $is_surjective$ 并初始化为 $true$ ，分别用于标记该函数是否为单射和满射。

接着首先验证该关系是否构成从 A 到 B 的合法函数，通过数组 a_up 统计 A 中每个元素作为定义域出现的次数，若存在某个元素出现次数不等于1，说明该关

系不满足函数的定义域完整性或单值性要求，此时直接将两个标志位均置为 *false* 并跳转至输出阶段。若关系合法则继续检查单射性质，通过双重循环遍历所有序偶对，外层循环变量 *i* 从 0 遍历到 $n - 1$ ，内层循环变量 *j* 从 $i + 1$ 遍历到 $n - 1$ ，检查是否存在不同的定义域元素对应相同的值域元素，即当 $v[i] == v[j]$ 时说明多个输入映射到同一输出，违背单射定义，发现反例后立即跳出循环。

然后检查满射性质，声明布尔数组 *mapb* 并初始化为 *false*，遍历所有序偶将 *mapb*[*v*[*i*]] 置为 *true* 以标记值域中哪些元素被映射到，随后遍历集合 *B* 的所有元素 1 到 *sizeB*，若存在某个 *mapb*[*i*] 仍为 *false* 说明该元素未被任何定义域元素映射，违背满射定义。最后根据两个标志位的组合情况输出判定结果，若同时为 *true* 输出 3 表示双射，仅单射为 *true* 输出 1，仅满射为 *true* 输出 2，两者均为 *false* 输出 -1 表示既非单射也非满射

时间复杂度分析： $O(n^2 + \text{sizeB})$ ，其中 *n* 为序偶数量，*sizeB* 为集合 *B* 的大小。检查单射性质需要双重循环遍历序偶对，最坏时间复杂度为 $O(n^2)$ ；检查满射性质需要遍历 *n* 个序偶标记映射关系再遍历 *sizeB* 个元素验证覆盖情况，时间复杂度为 $O(n + \text{sizeB})$ 。整体时间复杂度由单射检查主导，为 $O(n^2)$ 。

空间复杂度分析： $O(n + \text{sizeA} + \text{sizeB})$ ，主要用于存储 *n* 个序偶的两个一维数组 *u* 和 *v*，以及统计定义域出现次数的数组 *a_up* 和标记值域覆盖情况的数组 *mapb*。数组大小均与输入规模线性相关，除此之外仅使用了常数个整型和布尔型变量，额外空间开销为 $O(1)$ ，因此整体空间复杂度为线性级别。

彭婉茹
 49826

函数的判断

实验总用时: 00:21:59

更多操作

第2关: 函数的类型

100

学习内容

记录

评论

任务描述

给定集合 A 与 B ，再给定集合 A 到 B 的函数 f ，判断 f 是否是 A 到 B 上的单射、满射，或双射。

相关知识

为完成本关任务，你需要掌握：函数的类型。

函数的类型

设 f 是从 A 到 B 的函数：

- 单射 (Injective)**：对任意 $x_1, x_2 \in A$ ，如果 $x_1 \neq x_2$ ，有 $f(x_1) \neq f(x_2)$ ，则称 f 为从 A 到 B 的**单射** (不同的 x 对应不同的 y)。
- 满射 (Surjective)**：如果 $\text{ran } f = B$ ，则称 f 为从 A 到 B 的**满射**。
- 双射 (Bijective)**：若 f 既是单射又是满射，则称 f 为从 A 到 B 的**双射**。
- 若 $A = B$ ，则称 f 为 A 上的函数；当 A 上的函数 f 是双射时，称 f 为一个**变换**。

代码文件

```

1 #include <stdio.h>
2 #include <stdbool.h> // for bool type
3 #include <string.h> // for memset
4
5 int main() {
6     // sizeA: 集合 A 的大小, sizeB: 集合 B
6     // 的大小
7     int sizeA, sizeB;
8     scanf("%d %d", &sizeA, &sizeB);
9
10    // n: 函数 f 中的序偶数量
11    int n;

```

测试结果

20/20 全部通过

测试集	消耗内存	代码执行时长	结果
测试集1	20.86MB	0.05秒	通过
测试集2	20.86MB	0.04秒	通过
测试集3	20.86MB	0.03秒	通过
测试集4	20.86MB	0.03秒	通过
测试集5	20.86MB	0.04秒	通过
测试集6	20.86MB	0.04秒	通过
测试集7	20.86MB	0.05秒	通过
测试集8	20.86MB	0.04秒	通过
测试集9	20.86MB	0.03秒	通过

说点什么

本关最大执行时间: 3秒 本次评测耗时(编译...

上一关

自测运行

(3) 函数的运算——函数的复合运算与置换：

程序设计思想：

首先读取集合大小 n 和置换函数 f 的映射关系，将 f 存储为数组 f_map ，其中 $f_map[i - 1] = j$ 表示 $f(i) = j$ 。然后读取复合次数 m ，初始化结果数组 res_map 为恒等置换，即 $res_map[i] = i + 1$ 。

接着通过 m 次循环计算函数复合，每次迭代中遍历所有元素 i ，计算 $temp_map[i] = f_map[res_map[i] - 1]$ ，表示先将元素 $i + 1$ 经 res_map 映射，再经 f 映射得到新结果。核心代码为 $temp_map[i] = f_map[res_map[i] - 1]$ 。单次复合完成后将 $temp_map$ 复制回 res_map ，为下一轮迭代做准备。

最后按序输出每个元素 $i + 1$ 经 m 次复合后的映射结果 $res_map[i]$ ，格式为 <

$i + 1, res_map[i] >$ 。

时间复杂度分析： $O(m \times n)$ ，其中 n 为集合大小， m 为复合次数。每次复合需遍历 n 个元素计算映射，共执行 m 次复合操作。

空间复杂度分析： $O(n)$ ，主要用于存储置换函数数组 f_map 、结果数组 res_map 和临时数组 $temp_map$ ，三个数组大小均为 n 类。

任务描述

给定集合 A 与 A 上的置换函数 f ，求其复合 m 次后的置换。

相关知识

为完成本关任务，你需要掌握：

1. 函数的复合运算
2. 置换函数（排列）

函数的复合运算

设 $f: A \rightarrow B, g: B \rightarrow C$ 是两个函数，则 g 与 f 的**复合函数** $g \circ f: A \rightarrow C$ 定义为：

$$g \circ f = \{ \langle x, z \rangle \mid x \in A, z \in C, \exists y \in B (y = f(x) \wedge z = g(y)) \}.$$

复合运算满足结合律，可推广到多次复合。

置换函数

设 $A = \{a_1, a_2, \dots, a_n\}$ 为有限集合，则从 A 到 A 的**双射函数**称为 A 上的**置换**

代码文件

```
3
4 int main() {
5     // n: 集合 A = {1, 2, ..., n} 的大小
6     int n;
7     scanf("%d", &n);
8
9     // f_map: 存储置换函数 f. f_map[i-1] =
10    j 表示 f(i) = j
11    int f_map[50];
12    for (int i = 0; i < n; i++) {
13        int u, v;
14        scanf("%d %d", &u, &v);
15        f_map[u - 1] = v;
16    }
17
18    // m: 复合的次数
19    int m;
20    scanf("%d", &m);
21
22    // res_map: 存储最终复合 m 次后的置换结果
23    int res_map[50];
24
25    // temp_map: 用于在循环中存储临时的计算结果
26    int temp_map[50];
```

测试结果

20/20 全部通过

测试集	消耗内存	代码执行时长	结果
测试集1	27.76MB	0.09秒	通过
测试集2	27.76MB	0.04秒	通过
测试集3	27.76MB	0.03秒	通过

本关最大执行时间：3秒 本次评测耗时(编译... 下一关 自测运行

(4) 函数的运算——函数的应用：

程序设计思想：

首先读取集合 A 的大小 n 和 n 个元素存入数组 set_A ，然后计算幂集大小 $total = 2^n$ ，通过位运算 $1 \ll n$ 高效实现。

接着外层循环变量 i 从0遍历到 $2^n - 1$ ，每个 i 的 n 位二进制表示对应幂集中

的一个子集及其 01 特征字符串。内层循环先输出子集部分，通过位运算 $(i \gg (n - 1 - j)) \& 1$ 检查 i 的第 j 位是否为 1，若为 1 则将对元素 $set_A[j]$ 加入子集输出，使用 $first$ 变量控制逗号格式，空集时直接输出 $\{\}$ 。子集输出完成后添加 $\}->$ 分隔符，然后再次遍历 n 位输出对应的 01 字符串， b_1 对应最高位、 b_n 对应最低位，确保子集元素与特征位一一对应。

最后每个子集及其 01 字符串输出完毕后换行，循环结束后即完成幂集 $P(A_n)$ 到 B_n 所有元素的枚举输出。

时间复杂度分析： $O(n \times 2^n)$ ，其中 n 为集合大小。需要枚举 2^n 个子集，每个子集的构造和输出需要遍历 n 个元素检查位状态，总操作次数为 $n \times 2^n$ 。

空间复杂度分析： $O(n)$ ，主要用于存储集合 A_n 的 n 个元素数组 set_A ，除此之外仅使用常数个整型变量存储循环索引和中间状态，额外空间开销为 $O(1)$ 。

彭婉茹49826

函数的运算
实验总用时: 00:40:51

更多操作

第2关: 函数的应用100

学习内容记录评论

任务描述

设 $A_n = \{a_1, a_2, a_3, \dots, a_n\}$ 是一个包含 n 个元素的有限集合,
定义集合 $B_n = \{b_1b_2b_3 \dots b_n \mid b_i \in \{0, 1\}\}$
, 即所有长度为 n 的 01 字符串的集合。
试建立从幂集 $P(A_n)$ 到 B_n 的一个双射, 满足:

- 对任意子集 $S \in P(A_n)$, 令
 $f(S) = b_1b_2b_3 \dots b_n$
其中
$$b_i = \begin{cases} 1, & a_i \in S \\ 0, & a_i \notin S \end{cases}, \quad i = 1, 2, \dots, n$$

编程要求

- 第一行给定一个正整数 n , 含义如上。
- 第二行给定 n 个正整数, 表示集合 A_n 中的元素。
- 你需要构造 $P(A_n) \rightarrow B_n$ 的一个双射, 输出 n 行, 每行形如:
 $\{a_i, a_j, \dots, a_k\} \rightarrow S$
表示 $P(A_n)$ 到 B_n 的一个双射, 其中 S 是一个长度为 n 的 01 字符串。

说点什么

代码文件

```
1 #include <stdio.h>
2 #include <math.h> // For pow function, or
  can use bit shift (1 << n)
3
4 int main() {
5     // n: 集合 An 的大小
6     int n;
7     scanf("%d", &n);
8
9     // set_A: 存储集合 An 的 n 个元素
10    int set_A[12];
11    for (int i = 0; i < n; i++) {
12        scanf("%d", &set_A[i]);
13    }
14
15    //***** Begin *****/
16    // 在此区域编写核心代码
17    // 1. 循环 2^n 次, 每次循环代表 P(An)
```

测试结果

20/20 全部通过

▶ 测试集1	消耗内存34.43MB	代码执行时长: 0.07秒	✓
▶ 测试集2	消耗内存34.43MB	代码执行时长: 0.06秒	✓
▶ 测试集3	消耗内存34.43MB	代码执行时长: 0.05秒	✓
▶ 测试集4	消耗内存34.43MB	代码执行时长: 0.05秒	✓
▶ 测试集5	消耗内存34.43MB	代码执行时长: 0.09秒	✓
▶ 测试集6	消耗内存34.43MB	代码执行时长: 0.05秒	✓
▶ 测试集7	消耗内存34.43MB	代码执行时长: 0.05秒	✓

本关最大执行时间: 3秒 本次评测耗时(编译...)

上一关

自测运行

43

3 初等数论及组合数学

3.1 实验目的

本实验旨在掌握初等数论和组合数学中的基本理论与典型算法，理解模运算、素数判定、最大公约数、扩展欧几里得算法、中国剩余定理以及 RSA 等密码学基础知识，同时掌握组合计数、排列组合等基本方法。通过程序实现各类数论与组合算法，提高数学建模能力、算法设计能力以及利用数学方法解决实际问题的能力。

3.2 实验内容

3.2.1 初等数论

(1) 除法和模运算——判断是否同余

本关任务：编写一个程序，计算两个大整数（最多可达 10^5 位）是否模 m 同余。

(2) 除法和模运算——二进制模幂算法

本关任务：在编写一个快速计算一个数在模运算下的若干次方的程序。例如，计算 $a^n \pmod{m}$

(3) 素数和最大公因数——实现试除法判断素数

本关任务：编写一个实现试除法判断素数的程序。

(4) 素数和最大公因数——唯一分解定理

本关任务：找出给定整数的素因数分解。

(5) 素数和最大公因数——扩展欧几里得算法

本关任务：找到最大公约数（GCD）的一组线性表示，即找到整数 x, y 使得 $\gcd(a, b) = ax + by$ 。

(6) 同余方程——求一个整数模 m 的逆元

本关任务：编写一个程序，求一个整数 a 模 m 的乘法逆元。

(7) 同余方程——求解线性同余方程 $ax \equiv b \pmod{m}$

本关任务：编写一个程序，求解形如 $ax \equiv b \pmod{m}$ 的线性同余方程。

说明：题目保证 a 和 m 互质，因此可以直接利用乘法逆元求解

(8) 同余方程——使用中国剩余定理求解同余方程组

本关任务：编写一个程序，使用中国剩余定理（CRT）求解一元线性同余方程组。

该方程组形如：

$$\begin{cases} x \equiv a_1(\text{mod}m_1) \\ x \equiv a_2(\text{mod}m_2) \\ \vdots \\ x \equiv a_n(\text{mod}m_n) \end{cases}$$

(9) 密码学——仿射密码

本关任务：编写一个程序来实现仿射密码的加密过程。

仿射密码是移位密码（凯撒密码）的一种泛化。凯撒密码的加密函数是 $C = (P + k)(\text{mod}26)$ ，而仿射密码采用了更一般的线性函数： $C = (a \cdot P + b)(\text{mod}26)$ 。

(10) 密码学——寻找私钥

本关任务：编写一个程序来模拟 RSA 密码系统中私钥的生成。

RSA 是一种应用最广泛的公钥密码系统。它使用一对密钥：

- i. 公钥 (n, e) ：用于加密，可以公开给任何人。
- ii. 私钥 (n, d) ：用于解密，必须由持有者严格保密。

(11) 密码学——RSA 加密

本关任务：编写一个程序来实现 RSA 密码系统的加密过程。

RSA 加密是公钥密码学的核心操作。持有者公开其公钥 (n, e) ，任何人都可以使用这个公钥将明文消息 M 加密为密文 C 。

3.2.2 组合数学

(1) 计数基础——网站用户名

本关任务：应用组合数学中的加法法则和乘法法则，计算一个网站所有可能的合法用户名总数。

题目背景：一个网站的用户名设计规则如下：

规则一：可以由4个小写英文字母组成。

规则二：或者，可以由1个大写英文字母开头，后跟2个数字组成。

请问总共有多少种合法的用户名？

(2) 计数基础——相邻的约束

本关任务：你需要构造一个长度为 N 的密码，密码的每一位都是0-9的数字。但是，存在 K 对“不吉利”的相邻数字。例如，若(1,3)是不吉利的，则密码中不允许出现'1'后面紧跟着'3'的情况。请你计算可以构造出多少种合法的 N 位密码。

由于结果可能很大，请对 $10^9 + 7$ 取模。

(3) 巢鸽原理——采摘果实

本关任务：应用广义鸽巢原理解决一个问题。

题目背景：果园里有 K 种果树，每种果树上的果实都取之不尽。为了保证你采摘的果实中，至少有 M 个是属于同一种类的，你最少需要采摘多少个果实？

(4) 巢鸽原理——子集和

本关任务：利用鸽巢原理和前缀和技巧，解决“子集和”。

题目背景：给定一个包含 N 个正整数的数组 A 。请找出一个非空子集，使得该子集中所有元素的和能被 N 整除。如果存在多个解，输出任意一个即可。题目保证有解。

(5) 巢鸽原理——平面上的整中点

本关任务：利用鸽巢原理，解决“平面上的整中点”问题。

题目背景：在二维平面上给定 N 个坐标都是整数的点。请找出一对点，使得它们的中点坐标也都是整数。如果存在多组解，输出任意一组即可。题目保证 $N \geq 5$ 。

(6) 排列与组合——饼干选购

本关任务：应用可重复组合的知识，解决“饼干选购”问题。

题目背景：一家饼干店有 N 种不同口味的饼干，每种口味的饼干数量都足够多。你需要购买 K 块饼干，问总共有多少种不同的购买方案？（不考虑选取饼干的顺序）。

(7) 排列与组合——单词重排

本关任务：应用组合数学中带有多重集的排列知识，解决“单词重排”问题。

题目背景：给定一个可能包含重复字母的字符串（例如 "SUCCESS"），计算重新排列这个字符串中的所有字母可以组成多少个不同的新字符串。

(8) 生成函数——游戏

本关任务：小明和小红是两个喜欢收集奇特石头的朋友。小明有一个袋子，里面装着 N 块石头，每块石头上都标有一个“魔力值”。小红也有一个袋子，里面装着 M 块石头，同样每块石头有自己的魔力值。

现在，他们俩要玩一个游戏：各自从自己的袋子里随机取出一块石头。一个组合的“总魔力值”等于两人取出的石头魔力值之和。

请你计算，所有可能的“总魔力值”及其出现的组合数。

(9) 生成函数——特殊的字符串

本关任务：你需要使用字符 ' A ', ' B ', ' C ' 来构造一个长度为 N 的字符串，要求构造出的字符串中：

- ✧ ' A '必须出现偶数次。
- ✧ ' B '必须出现奇数次。
- ✧ ' C '的出现次数没有限制。

请计算总共有多少种满足条件的字符串，由于结果可能很大，请对 $10^9 + 7$ 取模。

3.3 实验过程

3.3.1 初等数论

(1) 除法和模运算——判断是否同余：

程序设计思想：

程序设计思想：首先读取两个大整数字符串 a 、 b 以及模数 m ，定义两个整型变量 $remainder_a$ 和 $remainder_b$ 分别存储两数对 m 取模的余数，初始值均为0。

接着利用模运算的分配律性质逐位计算余数，遍历字符串 a 的每个字符，通过公式 $remainder_a = (remainder_a * 10 + (digit - '0')) \% m$ 更新余数，将字符转换为数字后累加并即时取模，避免大整数溢出；对字符串 b 采用相同逻辑计算 $remainder_b$ 。

最后比较两个余数是否相等，若 $remainder_a == remainder_b$ 则说明两数对模 m 同余，输出" yes "，否则输出" no "。

时间复杂度分析： $O(n)$ ，其中 n 为两个输入字符串长度的最大值。算法需要

分别遍历两个字符串的每一位数字进行模运算，每次运算为常数时间，总操作次数与字符串长度成线性关系。

空间复杂度分析： $O(1)$ ，仅使用了常数个整型变量存储余数和中间计算结果，不依赖输入规模的额外空间，字符串输入本身不计入算法额外空间开销。

彭婉茹49826

除法和模运算
实验总用时: 00:11:31

更多操作

第1关: 判断是否同余100

学习内容记录评论

- 任务描述
- 相关知识
 - 算法原理
- 编程要求
- 测试说明

任务描述

本关任务：编写一个程序，计算两个大整数（最多可达 10^5 位）是否模 m 同余。

相关知识

为了完成本关任务，你需要掌握：

- 模运算的相关知识：
 - 同余的定义与等价条件：
 - 两个整数 a 和 b 称为模 m 同余，当且仅当它们除以 m 的余数相同。数学上记为 $a \equiv b \pmod{m}$ 。
 - 模运算的封闭性（对于加法、减法、乘法）：
 - 加法： $(X + Y) \pmod{m} = ((X \pmod{m}) + (Y \pmod{m})) \pmod{m}$
 - 减法： $(X - Y)$

代码文件

```
1 #include <iostream>
2 #include <string>
3
4 using namespace std;
5
6 int main() {
7     string a, b;
8     long long m;
9
10    if (!(cin >> a >> b >> m)) {
11        return 0;
12    }
```

测试结果

5/5 全部通过

测试集1	消耗内存46.22MB	代码执行时长: 0.21秒	✓
测试集2	消耗内存46.22MB	代码执行时长: 0.07秒	✓
测试集3	消耗内存46.22MB	代码执行时长: 0.06秒	✓
测试集4	消耗内存46.22MB	代码执行时长: 0.08秒	✓
测试集5	消耗内存46.22MB	代码执行时长: 0.07秒	✓

本关最大执行时间: 3秒 本次评测耗时(编译... 下一关 自测运行

(2) 除法和模运算——二进制模幂算法：

程序设计思想：

首先读取底数 a 、指数 n 和模数 m 三个长整型变量，定义结果变量 ans 并初始化为 $1 \% m$ ，处理 $m = 1$ 时结果恒为0的边界情况。同时将底数 $base$ 初始化为 $(a \% m + m) \% m$ ，确保底数在模 m 意义下为非负值。

接着采用快速幂算法迭代计算，当指数 n 大于0时循环执行：若 n 的二进制最

低位为1（即 $n \& 1$ 为真），则将当前 $base$ 乘入结果 ans 并取模，核心代码为 $ans = (ans * base) \% m$ ；随后将 $base$ 平方并取模更新为下一轮的值，即 $base = (base * base) \% m$ ；最后将 n 右移一位等价于除以2，即 $n = n >> 1$ 。通过这种二进制分解的方式，将指数运算的乘法次数从线性级优化为对数级。

最后输出计算得到的 ans 值，即为 $a^n \bmod m$ 的最终结果。

时间复杂度分析： $O(\log n)$ ，其中 n 为指数。快速幂算法通过每次将指数折半的方式迭代，循环执行次数为 $\lceil \log_2 n \rceil + 1$ 次，每次循环内的乘法与取模运算均为常数时间。

空间复杂度分析： $O(1)$ ，仅使用了常数个长整型变量存储底数、指数、模数、结果及中间状态，不依赖输入规模的额外空间开销。

彭婉茹49826

第2关：二进制模幂算法200

学习内容记录评论

- 任务描述
- 相关知识
 - 二进制模幂算法
 - 算法原理
- 编程要求
- 测试说明

任务描述

本关任务：编写一个快速计算一个数在模运算下的若干次方的程序。例如，计算 $a^n \pmod m$ 。

相关知识

为了完成本关任务，你需要掌握：二进制模幂算法，又称快速幂。

二进制模幂算法

二进制模幂算法（Binary Exponentiation），又称快速幂（Fast Powering Algorithm），是一种高效计算幂运算 a^n 或模幂运算 $a^n \pmod m$ 的算法。当指数 n 较大时，朴素的 $n - 1$ 次乘法（时间复杂度为 $\Theta(n)$ ）会非常耗时。快速幂算法利用指数的二进制表示，可以将时

除法和模运算
实验总用时：00:14:44

代码文件

```
1 #include <iostream>
2
3 using namespace std;
4
5 int main() {
6     long long a, n, m;
7
8     if (!(cin >> a >> n >> m)) {
9         return 0;
10    }
11
12    long long ans;
13
14    // ***** Begin ***** //
15    //如果初始指数 n=0，循环不执行，ans 保持
16    //为 1，所以这里需要初始化
17    ans = 1 % m;
18    // 将底数 base 初始化为 a % m
```

测试结果

5/5 全部通过

测试集1	消耗内存54.1MB	代码执行时长：0.03秒	✓
测试集2	消耗内存54.1MB	代码执行时长：0.02秒	✓
测试集3	消耗内存54.1MB	代码执行时长：0.02秒	✓
测试集4	消耗内存54.1MB	代码执行时长：0.02秒	✓
测试集5	消耗内存54.1MB	代码执行时长：0.02秒	✓

本关最大执行时间：3秒 本次评测耗时(编译... 上一关 自测运行

49

(3) 素数和最大公约数——实现试除法判断素数:

程序设计思想:

首先通过标准输入读取待判断的长整型数值 n ，该数值可能达到较大范围，因此使用`longlong`类型存储以避免溢出。接着定义布尔型标志变量`is_prime`并初始化为`true`，用于标记该数是否满足素数的定义，即大于1且只能被1和自身整除的自然数。

接着进入核心判断逻辑，首先处理特殊情况以确保算法的正确性和效率。当 $n \leq 1$ 时，根据素数定义直接判定为非素数，将`is_prime`置为`false`；当 n 等于2时，作为最小的素数且唯一的偶素数，保持`is_prime`为`true`；当 n 为大于2的偶数时，必然能被2整除，因此不是素数，将`is_prime`置为`false`。这些预处理步骤能够快速排除大量非素数情况，避免不必要的循环计算。

对于剩余的奇数情况，采用试除法进行因子检测。首先计算判断上限 $limit = \sqrt{n}$ ，基于数学原理：若 n 存在大于 $\sqrt{n}$ 的因子，则必然对应存在一个小于 $\sqrt{n}$ 的因子，因此只需检查到 $\sqrt{n}$ 即可。然后通过`for`循环从3开始以步长2遍历所有奇数，跳过偶数因子以减少一半的计算量。在循环体内通过取模运算检查当前奇数 i 是否能整除 n ，核心判断代码为

```
if(n % i == 0) { is_prime = false; break; }
```

一旦发现因子立即将标志位置为`false`并使用`break`语句跳出循环，实现提前终止以优化性能。

最后根据`is_prime`标志位的最终状态输出判定结果，若标志位仍为`true`则说明遍历完所有可能因子均未发现整除情况，确认 n 为素数，输出`"yes"`；否则输出`"no"`表示 n 不是素数。整个算法通过数学优化和提前终止策略，在保证正确性的同时显著提升了判断效率。

时间复杂度分析： $O(\sqrt{n})$ 其中 n 为待判断的数值。算法的核心开销在于试除循环，需要遍历从3到 $\sqrt{n}$ 的所有奇数进行整除判断，最坏情况下（即 n 为素数时）需要执行约 $\sqrt{n}/2$ 次取模运算。由于每次取模运算为常数时间操作，因此整体时间复杂度为平方根级别，能够高效处理较大范围的素数判断需求。

空间复杂度分析： $O(1)$ ，算法仅使用了常数个变量存储输入数值 n 、循环索

引 i 、判断上限 $limit$ 以及标志位 is_prime ，所有变量均为基本数据类型，不依赖输入规模的额外空间开销。无论输入数值多大，额外内存占用始终保持固定，因此空间复杂度为常数级别。

彭婉茹49826

第1关：实现试除法判...100

学习内容记录评论

理解的一种。当我们需要判断一个整数 n 是否为素数时，试除法是最基本的方法。

定义：
素数（或质数）是指在大于 1 的自然数中，除了 1 和它本身以外不再有其他因数的自然数。

基本思想：
要判断一个整数 n 是否为素数，我们可以尝试用从 2 开始到 $n - 1$ 的所有整数去除 n 。如果这些数中任意一个能够整除 n ，那么 n 就不是素数，否则 n 就是素数。

优化：
1. 特殊情况处理：

- 如果 $n \leq 1$ ，则 n 不是素数。
- 如果 $n = 2$ ，则 n 是素数（2 是唯一的偶素数）。
- 如果 $n > 2$ 且 n 是偶数，则 n 不是素数（因为它可以被 2 整除）。

2. 减少试除范围：
我们实际上不需要检查到 $n - 1$ 。如果 n 有一个大于 $\sqrt{n}$ 的因数 a ，那么它必然也有一个小于 $\sqrt{n}$ 的因数 b ，因为 $n = a \times b$ 。如果 $a > \sqrt{n}$ 且

素数和最大公因数
实验总用时: 00:10:18

代码文件

```
37  
38  
39  
40  
41 // ***** End *****  
42  
43 if (is_prime) {  
44     cout << "yes" << endl;  
45 } else {  
46     cout << "no" << endl;  
47 }  
48  
49 return 0;  
50 }
```

测试结果

5/5 全部通过

▶ 测试集1	消耗内存143.71MB	代码执行时长: 0.02秒	✓
▶ 测试集2	消耗内存143.71MB	代码执行时长: 0.02秒	✓
▶ 测试集3	消耗内存143.71MB	代码执行时长: 0.04秒	✓
▶ 测试集4	消耗内存143.71MB	代码执行时长: 0.02秒	✓
▶ 测试集5	消耗内存143.71MB	代码执行时长: 0.02秒	✓

本关最大执行时间: 3秒 本次评测耗时(编译... 下一关 自测运行

（4）素数和最大公约数——唯一分解定理：

程序设计思想：

首先读取待分解的正整数 n ，随后采用试除法进行质因数分解。程序通过 for 循环从整数2开始遍历至 n 的平方根，对于每个遍历到的 i ，利用 $while$ 循环统计 i 能整除 n 的次数 cnt ，并在每次整除后将 n 除以 i 以不断缩小待分解的规模。若当前 i 的计数 cnt 大于 0，则说明 i 是 n 的一个质因数，程序直接输出该质因数及其指数。

当外层循环结束后，若剩余的 n 仍大于 1，则表明原数中包含一个大于初始平方根的质因数，此时直接输出该剩余值及其指数 1。整个过程通过不断约分并严格限制遍历范围，高效且完整地提取了所有质因数及其幂次。

时间复杂度分析： $O(\sqrt{n})$ ，其中 n 为输入的正整数。算法的外层循环最多执行至 $\sqrt{n}$ 次，内部的 while 循环通过连续整除迅速减小 n 的值，使得总操作次数被严格限制在 $O(\sqrt{n})$ 范围内，即使在 n 本身为质数的最坏情况下也能保持稳定的执行效率。

空间复杂度分析： $O(1)$ ，程序仅使用常数个整型变量存储输入值、循环索引、计数器和中间状态，不依赖输入规模的额外内存分配，空间开销始终保持恒定。

彭婉茹

49826

素数和最大公因数

实验总用时: 00:10:49

更多操作

⏻

第2关：唯一分解定理

100

学习内容

记录

评论

任务描述

相关知识

编程要求

测试说明

任务描述

本关任务：找出给定整数的素因数分解

相关知识

为了完成本关任务，你需要掌握：唯一分解定理

算法原理

唯一分解定理

唯一分解定理指出：任何一个大于1的自然数 N ，可以唯一地分解成有限个素数的乘积，即 $N = P_1^{a_1} \times P_2^{a_2} \times \dots \times P_k^{a_k}$ ，这里 $P_1 < P_2 < \dots < P_k$ 均为素数， a_i 均为正整数。这种分解在不考虑排列次序的情况下是唯一的。

例如：

说点什么

👍

代码文件

```
1 #include <iostream>
2 #include <algorithm>
3 #include <cmath>
4
5 using namespace std;
6
7 int main() {
8     int n;
9     cin >> n;
10
11     /****** Begin *****/
12     for(int i=2; i*i<=n; i++)
13     {
```

测试结果

5/5 全部通过

测试集1

消耗内存49.43MB

代码执行时长: 0.08秒

✅

测试集2

消耗内存49.43MB

代码执行时长: 0.05秒

✅

测试集3

消耗内存49.43MB

代码执行时长: 0.07秒

✅

测试集4

消耗内存49.43MB

代码执行时长: 0.07秒

✅

测试集5

消耗内存49.43MB

代码执行时长: 0.06秒

✅

本关最大执行时间: 3秒

本次评测耗时(毫秒)

上一关

下一关

自测运行

(5) 素数和最大公约数——扩展欧几里得算法：

程序设计思想：

程序设计思想：首先读取两个长整型整数 a 和 b 作为输入，调用扩展欧几里得算法函数 `exgcd` 以求解贝祖等式 $ax + by = \gcd(a, b)$ 的整数解。算法采用递归分治策略，首先处理递归结束情况，当参数 b 等于 0 时，最大公约数即为 a ，此时直接赋值 $x = 1$ 、 $y = 0$ 并返回 a ，满足等式 $a \times 1 + 0 \times 0 = a$ 。若 b 不为 0，则递归调用 `exgcd(b, a % b, x1, y1)` 获取下一层的解 $x1$ 、 $y1$ 及公约数 d 。接着利用数论中的递推恒等式进行回溯更新，将当前层的 x ，赋值为递归返回的 $y1$ ，将 y 赋值为 $x1$ 减去 (a/b) 与 $y1$ 的乘积，核心代码为 $x = y1; y = x1 - (a/b) * y1;$ 。该过程通过不断交换参数并取模使问题规模迅速收敛，最终逐层返回计算出一组满足原方程的特解 (x, y) 及最大公约数 d 。最后在主函数中按顺序输出 x 、 y 和 d 的值，完成算法求解与结果展示。

时间复杂度分析： $O(\log(\min(a, b)))$ 。扩展欧几里得算法的递归深度与标准欧几里得算法一致，每次递归调用都会使较大参数至少减半，基于斐波那契数列最坏情况性质，递归次数与两数次值的对数成正比。每层递归内的算术运算与赋值操作均为常数时间，因此整体时间复杂度为对数级别，能够高效处理大整数求解。

空间复杂度： $O(\log(\min(a, b)))$ 。由于程序采用递归实现，系统调用栈的深度与递归次数相同，每层栈帧仅保存常数个局部变量与返回地址，不额外分配动态内存。因此空间复杂度与时间复杂度保持一致，为对数级别。

彭婉茹
 49826

素数和最大公因数

实验总用时: 00:08:56

更多操作

第3关: 扩展欧几里得... 100

学习内容 记录 评论

- 任务描述
- 相关知识
 - 算法原理
- 编程要求
- 测试说明

任务描述

本关任务: 找到最大公约数 (GCD) 的一组线性表示, 即找到整数 x, y 使得 $\gcd(a, b) = ax + by$ 。

相关知识

为了完成本关任务, 你需要掌握: 扩展欧几里得算法。

算法原理

扩展欧几里得算法不仅计算两个整数 a 和 b 的最大公约数 $\gcd(a, b)$, 还能找到一对整数 x 和 y , 使得 $a \cdot x + b \cdot y = \gcd(a, b)$ 。

递归推导:

假设我们要求解 $a \cdot x + b \cdot y = \gcd(a, b)$

代码文件

```

1 #include <iostream>
2
3 using namespace std;
4
5 long long exgcd(long long a, long long b,
6 long long &x, long long &y) {
7     /******* Begin *****/
8     if(b == 0)
9     {
10         x = 1;
11         y = 0;
12         return a;
13     }

```

测试结果

8/8 全部通过

▶ 测试集1	消耗内存39.52MB	代码执行时长: 0.08秒	✓
▶ 测试集2	消耗内存39.52MB	代码执行时长: 0.08秒	✓
▶ 测试集3	消耗内存39.52MB	代码执行时长: 0.05秒	✓
▶ 测试集4	消耗内存39.52MB	代码执行时长: 0.06秒	✓
▶ 测试集5	消耗内存39.52MB	代码执行时长: 0.06秒	✓
▶ 测试集6	消耗内存39.52MB	代码执行时长: 0.04秒	✓
▶ 测试集7	消耗内存39.52MB	代码执行时长: 0.05秒	✓
▶ 测试集8	消耗内存39.52MB	代码执行时长: 0.05秒	✓

本关最大执行时间: 3秒 本次评测耗时(编译)
 上一关
自测运行

(6) 同余方程——求一个整数模 m 的逆元:

程序设计思想:

首先读取输入的整数 a 和模数 m , 调用扩展欧几里得算法函数 $exgcd$ 求解线性丢番图方程 $ax + my = \gcd(a, m)$ 的整数解。算法采用递归分治策略, 当递归至 b 等于 0 的边界条件时, 直接返回最大公约数 a , 并赋值 x 为 1、 y 为 0。在递归回溯阶段, 利用下一层返回的解 x_1 和 y_1 , 通过数学恒等式 $x = y_1$ 与 $y = x_1 - (a/b) * y_1$ 逐层更新当前层的系数, 从而自底向上构造出原方程的一组特解。

接着将求得的特解 x 用于计算模逆元, 根据同余理论, 方程 $ax + my = 1$ 可转化为 $ax \equiv 1 \pmod{m}$, 此时 x 即为 a 模 m 的乘法逆元。程序通过调用 $exgcd(a, m, x, y)$ 获取满足条件的 x 值, 并采用表达式 $(x \% m + m) \% m$ 对结果进

行标准化处理。该操作利用双重取模确保无论递归求得的 x 为正数或负数，最终返回值均严格落在 $[0, m - 1]$ 的标准剩余系范围内，从而保证输出的逆元符合数学定义。

最后主函数输出经过规范化处理的逆元结果，完成模逆元的求解。整体流程将扩展欧几里得算法与同余方程理论紧密结合，通过递归求解与结果映射高效得出答案。

时间复杂度分析： $O(\log(\min(a, m)))$ 。扩展欧几里得算法的递归深度与辗转相除法一致，每次递归调用都会使参数规模至少减半，基于数论中的拉梅定理，递归次数与两数值的对数成正比。每层递归内的乘除取模及赋值运算均为常数时间复杂度，因此整体执行效率极高，能够迅速处理大整数模逆元计算。

空间复杂度分析： $O(\log(\min(a, m)))$ 。由于算法采用递归实现，系统调用栈的深度与递归次数保持一致，每层栈帧仅需保存常数个局部变量与函数返回地址，不额外分配动态堆内存。因此空间开销随输入规模呈对数级增长。

彭婉茹

49826

同余方程

实验总用时: 00:07:20

更多操作

电源

第1关: 求一个整数模 ...

100

学习内容

记录

评论

- 任务描述
- 相关知识
 - 模逆元的定义
 - 算法原理
- 编程要求
- 测试说明

任务描述

本关任务: 编写一个程序, 求一个整数 a 模 m 的乘法逆元。

相关知识

为了完成本关任务, 你需要掌握: **模逆元** 的定义和**扩展欧几里得算法**。

模逆元的定义

如果两个正整数 a 和 m 互质, 那么一定存在一个整数 x , 使得 $(a \cdot x) \pmod m = 1$ 。这个整数 x 就被称为 a 模 m 的乘法逆元。通常, 我们关注的是在 $[1, m - 1]$ 范围内的那个最小正整数解。

补充说明: a 模 m 的乘法逆元存在的**充要条件**是 a 和 m 互质, 即它们的最大公约数 $\gcd(a, m) = 1$ 。

代码文件

```

1 #include <iostream>
2
3 using namespace std;
4
5 long long exgcd(long long a, long long b,
6 long long &x, long long &y) {
7     /***** Begin *****/
8     if(b == 0)
9     {
10         x = 1;
11         y = 0;
12         return a;
13     }
14     long long x1, y1;

```

测试结果

8/8 全部通过

测试集1	消耗内存57.16MB	代码执行时长: 0.09秒	通过
测试集2	消耗内存57.16MB	代码执行时长: 0.06秒	通过
测试集3	消耗内存57.16MB	代码执行时长: 0.04秒	通过
测试集4	消耗内存57.16MB	代码执行时长: 0.05秒	通过
测试集5	消耗内存57.16MB	代码执行时长: 0.04秒	通过
测试集6	消耗内存57.16MB	代码执行时长: 0.08秒	通过
测试集7	消耗内存57.16MB	代码执行时长: 0.04秒	通过
测试集8	消耗内存57.16MB	代码执行时长: 0.05秒	通过

本关最大执行时间: 20秒

本次评测耗时(编译)

下一关

自测运行

(7) 同余方程——求解线性同余方程 $ax \equiv b \pmod m$:

程序设计思想:

首先读取线性同余方程 $ax \equiv b \pmod m$ 的三个参数 a 、 b 、 m , 调用模逆元函数 `modInverse` 计算 a 在模 m 意义下的乘法逆元 inv 。该函数内部通过扩展欧几里得算法 `exgcd` 求解贝祖等式 $ax + my = \gcd(a, m)$ 的整数解, 当递归至 b 等于 0 时返回最大公约数并赋值 $x = 1$ 、 $y = 0$, 回溯时利用恒等式 $x = y_1$ 与 $y = x_1 - (a/b) * y_1$ 逐层更新系数, 最终得到满足条件的特解 x 。接着通过表达式 $(x \% m + m) \% m$ 将逆元标准化至 $[0, m - 1]$ 范围, 确保结果为最小正整数。

然后根据同余方程求解公式，将逆元 inv 与常数项 b 相乘并对 m 取模，计算 $x = (inv * b) \% m$ 得到原方程的解。该步骤基于数论原理：当 $gcd(a, m) = 1$ 时，方程 $ax \equiv b \pmod{m}$ 等价于 $x \equiv a^{-1} \cdot b \pmod{m}$ ，通过逆元将除法运算转化为乘法运算。最后输出计算得到的 x 值，完成线性同余方程的求解。

时间复杂度分析： $O(\log(\min(a, m)))$ 。核心开销在于扩展欧几里得算法的递归求解，递归深度与辗转相除法一致，每次递归使参数规模至少减半。逆元计算与最终乘法取模均为常数时间操作，因此整体时间复杂度为对数级别。

空间复杂度分析： $O(\log(\min(a, m)))$ 。由于扩展欧几里得算法采用递归实现，系统调用栈深度与递归次数相同，每层栈仅保存常数个局部变量与返回地址。若不考虑递归栈开销，仅使用常数个整型变量存储中间状态。

彭婉茹49826

第2关：求解线性同余...100

学习内容记录评论

任务描述

相关知识

算法原理

编程要求

测试说明

任务描述

本关任务：编写一个程序，求解形如 $ax \equiv b \pmod{m}$ 的线性同余方程。

说明：题目保证 a 和 m 互质，因此可以直接利用乘法逆元求解。

相关知识

为了完成本关任务，你需要掌握：

1. 线性同余方程的概念
2. 如何利用乘法逆元求解线性同余方程
3. 如何使用扩展欧几里得算法求解乘法逆元

算法原理

线性同余方程 $ax \equiv b \pmod{m}$ 的求解过程可以转化为以下步骤：

1. 方程变形：

同余方程

实验总用时: 00:05:22

更多操作

代码文件

```
1 #include <iostream>
2
3 using namespace std;
4
5 long long exgcd(long long a, long long b,
6 long long &x, long long &y) {
7     if (b == 0) {
8         x = 1;
9         y = 0;
10        return a;
11    }
12    long long x1, y1;
13    long long d = exgcd(b, a % b, x1, y1);
14    x = y1;
15    y = x1 - (a / b) * y1;
16    return d;
17 }
18
19 long long modInverse(long long a, long
20 long m) {
21     long long x, y;
22     exgcd(a, m, x, y);
23     // 将 x 转换为 [0, m-1] 范围内的最小正整
24     数
25     return (x % m + m) % m;
26 }
```

测试结果

6/6 全部通过

测试集1

消耗内存40.88MB 代码执行时长: 0.2秒

测试集2

消耗内存40.88MB 代码执行时长: 0.04秒

测试集3

消耗内存40.88MB 代码执行时长: 0.06秒

本关最大执行时间: 20秒 本次评测耗时: 上关 下一关 自测运

(8) 同余方程——使用中国剩余定理求解同余方程组：

程序设计思想：

首先读取同余方程组的方程数量 n ，以及每组方程的余数 $a[i]$ 和模数 $m[i]$ ，存储于两个 *vector* 数组中。

接着调用 *solveCRT* 函数求解中国剩余定理，首先计算所有模数的乘积 M ，作为最终解的周期范围。

然后初始化结果 *result* 为 0，遍历每个方程计算对应的贡献项：对于第 i 个方程，计算 $M_i = M/m[i]$ ，即除去当前模数后其余模数的乘积。

接着调用扩展欧几里得算法 *exgcd* 求解 M_i 在模 $m[i]$ 意义下的乘法逆元 ti ，通过 *exgcd* ($M_i, m[i], ti, y$) 得到满足 $M_i \cdot ti + m[i] \cdot y = 1$ 的整数解 ti ，并通过表达式 $(ti \% m[i] + m[i]) \% m[i]$ 将逆元标准化至 $[0, m[i] - 1]$ 范围。随后将当前方程的贡献 $a[i] \times M_i \times ti$ 累加至 *result* 中，核心代码为 *result* += $a[i] * M_i * ti$ 。

遍历完成后，对 *result* 执行最终标准化 $(result \% M + M) \% M$ ，确保输出结果为非负最小整数解。最后主函数输出计算得到的 x 值，即为满足所有同余方程的最小非负整数解。

时间复杂度分析： $O(n \times \log(\max(m)))$ ，其中 n 为方程数量， $\max(m)$ 为模数中的最大值。计算总乘积 M 需遍历 n 个模数，时间开销为 $O(n)$ ；对每个方程求解逆元时调用扩展欧几里得算法，单次复杂度为 $O(\log(\max(m)))$ ，共执行 n 次，总开销为 $O(n \times \log(\max(m)))$ 。其余累加与取模操作均为常数时间，因此整体时间复杂度由逆元求解主导。

空间复杂度分析： $O(n)$ ，主要用于存储 n 个方程的余数数组 a 和模数数组 m ，使用 *vector* 动态分配空间。扩展欧几里得算法采用递归实现，递归栈深度为 $O(\log(\max(m)))$ ，但相对于数组存储可忽略不计。

彭婉茹

49826

同余方程

实验总用时: 00:10:31

更多操作

电源

第3关: 使用中国剩余...

100

学习内容

记录

评论

- 任务描述
- 相关知识
 - 算法原理
- 编程要求
- 测试说明

任务描述

本关任务: 编写一个程序, 使用**中国剩余定理 (CRT)** 求解一元线性同余方程组。

该方程组形如:

$$\begin{aligned} x &\equiv a_1 \pmod{m_1} \\ x &\equiv a_2 \pmod{m_2} \\ &\vdots \\ x &\equiv a_n \pmod{m_n} \end{aligned}$$

相关知识

为了完成本关任务, 你需要掌握:

- 中国剩余定理:** 理解其应用场景和核心思想。
- 扩展欧几里得算法:** 用于求解求解模逆元, 这是中国剩余定理中的关键一步。

算法原理

代码文件

```

1 #include <iostream>
2 #include <vector>
3
4 using namespace std;
5
6 long long exgcd(long long a, long long b,
7 long long &x, long long &y) {
8     /****** Begin *****/
9     if (b == 0) {
10         x = 1;
11         y = 0;
12         return a;
13     }
14     long long x1, y1;
15     long long d = exgcd(b, a % b, x1, y1);
16     x = y1;
17     y = x1 - (a / b) * y1;
18     return d;
19 }
20
21 int main() {
22     int n;
23     cin >> n;
24     vector<long long> a(n);
25     vector<long long> m(n);
26     for (int i = 0; i < n; i++) {
27         cin >> a[i];
28         cin >> m[i];
29     }
30     long long x = 0;
31     for (int i = 0; i < n; i++) {
32         long long ai = a[i];
33         long long mi = m[i];
34         long long xi, yi;
35         long long di = exgcd(mi, mprod, xi, yi);
36         long long ti = ai * yi * mprod / di;
37         x = (x + ti) % mprod;
38     }
39     cout << x << endl;
40     return 0;
41 }

```

测试结果

6/6 全部通过

测试集	消耗内存	代码执行时长	结果
测试集1	50.67MB	0.33秒	通过
测试集2	50.67MB	0.05秒	通过
测试集3	50.67MB	0.06秒	通过
测试集4	50.67MB	0.05秒	通过
测试集5	50.67MB	0.04秒	通过
测试集6	50.67MB	0.06秒	通过

本关最大执行时间: 20秒 本次评测耗时(编译)

[上一关](#)
[自测运行](#)

(8) 密码学——仿射密码:

程序设计思想:

首先遍历字符串 P 的每个字符进行加密处理, 对于当前字符 ch , 首先判断其是否为大写字母 (即 ch 在 'A' 到 'Z' 范围内)。若满足条件, 则将其映射为 0 到 25 的数字 $p = ch - 'A'$, 然后应用仿射变换公式 $c = (a * p + b) \% 26$ 计算密文对应的数字, 核心代码为 `int c = (a * p + b) % 26`。最后将计算结果映射回大写字母 `'A' + c` 并写入密文字符串 C 的对应位置。若字符不是大写字母 (如空格、标点或小写字母), 则跳过变换直接保留原字符, 从而实现仅对大写字母加密而保持其他字符不变的功能。

最后输出处理完成的密文字符串 C ，完成仿射密码的加密过程。整个算法通过字符映射与线性变换相结合，实现了经典的单表替换加密方案。

时间复杂度分析： $O(n)$ ，其中 n 为输入字符串的长度。算法需要遍历字符串的每个字符进行一次条件判断与可能的算术运算，每次操作均为常数时间，总执行次数与字符串长度成线性关系。

空间复杂度分析： $O(n)$ ，主要用于存储明文字符串 P 和密文字符串 C ，两者长度均为 L 。除此之外仅使用了常数个整型和字符型变量存储密钥及中间计算结果，额外空间开销为 $O(1)$ 。

彭婉茹49826

第1关：仿射密码100

学习内容记录评论

- 任务描述
- 相关知识
 - 仿射密码原理
 - 示例
- 编程要求
- 测试说明
 - 示例 1:
 - 示例 2:
 - 示例 3:

任务描述

本关任务：编写一个程序来实现仿射密码的加密过程。

仿射密码是移位密码（凯撒密码）的一种泛化。凯撒密码的加密函数是 $C = (P + k) \pmod{26}$ ，而仿射密码采用了更一般的线性函数： $C = (a \cdot P + b) \pmod{26}$ 。

相关知识

为了完成本关任务，你需要掌握：仿射密码的加密原理和模运算。

仿射密码原理

说点什么

密码学

实验总用时: 00:07:18

代码文件

```
1 #include <bits/stdc++.h>
2 using namespace std;
3
4 int main() {
5     ios::sync_with_stdio(false);
6     cin.tie(nullptr);
7
8     int a, b;
9     cin >> a >> b;
10    cin.ignore
11    (numeric_limits<streamsize>::max(), '\n');
12    string P;
```

测试结果

10/10 全部通过

▶ 测试集1	消耗内存146.5MB	代码执行时长: 0.06秒	✓
▶ 测试集2	消耗内存146.5MB	代码执行时长: 0.07秒	✓
▶ 测试集3	消耗内存146.5MB	代码执行时长: 0.04秒	✓
▶ 测试集4	消耗内存146.5MB	代码执行时长: 0.04秒	✓
▶ 测试集5	消耗内存146.5MB	代码执行时长: 0.04秒	✓
▶ 测试集6	消耗内存146.5MB	代码执行时长: 0.07秒	✓
▶ 测试集7	消耗内存146.5MB	代码执行时长: 0.04秒	✓
▶ 测试集8	消耗内存146.5MB	代码执行时长: 0.09秒	✓
▶ 测试集9	消耗内存146.5MB	代码执行时长: 0.04秒	✓

本关最大执行时间: 20秒 本次评测耗时(编译)

下一关

自测运行

(9) 密码学——寻找私钥：

程序设计思想：

首先读取两个大素数 p 、 q 以及公钥指数 e ，根据 RSA 算法原理计算欧拉函数 $\varphi = (p - 1) \times (q - 1)$ ，该值表示模 $n = p \times q$ 下与 n 互质的正整数个数，是求解私钥的关键参数。

接着调用扩展欧几里得算法函数 *exgcd* 求解贝祖等式 $e \cdot x + \varphi \cdot y = \gcd(e, \varphi)$ 的整数解。算法采用递归分治策略，当递归至参数 b 等于 0 的边界条件时，直接返回最大公约数 a 并赋值 $x = 1$ 、 $y = 0$ 。在回溯阶段利用数学恒等式 $x = y_1$ 与 $y = x_1 - (a/b) \cdot y_1$ 逐层更新系数，核心代码为 $x = y_1$; $y = x_1 - (a/b) * y_1$ 。由于 e 与 φ 互质，递归返回的 x 即为 e 在模 φ 意义下的乘法逆元候选值。随后通过表达式 $d = (x \% \varphi + \varphi) \% \varphi$ 对结果进行标准化处理，利用双重取模确保无论递归求得的 x 为正数或负数，最终私钥 d 均严格落在 $[0, \varphi - 1]$ 的标准剩余系范围内，满足 $1 < d < \varphi$ 且 $e \cdot d \equiv 1 \pmod{\varphi}$ 的数学要求。

最后输出计算得到的私钥 d 值，完成 RSA 密钥对中解密指数的求解。整个流程将数论中的扩展欧几里得算法与同余方程理论紧密结合，通过递归求解与结果映射高效得出符合密码学要求的私钥参数。

时间复杂度分析： $O(\log(\min(e, \varphi)))$ 。扩展欧几里得算法的递归深度与辗转相除法一致，每次递归调用都会使参数规模至少减半，基于拉梅定理递归次数与两数值的对数成正比。每层递归内的乘除取模及赋值运算均为常数时间，因此整体时间复杂度为对数级别，能够高效处理大整数模逆元计算。

空间复杂度分析： $O(\log(\min(e, \varphi)))$ 。由于算法采用递归实现，系统调用栈的深度与递归次数相同，每层栈帧仅保存常数个局部变量与返回地址，不额外分配动态内存。因此空间复杂度与时间复杂度保持一致，为对数级别。

彭婉茹

49826

密码学

实验总用时: 00:06:40

更多操作

第2关: 寻找私钥

100

学习内容

记录

评论

- 任务描述
- 相关知识
 - RSA 密钥生成原理
 - 核心挑战: 求解私钥 d
- 编程要求
- 测试说明
 - 示例 1:
 - 示例 2:
 - 示例 3:

任务描述

本关任务: 编写一个程序来模拟 RSA 密码系统中**私钥**的生成。

RSA 是一种应用最广泛的公钥密码系统。它使用一对密钥:

- 公钥 (n, e)** : 用于加密, 可以公开给任何人。
- 私钥 (n, d)** : 用于解密, 必须由持有者严格保密。

本关的任务就是根据生成公钥的参数, 反向计算出对应的私钥 d 。

相关知识

为了完成本关任务, 你需要掌握: RSA 密

代码文件

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 using int64 = long long;
5
6 // 扩展欧几里得: 求 gcd(a,b), 并找到 x,y 使得 a*x + b*y = gcd(a,b)
7 int64 exgcd(int64 a, int64 b, int64 &x, int64 &y) {
8     // ===== begin: 请补全扩展欧几里得算法 =====
9     // 目标: 返回 gcd(a,b), 并通过引用参数 x, y 给出一组解
10    // 提示:

```

测试结果

7/7 全部通过

测试集1	消耗内存146.17MB	代码执行时长: 0.05秒	✓
测试集2	消耗内存146.17MB	代码执行时长: 0.03秒	✓
测试集3	消耗内存146.17MB	代码执行时长: 0.03秒	✓
测试集4	消耗内存146.17MB	代码执行时长: 0.05秒	✓
测试集5	消耗内存146.17MB	代码执行时长: 0.04秒	✓
测试集6	消耗内存146.17MB	代码执行时长: 0.03秒	✓
测试集7	消耗内存146.17MB	代码执行时长: 0.03秒	✓

说点什么

👍

本关最大执行时间: 20秒

本次评测耗时

上一关

下一关

自测运行

(10) 密码学——RSA 加密:

程序设计思想:

首先读取明文 M , 公钥指数 e 和模数 n 三个长整型参数, 定义结果变量 $result$ 并初始化为1, 用于累乘存储最终加密结果。同时将底数 $base$ 初始化为 $M \% n$, 确保底数在模 n 意义下进行运算, 避免初始值过大导致溢出。

接着采用快速幂算法迭代计算模幂运算, 当指数 $zhishu$ 大于 0 时循环执行: 若当前指数的二进制最低位为 1 (即 $zhishu \& 1$ 为真), 则将当前 $base$ 乘入结果 $result$ 并立即对 n 取模, 核心代码为 $result = (result * base) \% n$, 该步骤确保每次乘法后数值始终控制在模数范围内。随后将 $base$ 平方并取模更新为下一轮的值, 即 $base = (base * base) \% n$, 实现底数的二进制倍增。最后将指数右移一

空间复杂度: $O(1)$ ，仅使用了常数个长整型变量存储底数、指数、模数、结果及中间状态，不依赖输入规模的额外空间开销，内存占用始终保持恒定。

程序设计思想：本题主要应用了组合数学中的乘法法则和加法法则来完成合法用户名总数的计算。由于题目规定用户名有两种不同的组成方式，并且两种方式互不重叠，因此可以先分别计算每一种规则对应的用户名数量，最后再利用加法法则得到最终结果。

程序中首先定义了 *calculateTotalUsernames()* 函数，将所有计算过程封装在该函数中，使主函数只负责输出结果，结构更加清晰，也便于后续修改和扩展。

对于第一种用户名规则，要求用户名由4个小写英文字母组成。根据乘法法则，用户名的四个位置需要依次确定，每一个位置都可以从26个小写字母中任选一个，并且各个位置之间互不影响，因此总的组合数量为：

$$26 \times 26 \times 26 \times 26 = 26^4$$

在程序中，使用 *pow(26,4)* 计算这一结果，并将返回值保存到变量 *power_rule1* 中。由于最终结果需要以整数形式参与计算，因此使用 *longlong* 类型进行存储，能够保证数据范围足够大，避免整数溢出。

对于第二种用户名规则，要求用户名由 1 个大写字母和 2 个数字组成。构造用户名需要分为三个步骤：首先确定首位的大写字母，共有 26 种选择；然后确定第二位数字，共有 10 种选择；最后确定第三位数字，同样有 10 种选择。根据乘法法则，这种用户名的总数量为：

$$26 \times 10 \times 10$$

程序中直接使用表达式 $26 * 10 * 10$ 进行计算，并将结果保存到变量 *power_rule2* 中。

由于题目规定用户名只能满足规则一或规则二，两种规则之间不存在重复情况，因此满足加法法则的应用条件。程序最后通过

```
return power_rule1 + power_rule2;
```

将两种规则对应的数量相加，并返回所有合法用户名的总数。

时间复杂度分析： $O(1)$ ，整个程序没有使用循环或条件判断，而是直接依据组合数学公式进行计算，算法思路简单清晰。

彭婉茹
 55135

计数基础

实验总用时: 00:06:12

更多操作

第1关: 网站用户名

100

学习内容

记录

评论

要么是规则一，要么是规则二，因此总数是两种规则下的数量之和。

乘法法则（分步相乘）

如果完成一件事情需要分成 n 个步骤，且这些步骤缺一不可，做第一步有 m_1 种不同的方法，做第二步有 m_2 种不同的方法，……，做第 n 步有 m_n 种不同的方法，那么完成这件事情总共有 $N = m_1 \times m_2 \times \dots \times m_n$ 种不同的方式。

核心思想：分步完成，各步方法数相乘。在本题中，构造一个具体用户名需要多个步骤，比如规则一需要确定4个位置的字母，每个位置都是一步。

编程要求

输入

- 本任务无输入。

输出

- 向标准输出打印一个整数，表示所有合法的用户名总数。

开始你的任务吧，祝你成功！

代码文件

```

1 #include <iostream>
2 #include <cmath>
3
4 using namespace std;
5
6 long long calculateTotalUsernames() {
7     /***** Begin *****/
8     long long power_rule1 = pow(26, 4);
9     long long power_rule2 = 26 * 10 * 10;
10    return power_rule1 + power_rule2;
11    /***** End *****/
12 }
13
14
15 int main() {
16     cout << calculateTotalUsernames() << endl;
17     return 0;
18 }

```

测试结果

1/1 全部通过

测试集1
 消耗内存366.25MB
 代码执行时长: 0.02秒

说点什么

本关最大执行时间: 20秒
 本次评测耗时(编译)
 下一关
 自测运行

（2）计数基础——相邻的约束

程序设计思想：本题要求计算满足相邻数字限制条件的密码总数。如果直接按照乘法原理进行计算，由于每一位数字的选择都会受到前一位数字的影响，因此各位置之间已经不再独立，无法简单地利用乘法法则直接求解。为了有效处理这种相邻元素之间存在约束的问题，本程序采用动态规划的方法进行求解，将复杂问题分解为多个规模较小的子问题，逐步计算最终结果。

程序首先定义了一个二维数组 `bad[10][10]` 来记录所有“不吉利”的数字对。由于密码中的数字只有0~9共10个，因此可以将`bad[u][v]`置为1，表示数字`u`后面不能紧跟数字`v`。这种二维数组的存储方式查询速度快，在后续状态转移过程中，只需要一次数组访问即可判断当前相邻数字是否合法。

随后程序建立二维动态规划数组 $dp[1001][10]$ 。其中， $dp[i][j]$ 表示构造长度为 i 的合法密码，并且最后一位数字为 j 时的方案总数。这样定义状态后，整个密码是否合法只需要关注最后两个数字之间的关系，而不需要重新检查整个密码，提高了计算效率。

对于动态规划的初始化，当密码长度为1时，每个数字都可以单独作为一个合法密码，因此程序利用循环将

```
dp[1][j] = 1;
```

初始化为1，表示长度为1且最后一位为数字 j 的密码各有一种方案。

完成初始化后，程序开始进行状态转移。外层循环依次枚举密码长度 i ，第二层循环枚举当前密码最后一位数字 $last$ ，第三层循环枚举长度为 $i - 1$ 时最后一位数字 pre 。如果 pre 与 $last$ 之间不是不吉利数字对，即满足

```
if(!bad[pre][last]);
```

说明可以在原来的合法密码后面继续添加数字 $last$ ，于是将所有以 pre 结尾的方案数累加到 $dp[i][last]$ 中，即：

```
dp[i][last] = (dp[i][last] + dp[i-1][pre]) % MOD;
```

由于题目要求最终结果对 $10^9 + 7$ 取模，因此程序在每次状态更新时都进行取模运算，不仅满足题目要求，也避免了方案数过大导致整数溢出。

当所有状态计算完成后，程序再遍历最后一位数字 $0 \sim 9$ ，将所有 $dp[n][j]$ 累加即可得到长度为 N 的所有合法密码数量。因为长度为 N 的密码可能以任意数字结尾，所以最终答案是所有状态的总和。

时间复杂度分析： $O(N \times 10 \times 10)$ ，整个算法充分利用了动态规划“保存已计算结果”的思想，每个状态只计算一次，不需要重复统计相同的子问题。由于密码长度最大为 1000，而每一层只需要枚举10个数字以及前一个位置的10个数字，因此算法的时间复杂度为 $O(N \times 10 \times 10)$ 。

彭婉茹

55135

计数基础

实验总用时: 00:40:10

更多操作

电源

第2关: 相邻的约束

100

学习内容

记录

评论

- 第二行是一个整数K。
- 接下来K行，每行两个数字 u 和 v ，由空格隔开，表示 (u, v) 是一对“不吉利”的相邻数字对。

预期输出格式:

- 一个整数，表示合法的密码总数，结果对 $10^9 + 7$ 取模。

约束条件:

- $1 \leq N \leq 1000$
- $0 \leq K \leq 100$
- $0 \leq u, v \leq 9$

测试用例1:

测试输入:

```

1. 3
2. 1
3. 1 3

```

预期输出:

```

1. 980

```

开始你的任务吧，祝你成功!

说点什么

👍

代码文件

```

1 #include <iostream>
2 #include <vector>
3 #include <stdio.h>
4
5 #define MOD 1000000007
6 using namespace std;
7
8 int main() {
9     int n, k;
10
11     cin >> n;
12     cin >> k;
13

```

测试结果

8/8 全部通过

▶ 测试集1	🔒	消耗内存37.97MB	代码执行时长: 0.04秒	✅
▶ 测试集2	🔒	消耗内存37.97MB	代码执行时长: 0.03秒	✅
▶ 测试集3	🔒	消耗内存37.97MB	代码执行时长: 0.02秒	✅
▶ 测试集4	🔒	消耗内存37.97MB	代码执行时长: 0.02秒	✅
▶ 测试集5	🔒	消耗内存37.97MB	代码执行时长: 0.02秒	✅
▶ 测试集6	🔒	消耗内存37.97MB	代码执行时长: 0.02秒	✅
▶ 测试集7	🔒	消耗内存37.97MB	代码执行时长: 0.02秒	✅
▶ 测试集8	🔒	消耗内存37.97MB	代码执行时长: 0.02秒	✅

本关最大执行时间: 20秒

本次评测耗时(编译)

上一关

自测运行

(3) 巢鸽原理——采摘果实

程序设计思想：本题主要应用了广义鸽巢原理来解决实际计数问题。题目要求计算为了保证采摘的果实中至少有 M 个属于同一种类，最少需要采摘多少个果实。由于要求的是“保证”满足条件，因此程序采用了广义鸽巢原理中常用的最坏情况分析方法，而不是逐种情况进行模拟计算。

程序首先从标准输入读取果树种类数 k 和目标数量 m 。其中， k 表示共有多少种不同的果树， m 表示希望保证至少有多少个果实属于同一种类。

求解过程中，程序重点考虑的是始终没有出现某一种果实达到 m 个时能够采摘的最多果实数量。为了尽可能延迟这一情况的发生，需要让每一种果实的数量尽量保持均衡。因此，在最坏情况下，每一种果实最多只能采摘 $m - 1$ 个，否则

某一种果实就已经达到 m 个，不再满足“尚未出现”的条件。

由于共有 k 种果树，每一种最多采摘 $m - 1$ 个，因此在仍然没有任何一种果实达到 m 个时，最多能够采摘的果实数量为：

$$k \times (m - 1)$$

当继续再采摘 1 个果实时，无论采摘到哪一种果实，都必然会使其中一种果实的数量达到 m 个，从而满足题目要求。根据广义鸽巢原理，可以得到最少需要采摘的果实数量为：

$$k \times (m - 1) + 1$$

程序中直接利用这一数学公式完成计算，并输出最终结果：

```
cout << k * (m - 1) + 1;
```

时间复杂度分析： $O(1)$ ，由于整个计算过程只涉及一次乘法、一次减法和一次加法，没有使用循环、递归或其他复杂的数据结构，因此时间复杂度为 $O(1)$ 。

三

第1关：采摘果实

100

学习内容

记录

评论

- 任务描述
- 相关知识
 - 广义鸽巢原理
- 算法原理
- 编程要求
- 测试说明

任务描述

本关任务：应用**广义鸽巢原理**解决一个问题。

题目背景：果园里有 K 种果树，每种果树上的果实都取之不尽。为了保证你采摘的果实中，至少有 M 个是属于同一种类的，你最少需要采摘多少个果实？

相关知识

为了完成本关任务，你需要掌握**广义鸽巢原理**。

广义鸽巢原理

如果将 N 个物体放入 K 个盒子中，那么至少有一个盒子包含至少 $\lceil N/K \rceil$ 个物体。

说点什么

代码文件

```

1  #include <iostream>
2
3  using namespace std;
4
5  int main() {
6      int k, m;
7      cin >> k >> m;
8
9      /***** Begin *****/
10     cout << k * (m - 1) + 1;
11
12     /***** End *****/
13
14     return 0;
15 }

```

测试结果

6/6 全部通过

▶ 测试集1	🔒	消耗内存34.14MB	代码执行时长：0.05秒	✓
▶ 测试集2	🔒	消耗内存34.14MB	代码执行时长：0.04秒	✓
▶ 测试集3	🔒	消耗内存34.14MB	代码执行时长：0.05秒	✓
▶ 测试集4	🔒	消耗内存34.14MB	代码执行时长：0.03秒	✓
▶ 测试集5	🔒	消耗内存34.14MB	代码执行时长：0.05秒	✓
▶ 测试集6	🔒	消耗内存34.14MB	代码执行时长：0.03秒	✓

本关最大执行时间：20秒

本次评测耗时(编译)

下一关

自测运行

(4) 巢鸽原理——子集和

程序设计思想：本题要求在一个包含 N 个正整数的数组中，找出一个非空子集，使其元素之和能够被 N 整除。由于题目保证一定存在解，因此程序的关键在于如何快速找到满足条件的子集。若采用枚举所有子集的方法，时间复杂度将达到 $O(2^N)$ ，显然无法满足 N 最大可达100000的数据规模。因此，本程序利用前缀和、模运算以及鸽巢原理相结合的方法，将问题转化为寻找两个具有相同余数的前缀和，从而在线性时间内完成求解。

程序首先读入数组长度 n 和所有元素，并使用`vector < longlong > a`存储数据。由于数组元素最大可达到 10^9 ，多个元素相加后可能超过`int`的表示范围，因此程序采用`longlong`类型保存数组元素和前缀和，保证计算过程不会发生整数溢出。

随后，程序定义数组 pos 来记录每一种余数第一次出现的位置，其中 $pos[i]$ 表示余数 i 首次出现的前缀和下标。开始时将整个数组初始化为 -1 ，表示当前所有余数都尚未出现。同时定义变量 sum 保存当前的前缀和。

程序随后依次遍历整个数组，在每次循环中先将当前元素加入前缀和：

```
sum += a[i];
```

然后计算当前前缀和除以 n 后的余数：

```
int r = sum % n;
```

接下来根据余数情况进行分类讨论。

第一种情况是当前余数等于 0 ，即 $r == 0$ 。这说明当前前缀和本身就是 n 的整数倍，因此从数组第一个元素到当前位置所有元素之和已经满足题目要求。程序直接输出这一段连续子集的长度及其所有元素，并结束程序。

第二种情况是当前余数此前已经出现过，即 $pos[r] \neq -1$ 。根据模运算的性质，如果两个前缀和除以 n 的余数相同，那么它们的差一定能够被 n 整除。设第一次出现该余数的位置为 $pos[r]$ ，当前位置为 i ，则数组中从 $pos[r] + 1$ 到 i 之间所有元素之和就是：

$$S_i - S_{pos[r]}$$

由于两者余数相同，因此该差值一定是 n 的倍数，所以这一连续子集就是题目要求的解。程序计算该区间长度后，将对应元素依次输出，并立即结束运行。

如果当前余数既不是 0 ，也没有出现过，则说明这是该余数第一次出现。程序将当前位置记录到数组 pos 中：

```
pos[r] = i;
```

方便后续继续判断是否存在相同余数。

整个算法的正确性来源于鸽巢原理。前缀和共有 N 个，而对 N 取模后的余数最多只有 N 种。如果某个余数为 0 ，则直接得到一个满足条件的前缀；否则所有余数只能分布在 $1 \sim N - 1$ 共 $N - 1$ 个取值中，根据鸽巢原理，必然有两个前缀和具有相同余数，从而一定能够构造出一个和能够被 N 整除的连续子集，因此题目保证一定存在解。

时间复杂度分析： $O(n)$ ，整个程序只需要遍历数组一次，每个元素仅进行常数运算，因此算法的时间复杂度为 $O(n)$

彭婉茹

55135

鸽巢原理

实验总用时: 00:04:58

更多操作

电源

第2关: 子集和

100

学习内容

记录

评论

- 任务描述
- 相关知识
 - 鸽巢原理
- 算法原理
- 编程要求
- 测试说明

任务描述

本关任务：利用**鸽巢原理**和**前缀和**技巧，解决“子集和”。

题目背景：给定一个包含 N 个正整数的数组 A 。请找出一个非空子集，使得该子集中所有元素的和能被 N 整除。如果存在多个解，输出任意一个即可。题目保证有解。

相关知识

为了完成本关任务，你需要掌握：

- 前缀和：**一种快速计算数组区间和的技巧。
- 模运算：**处理同余问题的基础。
- 鸽巢原理：**一个简单而强大的组合数学原理。

代码文件

```

1 #include <iostream>
2 #include <vector>
3
4 using namespace std;
5
6 int main() {
7     int n;
8     cin >> n;
9
10    vector<long long> a(n);
11    for(int i = 0; i < n; i++) {
12        cin >> a[i];
13    }
14
15    /******Begin***** */
16
17    vector<int> pos(n, -1);
18    long long sum = 0;

```

测试结果

8/8 全部通过

▶ 测试集1	消耗内存42.24MB	代码执行时长: 0.17秒	✓
▶ 测试集2	消耗内存42.24MB	代码执行时长: 0.03秒	✓
▶ 测试集3	消耗内存42.24MB	代码执行时长: 0.06秒	✓
▶ 测试集4	消耗内存42.24MB	代码执行时长: 0.02秒	✓
▶ 测试集5	消耗内存42.24MB	代码执行时长: 0.04秒	✓
▶ 测试集6	消耗内存42.24MB	代码执行时长: 0.03秒	✓
▶ 测试集7	消耗内存42.24MB	代码执行时长: 0.03秒	✓

本关最大执行时间: 20秒

本次评测耗时:

上一关

下一关

自测运行

（5） 巢鸽原理——平面上的整中点

程序设计思想：本题要求在平面上给定的若干整数坐标点中，找出一对点，使它们的中点坐标仍然是整数。由于题目保证点的数量不少于 5，因此可以利用**鸽巢原理**快速找到满足条件的一对点，而不需要枚举所有点对进行判断。若采用两层循环枚举任意两点，则时间复杂度为 $O(N^2)$ ，虽然在本题数据范围内可以完成计算，但没有充分体现鸽巢原理的应用。因此，本程序根据坐标的奇偶性进行分类，在一次遍历过程中即可找到满足条件的点对。

根据中点公式，设两点坐标分别为 (x_1, y_1) 和 (x_2, y_2) ，其中点坐标为：

$$\left(\frac{x_1 + x_2}{2}, \frac{y_1 + y_2}{2}\right)$$

若中点坐标均为整数，则要求 $x_1 + x_2$ 和 $y_1 + y_2$ 都是偶数。而两个整数相加为偶数，当且仅当它们具有相同的奇偶性。因此，只要两点的 x 坐标奇偶性相同，并且 y 坐标奇偶性也相同，它们的中点一定是整数点。

程序首先读入所有点的坐标，并利用 `vector < pair < int, int > >` 保存每个点的位置。随后建立一个长度为4的数组 `pos`，用于记录四种奇偶性组合第一次出现的位置。由于横坐标和纵坐标各只有奇、偶两种情况，因此共有四种组合：

- ✧ 偶，偶
- ✧ 偶，奇
- ✧ 奇，偶
- ✧ 奇，奇

程序分别计算当前点横纵坐标的奇偶性：

```
int x = abs(points[i].first) % 2;  
int y = abs(points[i].second) % 2;
```

这里使用 `abs()` 函数是为了保证负数坐标也能够正确计算奇偶性。例如 3 与 -3 都属于奇数，-4 与 4 都属于偶数，因此先取绝对值再求余更加直观可靠。随后程序利用

```
sum += a[i];
```

然后计算当前前缀和除以 n 后的余数：

```
int id = x * 2 + y;
```

将四种奇偶组合分别映射为编号 0~3。这样，每个点都能够快速确定属于哪一类别，便于后续查找。

在遍历过程中，程序首先判断当前类别是否已经出现过：

```
if(pos[id] != -1)
```

如果该类别已经存在一个点，则说明当前点与之前记录的点具有完全相同的横纵坐标奇偶性，因此它们的中点坐标一定都是整数。程序立即输出这两个点，并结束运行。

如果当前类别尚未出现，则说明这是该类别第一次出现，程序将当前点的下标保存下来：

```
pos[id] = i;
```

以便后续继续进行比较。

整个算法的正确性来源于鸽巢原理。由于平面整数点按照横纵坐标奇偶性最

多只能分为 4 类，而题目保证输入点数满足 $N \geq 5$ ，即有 5 个“鸽子”放入 4 个“鸽巢”中，因此根据鸽巢原理，至少有两个点属于同一类别。这两个点的横坐标奇偶性相同，纵坐标奇偶性也相同，因此它们连线的中点一定仍然是整数点，从而保证程序一定能够找到满足条件的一组解。

时间复杂度分析： $O(n)$ ，整个程序仅需遍历所有点一次，每个点只进行常数次运算，因此算法的时间复杂度为 $O(n)$

彭婉茹55135

第3关：平面上的整中点100

学习内容记录评论

输出

输出两行。每行包含一个点的坐标，表示你找到的满足条件的一对点中的一个。

测试说明

约束条件：

- $5 \leq N \leq 1000$
- $-1000 \leq x, y \leq 1000$

平台会对你编写的代码进行测试：

测试用例 1：

测试输入：

1. 5
2. 1 1
3. 2 3
4. 4 6
5. 5 7
6. 8 8

预期输出：

1. 1 1
2. 5 7

开始你的任务吧，祝你成功！

说点什么

鸽巢原理

实验总用时：00:03:17

代码文件

```
1 #include <iostream>
2 #include <vector>
3 #include <cmath>
4
5 using namespace std;
6
7 int main() {
8     int n;
9     cin >> n;
```

测试结果

6/6 全部通过

测试集1

消耗内存50.61MB 代码执行时长：0.06秒

测试集2

消耗内存50.61MB 代码执行时长：0.04秒

测试集3

消耗内存50.61MB 代码执行时长：0.04秒

测试集4

消耗内存50.61MB 代码执行时长：0.04秒

测试集5

消耗内存50.61MB 代码执行时长：0.06秒

测试集6

消耗内存50.61MB 代码执行时长：0.05秒

本关最大执行时间：20秒 本次评测耗时(编译)

上一关

自测运行

(6) 排列与组合——饼干选购

程序设计思想：本题要求计算从 N 种不同口味的饼干中购买 K 块饼干 的不同购买方案，并且每种口味的数量无限、购买顺序不作区分。这类问题属于组合数学中的可重复组合问题。如果采用枚举所有购买方案的方法，随着 N 和 K 的

增加，方案数量会迅速增长，计算效率较低。因此，本程序利用组合数学中的隔板法，将实际问题转化为组合数计算，从而直接得到最终结果。

根据题意，每一种口味的饼干都可以购买任意数量，因此可以把购买的 K 块饼干看作 K 个完全相同的元素（星号），再使用 $N - 1$ 个隔板将这些饼干划分成 N 个部分，每一部分对应一种口味购买的数量。这样，每一种隔板的摆放方式都唯一对应一种购买方案，因此问题就转化为在总共

$$N + K - 1$$

个位置中选择 K 个位置放置饼干（或者选择 $N - 1$ 个位置放置隔板）的组合问题，即：

$$C(N + K - 1, K)$$

程序首先定义了一个计算组合数的函数 $C(intn, intk)$ ，用于计算二项式系数。由于组合数满足

$$C(n, k) = C(n, n - k)$$

因此程序首先判断：

```
if(k > n - k)
    k = n - k;
```

将较大的 k 转换为较小的 $n - k$ ，减少循环次数，提高计算效率。

随后，程序利用组合数的递推计算方式完成计算，而没有直接使用阶乘公式。这是因为直接计算

$$\frac{n!}{k!(n - k)!}$$

容易导致中间结果过大，即使最终结果能够表示，中间计算过程也可能发生整数溢出。因此程序采用边乘边除的方法：

```
ans = ans * (n - k + i) / i;
```

即可得到所有不同购买方案的数量，并输出最终结果。

整个程序充分利用了可重复组合的数学模型，将实际问题直接转换为组合数计算，避免了复杂的枚举过程，程序结构简单明了，计算过程清晰易懂。由于题目中 N 和 K 的最大值均为6，因此计算规模很小，但程序采用了通用的组合数计算方法，具有较好的可扩展性。

时间复杂度分析： $O(1)$ 该算法主要由组合数函数完成计算，其中循环次数最多为 $\min(k, n - k)$ 次

彭婉茹

55135

排列与组合

实验总用时: 00:00:47

更多操作

⏻

第1关: 饼干选购

100

学习内容

记录

评论

从输入中读入两个正整数 N 和 K , 以空格分隔。

输出

- 向标准输出打印一个整数, 表示不同的购买方案总数。

测试说明

约束条件:

- $1 \leq N, K \leq 6$

平台会对你编写的代码进行测试:

测试用例 1:

- 测试输入:**

```
1. 4 6
```
- 预期输出:**

```
1. 84
```
- 说明:** 从 4 种口味中可重复地选择 6 块饼干, 方案数为: $C(4 + 6 - 1, 6) = C(9, 6) = C(9, 3) = \frac{9 \times 8 \times 7}{3 \times 2 \times 1} = 84$ 。

开始你的任务吧, 祝你成功!

代码文件

```

1 #include <iostream>
2
3 using namespace std;
4
5 // 计算组合数 C(n,k)
6 long long C(int n, int k)
7 {
8     if(k > n) return 0;
9     if(k > n - k) k = n - k;
10
11     long long ans = 1;
12     for(int i = 1; i <= k; i++)
13     {
14         ans = ans * (n - k + i) / i;
15     }
16     return ans;
17 }
```

测试结果

8/8 全部通过

▶ 测试集1	消耗内存62.92MB	代码执行时长: 0.11秒	✓
▶ 测试集2	消耗内存62.92MB	代码执行时长: 0.03秒	✓
▶ 测试集3	消耗内存62.92MB	代码执行时长: 0.06秒	✓
▶ 测试集4	消耗内存62.92MB	代码执行时长: 0.04秒	✓
▶ 测试集5	消耗内存62.92MB	代码执行时长: 0.06秒	✓
▶ 测试集6	消耗内存62.92MB	代码执行时长: 0.02秒	✓
▶ 测试集7	消耗内存62.92MB	代码执行时长: 0.06秒	✓
▶ 测试集8	消耗内存62.92MB	代码执行时长: 0.03秒	✓

本关最大执行时间: 20秒

本次评测耗时(编译)

下一关

自测运行

(7) 排列与组合——单词重排

程序设计思想: 本题要求计算一个可能包含重复字母的字符串能够重新排列成多少个不同的新字符串。由于字符串中可能存在多个相同字符, 因此不能直接使用全排列公式进行计算, 否则会产生大量重复计数。根据组合数学中带有重复元素的排列(多重集排列)公式, 可以在计算全排列数量的基础上, 除去相同字符之间交换位置所产生的重复排列, 从而得到最终结果。

程序首先读入输入字符串, 并定义 `map < char, int > cnt` 来统计每个字母出现的次数。`map` 的键为字符, 值为该字符出现的次数。随后利用范围 `for` 循环遍历整个字符串, 对每个字符进行计数:

```
for(char c : s)
```

```
cnt[c]++;
```

经过统计后，程序能够得到字符串中每一种字母对应的出现次数。例如对于字符串"*SUCCESS*"，统计结果为：字母*S*出现3次，*C*出现2次，*U*和*E*各出现1次。这些数据正是后续计算排列数所需要的信息。

为了方便计算，程序定义了一个*factorial()*函数，用于计算一个整数的阶乘。函数采用循环累乘的方法，从2开始依次相乘直到*n*，最终返回*n!*的值。由于题目规定字符串长度不超过10，因此最大的阶乘仅为*10! = 3628800*，使用*longlong*类型即可安全存储计算结果，不会发生整数溢出。

在完成字符统计后，程序首先计算字符串长度*n*的全排列数量：

```
long long ans = factorial(s.length());
```

这一步得到的是假设所有字符都互不重复时的排列总数，即*n!*。

随后，程序遍历*map*中保存的所有字符及其出现次数。对于每一种重复字符，根据多重集排列公式，需要除以该字符重复次数的阶乘，以消除交换相同字符位置所产生的重复排列，因此程序执行：

```
for(auto p : cnt)
    ans /= factorial(p.second);
```

例如，当某个字符出现3次时，需要除以3!；若出现2次，则除以2!；若仅出现1次，则除以1!，计算结果不会发生变化。最终得到的结果正好对应公式：

$$\frac{n!}{n_1! \times n_2! \times \cdots \times n_k!}$$

其中，*n*为字符串总长度，*n*₁、*n*₂、...、*n*_k分别表示每一种不同字符出现的次数。

完成所有重复字符的修正后，程序直接输出变量*ans*，即为所有不同排列的数量。

时间复杂度分析：*O(n)* 整个程序充分利用了组合数学中多重集排列的计算公式，将原本复杂的排列统计问题转化为简单的字符计数和阶乘运算，避免了生成所有排列再去重所带来的巨大计算开销。由于字符串长度最大仅为10，字符统计只需遍历一次字符串，而阶乘计算和遍历字符种类的规模都很小，因此算法的时间复杂度为 *O(n)*。

彭婉茹55135

排列与组合
实验总用时: 00:00:42

更多操作

第2关: 单词重排100

学习内容记录评论

测试说明

约束条件:

- 字符串长度介于 1 和 10 之间。

平台会对你编写的代码进行测试:

测试用例 1:

- 测试输入:

1. SUCCESS

- 预期输出:

1. 420

- 说明:

 - 字符串 "SUCCESS" 总长度 $n = 7$ 。
 - 字母统计: S 出现 3 次 ($n_1 = 3$), U 出现 1 次 ($n_2 = 1$), C 出现 2 次 ($n_3 = 2$), E 出现 1 次 ($n_4 = 1$)。
 - 根据公式计算:

$$\frac{7!}{3! \cdot 1! \cdot 2! \cdot 1!} = \frac{5040}{6 \times 1 \times 2 \times 1}$$

开始你的任务吧, 祝你成功!

说点什么

代码文件

```
1 #include <iostream>
2 #include <string>
3 #include <vector>
4 #include <map>
5
6 using namespace std;
7
8 long long factorial(int n)
9 {
10     long long res = 1;
11     for(int i = 2; i <= n; i++)
12         res *= i;
13     return res;
14 }
```

测试结果

6/6 全部通过

▶ 测试集1	消耗内存52.49MB	代码执行时长: 0.06秒	✓
▶ 测试集2	消耗内存52.49MB	代码执行时长: 0.04秒	✓
▶ 测试集3	消耗内存52.49MB	代码执行时长: 0.04秒	✓
▶ 测试集4	消耗内存52.49MB	代码执行时长: 0.04秒	✓
▶ 测试集5	消耗内存52.49MB	代码执行时长: 0.04秒	✓
▶ 测试集6	消耗内存52.49MB	代码执行时长: 0.05秒	✓

本关最大执行时间: 20秒 本次评测耗时(编译)

上一关

自测运行

（8）生成函数——游戏

程序设计思想：本题要求统计两个人分别取出一块石头后，所有可能的魔力值之和以及对应的组合数量。如果直接枚举每一种魔力值进行统计，不容易体现组合数学中的思想。因此，本程序利用母函数的建模方法，将两组石头分别表示成两个多项式，通过多项式相乘求得结果多项式，从而得到每一种魔力值之和对应的组合数。

程序首先读入两个人袋子中的石头魔力值，并分别存放到 a_tones 和 b_tones 两个数组中。由于单个石头的魔力值范围为 $0 \sim 500$ ，程序进一步建立两个长度为 501 的数组 a 和 b ，用于保存母函数中各项的系数。其中，下标表示魔力值，数组中的元素表示具有该魔力值的石头数量。例如，当魔力值为 3 的石头有

两块时，就有：

```
a[3] = 2;
```

这对应于母函数中的一项：

$$2x^3$$

程序利用遍历分别统计两个人各种魔力值出现的次数：

```
for(int x : a_stones) a[x]++;  
for(int x : b_stones) b[x]++;
```

跳过这些无效项，减少了不必要的运算，提高了程序效率。

对于每一对能够参与运算的项，根据多项式乘法法则：

$$(aixi) \times (bjxj) = aibjxi + j$$

程序执行：

```
c[i + j] += a[i] * b[j];
```

表示将所有能够得到魔力值 $i + j$ 的组合数量累加到结果数组中。其中， $a[i]$ 表示第一人魔力值为 i 的石头数量， $b[j]$ 表示第二人魔力值为 j 的石头数量，两者相乘就是这一种组合能够形成的取法数量。随着循环不断进行，所有产生相同魔力值总和的组合都会自动累加到对应位置，最终 $c[s]$ 就表示总魔力值为 s 时所有可能的组合数量。

完成多项式乘法后，程序最后遍历结果数组：

```
for(int s = 0; s <= 1000; s++)
```

只输出组合数量大于 0 的项，即输出所有实际能够形成的魔力值总和及其对应的组合数，满足题目要求按照魔力值升序输出结果。

时间复杂度分析： $O(n^2)$ 整个程序利用母函数建立组合模型，再通过普通多项式乘法完成统计，实现过程与组合数学中的理论完全对应。由于魔力值范围固定为 0~500，因此两层循环最多进行 501×501 次计算，算法的时间复杂度为 $O(n^2)$ 。

彭婉茹

55135

生成函数

实验总用时: 00:02:12

更多操作

三

第1关: 游戏

100

学习内容

记录

评论

- 任务描述
- 相关知识
- 算法原理
- 编程要求
- 测试说明

任务描述

本关任务：小明和小红是两个喜欢收集奇特石头的朋友。小明有一个袋子，里面装着 N 块石头，每块石头上都标有一个“魔力值”。小红也有一个袋子，里面装着 M 块石头，同样每块石头有自己的魔力值。

现在，他们俩要玩一个游戏：各自从自己的袋子里随机取出一块石头。一个组合的“总魔力值”等于两人取出的石头魔力值之和。

请你计算，所有可能的“总魔力值”及其出现的组合数。

例如：小明的袋子有2块石头，魔力值为 {1, 2}。小红的袋子有3块石头，魔力值为 {2, 2, 3}。

他们可能的组合有：

- (小明1, 小红2) -> 总和 3

代码文件

```

1 #include <iostream>
2 #include <vector>
3 #include <algorithm>
4
5 using namespace std;
6
7 int main() {
8     int n;
9     cin >> n;
10    vector<int> a_stones(n);
11    for(int i = 0; i < n; i++) {
12        cin >> a_stones[i];
13    }
14
15    int m;
16    cin >> m;
17    vector<int> b_stones(m);
18    for(int i = 0; i < m; i++) {
19        cin >> b_stones[i];
20    }
21
22    // ... (rest of the code)
23
24    return 0;
25 }

```

测试结果

6/6 全部通过

测试集1	消耗内存45.77MB	代码执行时长: 0.05秒	✓
测试集2	消耗内存45.77MB	代码执行时长: 0.05秒	✓
测试集3	消耗内存45.77MB	代码执行时长: 0.05秒	✓
测试集4	消耗内存45.77MB	代码执行时长: 0.04秒	✓
测试集5	消耗内存45.77MB	代码执行时长: 0.05秒	✓
测试集6	消耗内存45.77MB	代码执行时长: 0.04秒	✓

本关最大执行时间: 20秒

本次评测耗时(编译)

下一关

自测运行

（9）生成函数——特殊的字符串

程序设计思想：本题要求构造长度为 N 的字符串，字符串只能由字符 'A'、'B'、'C' 组成，并满足 'A' 出现偶数次、'B' 出现奇数次、'C' 出现次数不限。如果直接枚举所有长度为 N 的字符串，再统计其中满足条件的方案数，则需要遍历 3^N 种可能，当 N 较大时计算量会迅速增长，因此不能采用暴力枚举的方法。

根据题目的数学分析，可以利用指数型母函数推导出满足条件的方案数公式：

$$\frac{3^N - (-1)^N}{4}$$

因此，程序无需进行动态规划状态转移，只需要根据该公式进行计算即可，大大提高了算法效率。

79

程序首先定义了快速幂函数`qpow()`，用于计算大整数的幂并同时进行取模运算。由于题目要求结果对 $10^9 + 7$ 取模，而 N 最大可达到1000，如果直接循环相乘计算 3^N ，虽然也能够完成，但快速幂是一种更加通用且效率更高的方法，其核心思想是利用二进制拆分指数，将时间复杂度由 $O(N)$ 降低到 $O(\log N)$ 。

在快速幂函数中，程序不断判断当前指数是否为奇数：

```
if(b & 1)
    res = res * a % MOD;
```

若当前二进制位为 1，则将当前底数乘入结果；随后不断对底数平方：

```
a = a * a % MOD;
```

并将指数右移一位：

```
b >>= 1;
```

直到指数变为 0，最终得到：

```
long long p = qpow(3, n);
```

即计算出公式中的 3^N 。

随后，程序根据 N 的奇偶性计算公式中的另一部分 $(-1)^N$ 。由于该项只有两种可能，因此无需再次调用快速幂，而是直接利用条件判断：

```
if(n % 2 == 0)
    term = 1;
else
    term = MOD - 1;
```

当 N 为偶数时， $(-1)^N = 1$ ；当 N 为奇数时， $(-1)^N = -1$ 。由于程序始终在模 $10^9 + 7$ 的意义下计算，而模运算中不能直接保存负数，因此将 -1 表示为 $MOD - 1$ ，这样既符合模运算规则，又避免了出现负数计算带来的问题。

接下来，程序按照公式计算分子：

```
long long ans = (p - term + MOD) % MOD;
```

这里额外加上 MOD 是为了防止减法结果出现负值，再取模即可保证最终结果始终处于合法范围。

由于模运算下不能直接进行整数除法，因此程序预先定义了

```
const long long INV4 = 250000002;
```

它表示数字 4 在模 $10^9 + 7$ 意义下的乘法逆元，即满足：

$$4 \times INV4 \equiv 1(\text{mod } 10^9 + 7)$$

因此，公式中的除以 4 可以转换为乘以逆元：

```
ans = ans * INV4 % MOD;
```


最终得到满足条件的字符串数量，并输出结果。

时间复杂度分析： $O(\log N)$ 程序的主要计算开销来自快速幂算法，因此时间复杂度为 $O(\log N)$ 。

彭婉茹55135

第2关：特殊的字符串100

学习内容

记录

评论

- 向标准输出打印一个整数，表示满足条件的字符串总数，结果对 $10^9 + 7$ 取模。

测试说明

约束条件：

- $1 \leq N \leq 1000$

平台会对你编写的代码进行测试：

测试用例 1：

- 测试输入：**
1. 3
- 预期输出：**
1. 7
- 说明：** 根据母函数推导的公式，当 $N = 3$ 时，方案数为：
$$\frac{3^3 - (-1)^3}{4} = \frac{27 - (-1)}{4} = \frac{28}{4} = 7$$

这 7 种字符串分别是：(BCC, CBC, CCB, BBB, BAA, ABA, AAB)。

开始你的任务吧，祝你成功！

说点什么

生成函数

实验总用时: 00:01:46

代码文件

```
28
29     long long term;
30     if(n % 2 == 0)
31         term = 1;           // (-1)^n = 1
32     else
33         term = MOD - 1;     // (-1)^n = -1
34
35     long long ans = (p - term + MOD) % MOD;
36     ans = ans * INV4 % MOD;
37
38     cout << ans;
39
40     /***** End *****/
41
42     return 0;
43 }
```

测试结果

6/6 全部通过

▶ 测试集1	消耗内存40.11MB	代码执行时长: 0.07秒	✓
▶ 测试集2	消耗内存40.11MB	代码执行时长: 0.03秒	✓
▶ 测试集3	消耗内存40.11MB	代码执行时长: 0.05秒	✓
▶ 测试集4	消耗内存40.11MB	代码执行时长: 0.03秒	✓
▶ 测试集5	消耗内存40.11MB	代码执行时长: 0.02秒	✓
▶ 测试集6	消耗内存40.11MB	代码执行时长: 0.05秒	✓

本关最大执行时间: 20秒 本次评测耗时(编译)

上一关

自测运行

4 图论

4.1 实验目的

本实验旨在掌握图论的基本概念、基本性质及经典算法，理解图的存储方式、遍历方法、树、最短路径、最小生成树及图匹配等重要内容。通过编程实现图论相关算法，加深对图结构及算法原理的理解，提高利用图论模型分析和解决实际问题的能力，为学习奠定基础。

4.2 实验内容

4.2.1 图论

(1) 图论的相关定义——图的定义与邻接矩阵

本关任务：给定一张图的点集和边集，求该图的邻接矩阵。

(2) 图论的相关定义——子图与补图

本关任务：给定一张图的点集和边集，输出其补图与指定的导出子图。

(3) 图论的相关定义——图的同构判断

本关任务：给定两张图的点集和边集，判断二者是否同构。

(4) 图论的相关定义——图的连通性

本关任务：给定有向图 $G=\langle V, E \rangle$ ，判断其连通性。

(5) 图论的应用——可图序列判定——Havel-Hakimi 算法

本关任务：给定一个非负整数数组，判断其是否为可图序列。

(6) 图论的应用——求长度为 m 的通路个数

本关任务：给定一张简单有向图及其邻接矩阵，求长度为 m 的通路（含回路）的总数。

(7) 特殊图——欧拉图的判定

本关任务：编写一个程序，求解形如 $ax \equiv b(mod m)$ 的线性同余方程。

说明：题目保证 a 和 m 互质，因此可以直接利用乘法逆元求解

(8) 特殊图——偶图的判定

本关任务：编写一个程序，使用中国剩余定理（CRT）求解一元线性同余方程组。

（9） 树——可树序列定理

本关任务：给定一个度数序列，判断其是否可以构成一棵树

4.3 实验过程

4.3.1 图论

（1） 图论的相关定义——图的定义与邻接矩阵

程序设计思想：本题要求根据输入的顶点集合和边集合构造图的邻接矩阵。邻接矩阵是图的一种顺序存储结构，利用一个二维数组表示图中各顶点之间的连接关系。如果顶点 i 到顶点 j 存在一条有向边，则矩阵中对应位置的元素为1，否则为0。由于题目中的顶点编号固定为 $1\sim n$ ，因此可以直接建立一个 $n \times n$ 的二维数组作为邻接矩阵进行存储。

程序首先读入图的顶点数 n 和边数 m ，随后定义二维数组`adj[50][50]`用于保存邻接矩阵。由于开始时默认图中不存在任何边，因此程序利用

```
memset(adj, 0, sizeof(adj));
```

将整个矩阵初始化为 0，表示任意两个顶点之间均没有连接关系。这一步完成了邻接矩阵的初始化，为后续记录边的信息做好准备。

随后，程序利用一个循环依次读取输入的每一条有向边：

```
for (int i = 0; i < m; i++) {  
    int u, v;  
    scanf("%d %d", &u, &v);  
    adj[u - 1][v - 1] = 1;  
}
```

每次循环中，变量 u 和 v 分别表示一条有向边的起点和终点。由于题目中顶点编号从1开始，而 C 语言数组下标从0开始，因此程序在访问二维数组时，将顶点编号减去1，即使用`adj[u - 1][v - 1]`表示从顶点 u 到顶点 v 的边。当输入存在有向边 $\langle u, v \rangle$ 时，将对应位置赋值为1，表示两个顶点之间存在连接关系。如果输入中没有对应边，则该位置仍保持初始化时的0。

所有边读取完成后，二维数组`adj`已经完整记录了整个图的邻接关系。最后，程序采用两层循环依次遍历邻接矩阵中的所有元素，并按行输出：

```
for (int i = 0; i < n; i++) {  
    for (int j = 0; j < n; j++) {  
        printf("%d ", adj[i][j]);  
    }  
}
```

```
printf("\n");
}
```

输出结果中，第 i 行第 j 列的元素表示顶点 $i + 1$ 是否能够直接到达顶点 $j + 1$ 。若输出为1，则表示存在一条有向边；若输出为0，则表示不存在对应的边。

时间复杂度分析： $O(m + n^2)$ 整个程序采用邻接矩阵存储图结构，数据组织方式简单直观，能够快速判断任意两个顶点之间是否存在边，只需一次数组访问即可完成查询，时间复杂度为 $O(1)$ 。程序读取边信息需要遍历 m 条边，输出邻接矩阵需要遍历整个二维数组，因此整体时间复杂度为 $O(m + n^2)$ 。

空间复杂度分析： $O(n^2)$ ，空间复杂度主要用于存储邻接矩阵，为 $O(n^2)$ 。

The screenshot displays a learning management system interface. On the left, a sidebar contains a user profile (彭婉茹, 55135), a progress bar for '第1关: 图的定义与邻...' (100%), and tabs for '学习内容', '记录', and '评论'. The main content area is divided into three sections: '任务描述' (Task Description), '相关知识' (Related Knowledge), and '图的定义' (Graph Definition). The '任务描述' section states: '给定一张图的点集和边集，求该图的邻接矩阵。' (Given a graph's vertex set and edge set, find its adjacency matrix). The '相关知识' section explains: '为了完成本关任务，你需要掌握：图的定义，邻接矩阵。' (To complete this task, you need to master: graph definition, adjacency matrix). The '图的定义' section defines a graph $G = \langle V, E \rangle$ and lists two properties: 1. V is a finite non-empty set of nodes (Nodal Point), and 2. E is a finite set of edges (Frontier Set). Below this, the '邻接矩阵' (Adjacency Matrix) section defines $G = \langle V, E \rangle$ and states that $V = \{v_1, v_2, \dots, v_n\}$. On the right, a code editor shows the implementation of the adjacency matrix using C. The code includes `<stdio.h>` and `<string.h>` for `memset`. It defines a `main` function that takes `n` (vertices) and `m` (edges) as input, declares a `adj` matrix of size `50x50`, and initializes it to 0 using `memset`. Below the code editor, a '测试结果' (Test Results) section shows 9 test cases, all of which passed (20/20). Each test case displays memory usage (20.79MB) and execution time (ranging from 0.02s to 0.05s). At the bottom, there are buttons for '下一关' (Next Level) and '自测运行' (Self-test Run), along with a note about the maximum execution time (3s) and compilation time.

(2) 图论的相关定义——子图与补图

程序设计思想：本题要求根据输入的有向图，分别输出补图的邻接矩阵以及

指定顶点集合导出的导出子图邻接矩阵。程序主要利用邻接矩阵存储图结构，在此基础上完成补图构造和导出子图输出。由于邻接矩阵能够直观地表示任意两个顶点之间是否存在边，因此非常适合完成本题要求。

程序首先读入顶点数 n 和边数 m ，然后定义二维数组 $adj[50][50]$ 保存原图的邻接矩阵，并利用

```
memset(adj, 0, sizeof(adj));
```

将整个矩阵初始化为 0，表示初始状态下任意两个顶点之间都不存在边。随后程序依次读取输入的每一条有向边，对于每条边 $\langle u, v \rangle$ ，将对应位置赋值为 1：

```
adj[u - 1][v - 1] = 1;
```

由于输入中的顶点编号从1开始，而数组下标从0开始，因此访问数组时需要将顶点编号减去1。完成所有边的读取后，数组 adj 就完整表示了原图的邻接关系。

接下来程序构造补图的邻接矩阵。根据补图的定义，在简单有向图中，若两个不同顶点之间原图不存在有向边，则补图中存在该边；若原图已经存在该边，则补图中不再保留。同时，补图也不允许存在自环，因此主对角线元素始终为 0。

程序定义二维数组 $comp$ 存储补图邻接矩阵，并利用两层循环遍历整个矩阵：

```
for (int i = 0; i < n; i++) {
    for (int j = 0; j < n; j++) {
        if (i == j)
            comp[i][j] = 0;
        else
            comp[i][j] = 1 - adj[i][j];
    }
}
```

其中，当 $i == j$ 时表示主对角线位置，即顶点自身，不允许出现自环，因此直接赋值为0；对于其他位置，如果原图中对应位置为0，则补图中变为1；如果原图中对应位置为1，则补图中变为0。程序利用表达式 $1 - adj[i][j]$ 完成这一转换，使补图的构造过程更加简洁。

完成补图计算后，程序按照邻接矩阵格式依次输出整个 $comp$ 数组，即得到补图的邻接矩阵。

随后程序输出导出子图。题目规定导出子图的顶点集合默认为 $\{1, 2, \dots, k\}$ 。根据导出子图的定义，只需要保留这些顶点之间原来存在的边即可，而这些边已经保存在原图邻接矩阵 adj 中。因此，不需要重新建立新的邻接矩阵，只需直接

输出 adj 左上角的 $k \times k$ 部分:

```
for (int i = 0; i < k; i++) {  
    for (int j = 0; j < k; j++) {  
        printf("%d ", adj[i][j]);  
    }  
    printf("\n");  
}
```

这样得到的矩阵正好表示由顶点集合 $\{1,2,\dots,k\}$ 导出的子图邻接矩阵, 实现简单且避免了重复计算。

时间复杂度分析: $O(m + n^2)$ 整个程序利用邻接矩阵作为图的存储结构, 补图的构造仅需遍历整个矩阵一次, 导出子图直接输出对应子矩阵即可, 因此算法实现较为直接。程序读取边信息需要 $O(m)$ 的时间, 构造补图需要遍历整个邻接矩阵, 时间复杂度为 $O(n^2)$, 输出补图和导出子图同样需要遍历矩阵, 因此整个算法的时间复杂度为 $O(m+n^2)$

空间复杂度分析: $O(n^2)$, 空间复杂度主要用于存储两个邻接矩阵, 为 $O(n^2)$ 。

彭婉茹

55135

图论的相关定义

实验总用时: 00:02:45

更多操作

电源

第2关: 子图与补图

100

学习内容

记录

评论

的子图的邻接矩阵。

数据范围:

$1 \leq n \leq 50, 1 \leq m \leq n^2$

保证不存在自环

测试说明

平台会对你编写的代码进行测试:

测试输入:

3 4

1 2

2 3

3 2

3 1

2

预期输出:

0 0 1

1 0 0

0 0 0

0 1

0 0

开始你的任务吧, 祝你成功!

说点什么

代码文件

```

1 #include <stdio.h>
2 #include <string.h> // For memset
3
4 int main() {
5     // n: 顶点数, m: 边数

```

测试结果

20/20 全部通过

测试集1

消耗内存20.88MB

代码执行时长: 0.03秒

✓

测试集2

消耗内存20.88MB

代码执行时长: 0.03秒

✓

测试集3

消耗内存20.88MB

代码执行时长: 0.03秒

✓

测试集4

消耗内存20.88MB

代码执行时长: 0.02秒

✓

测试集5

消耗内存20.88MB

代码执行时长: 0.02秒

✓

测试集6

消耗内存20.88MB

代码执行时长: 0.03秒

✓

测试集7

消耗内存20.88MB

代码执行时长: 0.02秒

✓

测试集8

消耗内存20.88MB

代码执行时长: 0.02秒

✓

测试集9

消耗内存20.88MB

代码执行时长: 0.02秒

✓

测试集10

消耗内存20.88MB

代码执行时长: 0.03秒

✓

测试集11

消耗内存20.88MB

代码执行时长: 0.03秒

✓

测试集12

消耗内存20.88MB

代码执行时长: 0.02秒

✓

本关最大执行时间: 3秒

本次评测耗时: 0.03秒

上一关

下一关

自测运行

（3）图论的相关定义——图的同构判断

程序设计思想：本题要求判断两张有向图是否同构。根据图同构的定义，如果能够找到一种顶点之间的一一对应关系，使得两张图中任意两个对应顶点之间的边及其重数完全一致，则说明两张图同构；否则两张图不同构。由于图同构问题目前尚未发现适用于一般情况的高效算法，而题目中顶点数量满足 $n \leq 10$ ，规模较小，因此程序采用枚举所有顶点映射（全排列）的方法进行判断，即通过回溯算法依次生成所有可能的双射函数，并验证是否满足同构条件。

程序首先读入顶点数 n ，随后分别读入两张图的邻接矩阵，保存到二维数组 $g1$ 和 $g2$ 中。由于题目中的图允许存在重边，因此邻接矩阵中的元素不仅可能为0或1，还可能大于1，因此程序直接保存矩阵中的整数值，在后续判断过程中要

求对应位置完全相等。

为了表示顶点之间的映射关系，程序定义了一维数组 p 。其中， $p[i]$ 表示第一张图中编号为 i 的顶点映射到第二张图中的顶点编号。例如，当 $p[0] = 2$ 时，表示第一张图中的顶点1对应第二张图中的顶点3。同时，程序利用数组 $used$ 记录第二张图中的顶点是否已经参与映射，从而保证得到的是一个合法的双射关系，不会出现多个顶点映射到同一个顶点的情况。

程序的核心部分是递归函数 $gen()$ 。该函数采用深度优先搜索与回溯的方法生成所有可能的排列。当递归深度 d 表示当前已经确定了前 d 个顶点的映射关系时，程序不断尝试将尚未使用的顶点加入当前映射：

```
p[d] = i;
used[i] = true;
gen(p, used, d + 1);
used[i] = false;
```

其中，先记录当前映射关系，再递归处理下一层；当递归返回后，将当前顶点恢复为未使用状态，这就是回溯过程，使程序能够继续尝试其他排列。

当递归深度达到 n 时，说明已经生成了一个完整的顶点映射，此时程序调用 $check()$ 函数判断该映射是否满足图同构条件。

$check()$ 函数采用两层循环遍历第一张图邻接矩阵中的所有元素：

```
if (g1[i][j] != g2[p[i]][p[j]])
    return false;
```

对于第一张图中任意一条边 (i, j) ，程序根据当前映射关系找到第二张图中对应的边 $(p[i], p[j])$ ，并比较两者邻接矩阵中的值是否完全一致。如果存在任意一个位置不同，则说明当前映射不能保持图的结构，立即返回 $false$ 。只有当整个邻接矩阵完全对应时，函数才返回 $true$ ，表示找到了满足条件的同构映射。

为了减少不必要的计算，程序设置了布尔变量 iso 作为标志位。当某一次排列已经验证成功后，立即将

```
iso = true;
```

并在递归开始位置进行判断：

```
if (iso)
    return;
```

这样程序无需继续枚举剩余排列，可以提前结束搜索，提高运行效率。

时间复杂度分析： $O(n! \times n^2)$ 整个算法本质上采用了回溯法枚举所有双射函

数，再利用邻接矩阵验证映射是否保持图结构。对于 n 个顶点，共有 $n!$ 种可能的映射关系，而每一种映射都需要比较整个邻接矩阵，因此算法的时间复杂度约为 $O(n! \times n^2)$

空间复杂度分析： $O(n)$ ，空间复杂度主要来自排列数组和递归调用，为 $O(n)$ 。

图论的相关定义
实验总用时: 00:01:28

第3关：图的同构判断 100

学习内容 **记录** **评论**

注：本题中图为有向图，且可能存在重边，即邻接矩阵中的元素可能大于1。

数据范围：
 $1 \leq n \leq 10$

提示：由于图同构问题暂未发现有效算法，所以本题测试数据范围较小，可以用朴素枚举双射函数的方法解决。

测试说明

平台会对你编写的代码进行测试：

测试输入：

```
3
0 1 0
0 0 1
1 0 0
```

预期输出：

```
YES
```

开始你的任务吧，祝你成功！

代码文件

```
1  /*
2   鉴于此题代码难度较高，实现不易。
3   已经给出此题的完整做法，请仔细阅读并学习。
4   */
5
6   #include <stdio.h>
7   #include <stdbool.h> // for bool type
8
9   // n: 顶点数
10  int n;
11  // g1: 图 G 的邻接矩阵
12  int g1[10][10];
13  // g2: 图 G' 的邻接矩阵
14  int g2[10][10];
15  // iso: 标记是否同构
16  bool iso = false;
17
18  // --- 辅助函数声明 ---
19  void gen(int *p, bool *used, int d);
20  bool check(int *p);
21
22  int main() {
23      // 读取顶点数 n
24      scanf("%d", &n);
25
26      // 读取图 G 的邻接矩阵
```

测试结果

20/20 全部通过

测试集	消耗内存	代码执行时长	状态
测试集1	20.65MB	0.05秒	通过
测试集2	20.65MB	0.03秒	通过
测试集3	20.65MB	0.03秒	通过

本关最大执行时间: 3秒 本次评测耗时(编) [上一关](#) [下一关](#) [自测运行](#)

(4) 图论的相关定义——图的连通性

程序设计思想：本题要求判断一张有向图的连通性，并根据连通程度输出对应的结果。按照定义，有向图的连通性可分为强连通、单向连通、弱连通和非连通四种情况，因此程序按照由强到弱的顺序依次进行判断，一旦满足某一级别的连通性，就可以直接得到最终结果，不需要继续进行后续判断。

程序首先读入顶点数 n 和边数 m ，并利用二维数组 adj 保存图的邻接矩阵。随

后，为了提高遍历效率，又根据邻接矩阵建立了两组邻接表。其中，数组 *h*、*to*、*nxt* 用于保存原有有向图，方便后续进行可达性搜索；数组 *uh*、*uto*、*unxt* 用于保存忽略边方向后的无向图，为判断弱连通性提供数据基础。

程序首先计算图中任意两个顶点之间的可达关系，并保存在二维数组 *reach* 中。对于每一个顶点 *s*，程序都以该顶点作为起点执行一次广度优先搜索 (BFS)，搜索过程中不断访问所有能够到达的顶点：

```
reach[s][s] = 1;
q[rear++] = s;
```

每访问到一个新的顶点，就将对应位置 *reach[s][v]* 标记为 1，表示从顶点 *s* 可以到达顶点 *v*。经过 *n* 次 BFS 后，整个 *reach* 数组便构成了图的可达性矩阵，其中 *reach[i][j] = 1* 表示存在一条从顶点 *i* 到顶点 *j* 的有向路径。

完成可达性矩阵计算后，程序首先判断图是否为强连通图。根据强连通图的定义，对于任意两个不同顶点，都必须满足互相可达。因此程序利用两层循环遍历所有顶点对：

```
if (!reach[i][j] || !reach[j][i])
    strong = false;
```

只要存在一对顶点不能相互到达，就说明图不是强连通图；若所有顶点对均满足条件，则将结果直接设为 3。

如果图不是强连通图，则继续判断是否属于单向连通图。单向连通要求任意两个顶点之间至少存在一个方向可以到达，因此程序判断：

```
if (!reach[i][j] && !reach[j][i])
    unilateral = false;
```

若存在任意一对顶点既不能从 *i* 到 *j*，也不能从 *j* 到 *i*，则说明图不是单向连通图；否则可确定图属于单向连通图，结果设为 2。

若图既不是强连通图，也不是单向连通图，则程序继续判断是否属于弱连通图。根据弱连通图的定义，需要忽略所有边的方向，将原有有向图转换成无向图，再判断其是否连通。因此程序在建立邻接表时，同时建立了无向图的邻接关系，并从顶点 1 开始再次执行一次 BFS：

```
vis[1] = true;
q[rear++] = 1;
```

搜索结束后，程序检查是否所有顶点都被访问：

```
if (!vis[i])
```

```
weak = false;
```

若所有顶点均能够访问到，则说明忽略边方向后图是连通的，因此该图属于弱连通图，结果设为1；否则说明图连弱连通都不满足，最终结果为 0。

时间复杂度分析： $O(n + m)$ 整个程序采用广度优先搜索结合可达性分析的方法完成四种连通性的判断。计算可达矩阵需要从每个顶点执行一次 BFS，总时间复杂度约为 $O(n(n + m))$ ；最后一次用于判断弱连通性的 BFS 时间复杂度为 $O(n + m)$ 。

空间复杂度分析：程序主要使用邻接矩阵、邻接表以及可达性矩阵进行存储，因此空间复杂度为 $O(n^2)$ 。

图论的相关定义
实验总用时: 00:25:12

代码文件

```
1 #include <stdio.h>
2 #include <stdbool.h> // for bool type
3 #include <string.h> // for memset
4
5 // --- 建议使用的全局变量 ---
6 // n: 顶点数, m: 边数
7 int n, m;
8 // adj: 存储有向图的邻接矩阵
```

测试结果

20/20 全部通过

测试集	消耗内存	代码执行时长	状态
测试集1	21.22MB	0.09秒	通过
测试集2	21.22MB	0.05秒	通过
测试集3	21.22MB	0.05秒	通过
测试集4	21.22MB	0.05秒	通过
测试集5	21.22MB	0.05秒	通过
测试集6	21.22MB	0.08秒	通过
测试集7	21.22MB	0.07秒	通过
测试集8	21.22MB	0.07秒	通过
测试集9	21.22MB	0.07秒	通过
测试集10	21.22MB	0.08秒	通过
测试集11	21.22MB	0.07秒	通过

本关最大执行时间: 20秒 本次评测耗时(编译)

上一关 自测运行

(5) 图论的应用——可图序列判定——Havel-Hakimi 算法

程序设计思想：本题要求判断一个非负整数序列是否能够作为某个简单图的度数序列，即判断其是否为可图序列。程序采用 Havel-Hakimi 算法进行判断，该算法通过不断消去当前最大度数并更新剩余顶点的度数，最终判断序列是否能够构造出一个简单图。

程序开始首先读入序列长度 n 以及各顶点的度数，并在读入过程中计算所有度数之和 sum 。根据图论中的握手定理，任意无向图所有顶点度数之和必定为偶数，因此程序首先进行一次必要条件判断：

```
if (sum % 2 != 0) {  
    is_graphic = false;  
}
```

如果度数和为奇数，则说明该序列一定不是可图序列，可以直接输出 NO，无需继续执行后续算法。

若满足偶数条件，程序进入 Havel-Hakimi 算法的主体循环。每轮循环首先利用`qsort()`函数将当前度数序列按降序排列：

```
qsort(deg, n, sizeof(int), compare_desc);
```

这样可以保证当前最大的度数始终位于数组首部，便于后续处理。由于每轮修改度数后都会改变序列大小关系，因此需要在每次循环开始重新排序，以满足算法要求。

排序完成后，程序首先检查最大的度数是否已经为 0：

```
if (deg[0] == 0) {  
    break;  
}
```

如果最大度数已经为 0，则说明整个序列全部变为了 0，此时所有边均已成功构造完成，可以判定该序列为可图序列，循环结束。

否则，程序取出当前最大的度数 d ：

```
int d = deg[0];
```

根据 Havel-Hakimi 算法，该顶点必须与剩余度数最大的 d 个顶点相连，因此程序首先判断该度数是否合法：

```
if (d < 0 || d >= n) {  
    is_graphic = false;  
    break;  
}
```

如果当前最大度数大于等于剩余顶点数，或者出现负数，则说明不存在满足要求的简单图，程序立即结束并输出 NO。

随后，程序将当前顶点视为已经连接完成，其度数置为 0：

```
deg[0] = 0;
```

然后依次对后面 d 个度数最大的顶点执行减一操作：

```
for (int i = 1; i <= d; i++) {  
    deg[i]--;  
  
    if (deg[i] < 0) {  
        is_graphic = false;  
        break;  
    }  
}
```

这一过程对应于为当前顶点连接 d 条边，同时使这些相邻顶点剩余需要连接的边数减少 1。如果在减一过程中出现负数，说明某个顶点原本度数不足，无法满足连接要求，因此该序列不是可图序列。

程序不断重复排序—取最大度数—更新剩余度数这一过程，直到所有度数均变为 0，或者中途出现非法情况。最后根据变量`is_graphic`的取值输出判断结果：若为`true`输出 YES，否则输出 NO。

时间复杂度分析： $O(n^2 \log n)$ 。整个算法严格按照 Havel–Hakimi 定理实现，每轮排序时间复杂度为 $O(n \log n)$ ，最多进行 n 轮，因此总体时间复杂度约为 $O(n^2 \log n)$ 。

空间复杂度分析： $O(n)$ 本程序主要使用了一个长度为 100 的数组`deg`来存储当前的度数序列，同时使用了少量整型变量（如`n`、`sum`、`d`）以及布尔变量`is_graphic`来记录算法执行状态。Havel–Hakimi 算法在判断过程中直接对原数组进行修改，每次排序和度数更新都属于原地操作，没有额外申请新的数组或递归调用所需的栈空间。因此，程序所需的额外存储空间仅与输入序列长度成正比，空间复杂度为 $O(n)$ 。

彭婉茹

55135

图论的应用

实验总用时: 00:05:18

更多操作

⏻

第1关: 可图序...

100

学习内容

记录

评论

任务描述

给定一个非负整数数组, 判断其是否为可图序列。

相关知识

为了完成本关任务, 你需要掌握: 可图序列的定义与 Havel-Hakimi 算法。

可图序列的定义

- 度数序列

设图

$$G = \langle V, E \rangle$$

的结点集为

$$V = \{v_1, v_2, \dots, v_n\}$$

, 则

$$(\deg(v_1), \deg(v_2), \dots, \deg(v_n))$$

称为图

$$G$$

代码文件

```

1  #include <stdio.h>
2  #include <stdlib.h> // for qsort
3  #include <stdbool.h> // for bool type
4
5  // 比较函数, 用于 qsort 降序排序
6  int compare_desc(const void *a, const void *b) {
7      return (*(int*)b - *(int*)a);
8  }
9
10 int main() {
11     // n: 序列的当前长度
12     int n;
13     scanf("%d", &n);
14
15     // deg: 存储度数序列
16     int deg[100], sum = 0;
17     for (int i = 0; i < n; i++) {
18         scanf("%d", &deg[i]);
19         sum += deg[i];
20     }

```

测试结果

20/20 全部通过

▶ 测试集1	消耗内存20.99MB	代码执行时长: 0.03秒	✓
▶ 测试集2	消耗内存20.99MB	代码执行时长: 0.03秒	✓
▶ 测试集3	消耗内存20.99MB	代码执行时长: 0.03秒	✓
▶ 测试集4	消耗内存20.99MB	代码执行时长: 0.03秒	✓
▶ 测试集5	消耗内存20.99MB	代码执行时长: 0.02秒	✓
▶ 测试集6	消耗内存20.99MB	代码执行时长: 0.03秒	✓

本关最大执行时间: 3秒

本次评测耗时(编译...

下一关

自测运行

评测

(6) 图论的应用——长度为 m 的通路个数

程序设计思想: 本题要求统计图中所有长度为 m 的通路 (包含回路) 数量。根据图论中的相关定理, 若图的邻接矩阵为 A , 则矩阵 $A^{<\sup>m</sup>}$ 的第 (i, j) 个元素表示从顶点 i 到顶点 j 长度恰好为 m 的通路数。因此, 本题的关键在于计算邻接矩阵的 m 次幂, 最后将矩阵中所有元素累加即可得到答案。

程序首先读入图的顶点数 n , 然后读入邻接矩阵 adj , 其中 $adj[i][j] = 1$ 表示存在一条从顶点 i 指向顶点 j 的边, 否则为 0。由于矩阵幂运算过程中元素可能不断增大, 因此程序采用 *long long* 类型存储矩阵元素, 避免整数溢出。

为了计算矩阵的 m 次幂, 程序定义了两个矩阵: *res* 用于保存当前计算结果,

*tmp*用于保存一次矩阵乘法后的临时结果。程序首先利用：

```
memcpy(res, adj, sizeof(adj));
```

将邻接矩阵复制到*res*中，此时*res*即为 A^1 。

随后程序执行 $m-1$ 次矩阵乘法，每次将当前矩阵*res*与原始邻接矩阵*adj*相乘，从而依次得到 A^2 、 A^3 $A^{^m}$ 。在每轮计算开始时，首先将临时矩阵全部初始化为0：

```
memset(tmp, 0, sizeof(tmp));
```

然后采用三重循环完成矩阵乘法：

```
for (int i = 0; i < n; i++) {
    for (int k = 0; k < n; k++) {
        if (res[i][k] == 0)
            continue;

        for (int j = 0; j < n; j++) {
            tmp[i][j] += res[i][k] * adj[k][j];
        }
    }
}
```

其中，外层循环枚举行号 i ，中间循环枚举乘法中的中间顶点 k ，内层循环枚举列号 j 。按照矩阵乘法公式：

$$C_{ij} = \sum_{k=1}^n A_{ik} \times B_{kj}$$

不断累加即可得到新的矩阵元素。程序还增加了：

```
if (res[i][k] == 0)
    continue;
```

这一判断，当当前元素为 0 时直接跳过对应计算，可以减少大量无效乘法，提高程序运行效率。

完成一次矩阵乘法后，再利用

```
memcpy(res, tmp, sizeof(tmp));
```

将本轮计算得到的新矩阵复制回*res*，作为下一轮计算的基础。经过 $m-1$ 次循环后，*res*即为邻接矩阵 $A^{^m}$ 。

最后，根据图论定理，长度为 m 的所有通路数量等于矩阵 $A^{^m}$ 中全部元素之和，因此程序再次遍历整个矩阵：

```
for (int i = 0; i < n; i++) {
```

```

    for (int j = 0; j < n; j++) {
        sum += res[i][j];
    }
}

```

时间复杂度分析： $O(mn^3)$ 整个程序利用邻接矩阵幂运算实现通路计数，充分利用了矩阵乘法与图中通路数量之间的对应关系。矩阵规模为 $n \times n$ ，每次矩阵乘法时间复杂度为 $O(n^3)$ ，共进行 $m - 1$ 次乘法，因此总时间复杂度为 $O(mn^3)$ 。

空间复杂度分析：由于程序主要保存三个 $n \times n$ 的矩阵（ adj 、 res 和 tmp ），因此空间复杂度为 $O(n^2)$ 。

彭婉茹
55135

第2关：求长度为m的...
100

学习内容
记录
评论

任务描述

给定一张简单有向图及其邻接矩阵，求长度为 m 的通路（含回路）的总数。

相关知识

为了完成本关任务，你需要掌握：[通路](#)与[回路](#)的定义 以及 [通路数目求法](#)。

通路和回路的定义

对于图

$$G = \langle V, E \rangle$$

，若结点与边交替出现的序列

$$\Gamma = v_0 e_1 v_1 e_2 v_2 \dots e_k v_k$$

满足：每条边

$$e_i$$

的端点（有向图中为始点与终点）恰好是

$$v_{i-1}$$

图论的应用
实验总用时: 00:14:29

代码文件

```

1 #include <stdio.h>
2 #include <string.h> // For memcpy and
  memset
3
4 int main() {
5     // n: 结点数
6     int n;
7     scanf("%d", &n);
8
9     // adj: 邻接矩阵 A
10    long long adj[15][15];
11    for (int i = 0; i < n; i++) {
12        for (int j = 0; j < n; j++) {
13            scanf("%lld", &adj[i][j]);
14        }
15    }
16

```

测试结果

20/20 全部通过

▶ 测试集1	消耗内存20.66MB	代码执行时长: 0.05秒	✓
▶ 测试集2	消耗内存20.66MB	代码执行时长: 0.04秒	✓
▶ 测试集3	消耗内存20.66MB	代码执行时长: 0.04秒	✓
▶ 测试集4	消耗内存20.66MB	代码执行时长: 0.04秒	✓
▶ 测试集5	消耗内存20.66MB	代码执行时长: 0.03秒	✓
▶ 测试集6	消耗内存20.66MB	代码执行时长: 0.05秒	✓
▶ 测试集7	消耗内存20.66MB	代码执行时长: 0.04秒	✓

本关最大执行时间: 3秒
本次评测耗时(编译...
[上一关](#)
[自测运行](#)

(7) 特殊图——欧拉图的判定

程序设计思想：本题要求判断一张有向图是否为欧拉图；若不是欧拉图，则

进一步判断是否存在欧拉通路。根据欧拉图的判定定理，在题目保证图已经满足弱连通的前提下，只需分析各顶点的入度与出度关系即可完成判断，因此程序主要围绕各顶点的度数进行统计和分类。

程序首先读入顶点数 n 和边数 m ，随后依次读取每一条有向边 (u, v) 。对于每条边，起点 u 的出度加一，终点 v 的入度加一，因此分别利用数组 $out[]$ 和 $ins[]$ 统计所有顶点的出度和入度，为后续判定提供依据。

完成度数统计后，程序遍历所有顶点，比较每个顶点的入度和出度之间的关系，并统计满足不同条件的顶点数量。若某个顶点满足 $out = in + 1$ ，说明该顶点只能作为欧拉通路的起点，因此变量 $start$ 加一；若满足 $in = out + 1$ ，说明该顶点只能作为欧拉通路的终点，因此变量 end 加一。对于入度与出度完全相等的顶点，不需要进行额外处理，因为它们既可以作为通路中的中间顶点，也可以出现在欧拉回路中。

与此同时，程序还需要检查是否存在非法情况。如果某个顶点的入度和出度之差的绝对值大于1，则说明该顶点无法满足欧拉图或欧拉通路的构造条件，因此立即将标志变量 bad 置为 $true$ 。这种情况意味着图一定不存在欧拉通路，也不存在欧拉回路。

所有顶点检查结束后，程序根据统计结果进行最终分类判断。首先考虑特殊情况，当图中不存在任何边（即 $m = 0$ ）时，根据欧拉图定义，平凡图也属于欧拉图，因此程序直接输出结果2。

若存在非法顶点，即 $bad = true$ ，则说明图不满足欧拉图的必要条件，程序输出0。如果不存在非法情况，并且所有顶点都满足入度等于出度，即 $start = 0$ 且 $end = 0$ ，则说明每个顶点都能够实现进入一次、离开一次的平衡关系，因此图存在欧拉回路，为欧拉图，输出结果2。

如果不存在欧拉回路，但恰好只有一个顶点满足 $out = in + 1$ ，另一个顶点满足 $in = out + 1$ ，即 $start = 1$ 且 $end = 1$ ，说明图能够从起点出发，最终到达终点，存在欧拉通路但不存在欧拉回路，因此输出结果1。

除上述两种情况外，其余所有情况均不满足欧拉图或欧拉通路的判定条件，程序最终输出0，表示不存在欧拉通路。

时间复杂度分析： $O(m + n)$ 整个程序仅需遍历一次所有边统计度数，再遍历

一次所有顶点进行判断，因此时间复杂度为 $O(n + m)$ ，其中 n 为顶点数， m 为边数。

空间复杂度分析： $O(n)$.程序主要使用两个长度为 n 的数组分别保存每个顶点的入度和出度，以及少量辅助变量，因此空间复杂度为 $O(n)$ 。

彭婉茹55135

第1关：欧拉图的判定100

学习内容记录评论

- 1
：不是欧拉图，但存在欧拉通路；
- 0
：不存在欧拉通路。

数据范围：
 $1 \leq n, m \leq 10^5$

测试说明

测试输入：

```
4 4
1 2
2 3
3 4
4 1
```

预期输出：

```
2
```

开始你的任务吧，祝你成功！

特殊图

实验总用时：00:03:30

代码文件

```
1 #include <stdio.h>
2 #include <string.h> // For memset
3 #include <stdbool.h> // For bool type
4
5 // 建议使用的全局/主要变量
6 // n: 顶点数, m: 边数
7 int n, m;
8 // ins: 存储每个顶点的入度
9 // out: 存储每个顶点的出度
10 int ins[100001];
11 int out[100001];
12
13
14
15 int main() {
```

测试结果

20/20 全部通过

测试集1	消耗内存21MB	代码执行时长：0.14秒	✓
测试集2	消耗内存21MB	代码执行时长：0.14秒	✓
测试集3	消耗内存21MB	代码执行时长：0.14秒	✓
测试集4	消耗内存21MB	代码执行时长：0.17秒	✓
测试集5	消耗内存21MB	代码执行时长：0.12秒	✓
测试集6	消耗内存21MB	代码执行时长：0.1秒	✓
测试集7	消耗内存21MB	代码执行时长：0.02秒	✓
测试集8	消耗内存21MB	代码执行时长：0.05秒	✓

本关最大执行时间：3秒 本次评测耗时(编译... 下一关 自测运行

(8) 特殊图——偶图的判定

程序设计思想：本题要求判断一张无向图是否为偶图（二分图）。根据图论中的相关定理，无向图是二分图的充要条件是图中不存在奇数长度回路。因此，程序采用广度优先搜索（BFS）结合二染色法进行判定，通过给相邻顶点染不同颜色来验证是否能够完成合法划分。

程序首先读入顶点数 n 和边数 m ，并利用邻接表存储无向图。由于每条无向

边需要分别存储两个方向，因此程序调用两次`add()`函数：

```
add(u, v);  
add(v, u);
```

这样能够方便后续遍历每个顶点的所有邻接点，同时邻接表的存储方式相比邻接矩阵更加节省空间，适用于本题较大的数据规模。

随后，程序定义数组`color`记录每个顶点的染色情况。其中：`0`表示尚未染色；`1`和`2`分别表示两种不同颜色。

由于图可能由多个连通分量组成，因此程序不能只从一个顶点开始搜索，而是依次检查每个顶点：

```
for (int s = 1; s <= n && is_bipartite; s++)
```

如果当前顶点已经染色，则说明它所在连通分量已经遍历完成，直接继续处理下一个顶点；若尚未染色，则说明进入了一个新的连通分量，需要重新开始一次 BFS。

每次搜索开始时，将起始顶点染成第一种颜色：

```
color[s] = 1;  
queue[rear++] = s;
```

随后利用队列进行广度优先搜索，不断取出队首顶点，并遍历它的所有邻接点：

```
for (int i = head[u]; i != 0; i = edges[i].next)
```

对于每个相邻顶点，程序分两种情况处理。

第一种情况是该顶点尚未染色：

```
if (color[v] == 0) {  
    color[v] = 3 - color[u];  
    queue[rear++] = v;  
}
```

程序利用`3 - color[u]`在颜色`1`和颜色`2`之间切换。例如当前顶点颜色为`1`，则相邻顶点染成`2`；若当前颜色为`2`，则相邻顶点染成`1`，从而保证每条边连接的两个顶点颜色不同。

第二种情况是相邻顶点已经染色：

```
else if (color[v] == color[u]) {  
    is_bipartite = false;  
    break;  
}
```

若发现两个相邻顶点颜色相同，则说明无法满足二分图的划分要求，也就是

说图中存在奇数长度回路，因此立即将标志变量`is_bipartite`置为`false`，结束搜索。

当所有连通分量均完成搜索后，如果整个过程中没有出现颜色冲突，则说明所有相邻顶点均成功染成不同颜色，可以将所有顶点划分为两个互不相交的集合，因此该图为偶图，输出 YES；否则输出 NO。

时间复杂度分析：整个程序采用邻接表 + BFS 二染色算法完成偶图判定。由于每个顶点最多访问一次，每条边最多遍历两次，因此时间复杂度为： $O(n + m)$ 。其中 n 为顶点数， m 为边数。

空间复杂度分析：程序主要使用邻接表存储图结构、颜色数组以及 BFS 队列，这些辅助空间均与顶点数和边数成线性关系，因此空间复杂度为： $O(n + m)$ 。

彭婉茹55135

第2关：偶图的判定100

学习内容记录评论

v

, 表示无向边

(u, v)

.

若图为偶图，输出 YES；否则输出 NO。

数据范围：

$1 \leq n, m \leq 10^5$

测试说明

测试输入：

4 4
1 2
2 3
3 4
4 1

预期输出：

YES

开始你的任务吧，祝你成功！

说点什么

特殊图
实验总用时：00:04:19

代码文件

```
1 #include <stdio.h>
2 #include <string.h> // For memset
3 #include <stdbool.h> // For bool type
4
5 // --- 邻接表所需的数据结构 ---
6 // 边结构体
7 struct Edge {
8     int to;
9     int next;
10 };
11
12 // 边数组
13 struct Edge edges[200002];
14 int head[100001];
15 int e_cnt;
16
```

测试结果

20/20 全部通过

▶ 测试集1	消耗内存21.12MB	代码执行时长：0.06秒	✓
▶ 测试集2	消耗内存21.12MB	代码执行时长：0.05秒	✓
▶ 测试集3	消耗内存21.12MB	代码执行时长：0.04秒	✓
▶ 测试集4	消耗内存21.12MB	代码执行时长：0.07秒	✓
▶ 测试集5	消耗内存21.12MB	代码执行时长：0.16秒	✓
▶ 测试集6	消耗内存21.12MB	代码执行时长：0.04秒	✓
▶ 测试集7	消耗内存21.12MB	代码执行时长：0.1秒	✓

本关最大执行时间：3秒 本次评测耗时(编译... 上一关 自测运行

(9) 树——可树序列定理

程序设计思想：本实验要求判断给定的一个度数序列能否构成一棵树。根据离散数学中的可树序列定理，对于一个包含 n 个结点的树，当 $n \geq 2$ 时，其所有结点的度数之和必须满足

$$\sum_{i=1}^n d_i = 2n - 2$$

因此，本程序不需要真正构造出树的结构，而是利用该定理直接完成判断，从而降低了算法复杂度，提高了程序运行效率。

程序开始首先读取结点数 n ，随后依次输入每个结点对应的度数。在输入过程中，使用数组保存整个度数序列，同时利用变量 sum 对所有度数进行累加，以便后续进行总度数判断。由于树中除了只有一个结点的特殊情况外，不允许存在孤立结点，因此程序在遍历数组时还会检查是否存在度数为0的结点。如果发现当 $n > 1$ 时仍存在度数为0，则说明该序列不可能对应一棵树，此时利用布尔变量记录当前序列已经不能满足构成树的基本条件。

程序中对特殊情况进行了单独处理。当图中只有一个结点时，不存在任何边，因此只有当该结点度数为0时，才能认为它构成一棵树。如果唯一结点度数不是0，则直接输出 NO。这种特殊情况的判断保证了程序在所有输入规模下都能够得到正确结果。

完成输入和统计之后，程序进入最终判断阶段。首先检查前面是否已经发现非法度数；若不存在非法情况，则进一步判断所有结点度数之和是否等于 $2 * n - 2$ 。若满足该条件，则说明该序列满足可树序列定理，可以构成一棵树，程序输出 YES；否则输出 NO。

时间复杂度分析： $O(n)$ 。整个程序没有采用递归、搜索或图的构造等复杂算法，仅通过一次遍历即可完成大部分统计工作，最后依据数学定理进行判断，因此程序结构比较简单，逻辑也较为清晰。整个流程可以概括为：输入数据→统计度数总和→检查特殊情况→依据可树序列定理判断→输出结果。由于只需遍历一次度数序列，因此算法时间复杂度为 $O(n)$ 。

空间复杂度分析： $O(n)$ 。若使用数组保存度数序列；若边输入边统计，则可进一步优化到 $O(1)$ 。

彭婉茹55135

树

实验总用时: 00:05:45

更多操作

第1关: 可树序列定理100

学习内容记录评论

- 第一行给出一个整数
 n
表示点数。
- 第二行给出
 n
个非负整数, 表示度数序列。

若该序列可构成一棵树, 输出 YES; 否则输出 NO。

数据范围:

$$1 \leq n \leq 100$$

测试说明

测试输入:

```
5
2 2 2 1 1
```

预期输出:

```
YES
```

开始你的任务吧, 祝你成功!

说点什么

代码文件

```
9 // sum: 存储度数总和
10 long long sum = 0;
11 // zerodeg: 标记是否存在度为0的节点
12 bool zerodeg = false;
13
14 // 读取 n 个数, 计算总和, 并检查是否有度
  为0的节点
15 for (int i = 0; i < n; i++) {
16     int deg;
17     scanf("%d", &deg);
18     if (deg == 0) {
19         zerodeg = true;
20     }
21     sum += deg;
22 }
23
24 // can_be_tree: 标记序列是否可以构成树,
  默认为 false
25 bool can_be_tree = false;
26
27 /***** Begin *****/
28 // 在此区域根据可树序列定理进行判断
29
30 // 1. 检查度数之和是否等于 2 * (n - 1)。
31 //
32 // 2. 如果 n > 1 还需要检查序列中是否存
```

测试结果

20/20 全部通过

测试集1

消耗内存21.19MB 代码执行时长: 0.08秒

测试集2

消耗内存21.19MB 代码执行时长: 0.04秒

测试集3

消耗内存21.19MB 代码执行时长: 0.04秒

本关最大执行时间: 3秒 本次评测耗时(编译...

自测运行

评测

5 总结与课程建议

5.1 课程总结

通过本学期《离散数学》课程的学习以及实验的完成，我对离散数学的知识体系有了更加系统、深入的认识。刚开始接触这门课程时，我觉得很多概念比较抽象，例如命题逻辑、关系、偏序、图论等内容，理解起来比较困难，而且不知道这些知识在实际中有什么作用。但随着课程不断深入，尤其是在完成实验的过程中，我逐渐发现，离散数学不仅是一门理论课程，更是计算机科学的重要基础。许多程序设计中的思想、算法设计的方法以及人工智能中的知识表示，都能够从离散数学中找到理论依据。

在数理逻辑部分，我学习了命题逻辑、推理规则、逻辑等价以及谓词逻辑等内容。通过利用程序验证推理规则、生成真值表以及完成逻辑表达式的化简，我不仅理解了各种逻辑规则为什么成立，也体会到了利用计算机验证数学结论的过程。这部分实验让我认识到，很多人工智能中的知识表示、自动推理系统实际上都建立在逻辑推理的基础之上。

在集合、关系与函数部分，我进一步学习了集合运算、关系矩阵、等价关系、偏序关系、函数以及闭包等内容。课堂上这些概念容易混淆，而实验要求利用程序完成各种运算，使我能够更加直观地理解它们之间的区别。例如，通过实现 Warshall 算法计算传递闭包，我更加清楚地理解了图和关系之间的联系；通过判断函数、单射、满射和双射，也进一步理解了映射关系在数学和计算机中的实际意义。

初等数论与组合数学部分让我接触到了许多经典算法，例如扩展欧几里得算法、中国剩余定理、快速模幂、RSA 加密算法等。这些内容以前只是听说过，在实验中真正编写程序之后，才发现数学理论能够直接应用于密码学等实际问题。同时，组合数学中的计数问题、排列组合以及递推思想，也让我认识到许多算法实际上都是建立在数学分析的基础之上，而不仅仅是编写代码。

图论部分是我印象最深的一部分内容。从图的存储、遍历，到最短路径、最小生成树、拓扑排序等经典算法，每一个算法都有明确的应用场景。完成实验之后，我更加理解了为什么现实中的导航系统、网络通信、任务调度等问题都可以

抽象为图论问题。相比于单纯学习算法模板，通过实验自己实现这些算法，对算法执行过程和时间复杂度的理解更加深刻。

整个实验过程中，我最大的收获不仅仅是完成了各个程序，更重要的是学会了如何把数学问题转换成计算机能够处理的问题。很多数学定义看起来只有几句话，但真正实现时，需要先分析问题，再设计数据结构和算法，最后完成程序编写和调试。在不断修改程序、分析错误的过程中，我的编程能力和分析问题的能力都有了明显提高，也进一步体会到理论知识与实践结合的重要性。

5.2 问题及建议

总体来说，本课程内容安排合理，实验内容能够较好地帮助我们理解课堂知识。但在完成实验的过程中，我也发现了一些可以进一步改进的地方。

首先，实验题目的代码语言并没有完全统一，有的实验提供的是 C 语言，有的使用 Python，还有部分采用 C++ 实现。由于目前课程主要学习的是 C 语言 Python 和 C++ 并没有进行系统学习，因此在完成实验时，除了理解算法本身，还需要花费额外的时间学习不同语言的语法和库函数。这在一定程度上增加了实验难度，也容易让学生把精力放在语言转换上，而不是算法思想的理解上。建议后续实验能够统一采用 C 语言作为参考实现，与课程教学内容保持一致，使学生能够更加专注于算法设计和离散数学知识本身。

另外，希望今后的课程能够适当增加综合性的实验项目，将多个章节的知识结合起来。例如设计一个涉及图论、集合、关系以及逻辑推理的综合实验，让我们能够体会不同知识之间的联系，也能够进一步提高综合分析问题和解决问题的能力。